Virtualizing Data Infrastructures

by

Geoffrey X. Yu

B.S.E., University of Waterloo (2018)
M.Sc., University of Toronto (2020)

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

September 2026

Authored by: Geoffrey X. Yu
Department of Electrical Engineering and Computer Science
August 14, 2026

Certified by: Tim Kraska
Professor of Electrical Engineering and Computer Science
Thesis Supervisor

Accepted by: Leslie A. Kolodziejski
Professor of Electrical Engineering and Computer Science
Chair, Department Committee on Graduate Students

THESIS COMMITTEE

Thesis Supervisor

Tim Kraska
Professor of Electrical Engineering and Computer Science
Department of Electrical Engineering and Computer Science

Thesis Readers

Samuel Madden
MIT College of Computing Distinguished Professor
Department of Electrical Engineering and Computer Science

Ryan Marcus
Assistant Professor of Computer Science
Department of Computer and Information Science, University of Pennsylvania

Virtualizing Data Infrastructures

by

Geoffrey X. Yu

Submitted to the Department of Electrical Engineering and Computer Science
on August 14, 2026 in partial fulfillment of the requirements for the degree of

DOCTOR OF PHILOSOPHY

ABSTRACT

Organizations (e.g., corporations, educational institutions, nonprofits) usually manage their data using multiple specialized database engines (e.g., Aurora, BigQuery, PostgreSQL, Redshift, Snowflake) to take advantage of each engine's individual strengths on different parts of their workload. However, designing and managing such multi-engine infrastructures is difficult; there can be many designs, each offering different performance and a different cost. Changing the design later (e.g., due to business growth) is also challenging since application code can end up tightly coupled to the engines, making migrations risky.

This thesis proposes *data infrastructure virtualization*: a new unifying framework that addresses these challenges while also enabling new optimization opportunities. The key idea is to declare a set of *virtual database engines* (VDBEs), which specify an engine's application-facing properties (e.g., query interface and performance) and its tables, but do not prescribe a concrete engine. Crucially, each VDBE represents a distinct logical snapshot of its tables, which allows the same table to appear in multiple VDBEs serving different data use cases (e.g., a ticket sales VDBE and a sales reporting VDBE). An automated planner (or a data architect) can then map these VDBEs to one or more physical engines based on the workload as well as cost and performance constraints. Clients connect to VDBE endpoints and remain oblivious to the underlying physical engines, allowing for transparent infrastructure changes.

We first describe the VDBE abstraction, show how it can be used to declaratively express a variety of common data use cases, and discuss how to realize VDBEs using physical engines while maintaining the abstraction's data consistency and transactional isolation semantics. Next, we introduce blueprint planning: a novel holistic cost-based optimization over the infrastructure design space that automatically realizes a virtualized data infrastructure (i.e., a set of VDBEs) using physical database engines while optimizing for cost under a performance constraint or vice versa. Finally, we show how to leverage VDBEs to non-invasively integrate workload-specific query acceleration techniques into any database engine that supports data import and export. Our experimental results show that automated infrastructure design achieves 1.6×–13× lower cost than using existing cloud serverless engines, while transparently applying workload-specific query execution techniques provides geomean speedups of up to 1.38×, 1.76×, and 1.28× on three distinct workloads.

Thesis supervisor: Tim Kraska
Title: Professor of Electrical Engineering and Computer Science

For my parents.

Acknowledgments

Going into my PhD, I had heard that the experience can be like riding a roller coaster, with its many ups and downs. Knowing this, I thought I was prepared for what to expect. Now that I'm sitting here and writing this acknowledgments section, I can tell you that there is a big difference between knowing about the ride ahead and actually *experiencing* it. My PhD has been like a roller coaster in almost every way: soaring highs, deep lows, unexpected sharp turns, and anxious stretches where all you can do is hold on. But through it all, what I will remember most is just how exhilarating and intense the experience was, and how much I enjoyed the ride. I have many people to thank for helping me along the way.

First, I have to thank my advisor, Tim Kraska. Tim has a strong instinct for identifying important problems and staying at the forefront of technology. I've learned many things from working with Tim over the years. Perhaps the most important lesson is to think big and tackle research problems that are grounded in practical needs.

I also thank Sam Madden and Ryan Marcus for serving on my thesis committee. Sam has a remarkable breadth of knowledge and experience. Despite his busy schedule, he has always made time to talk with me and provide research and career advice, for which I am grateful. I very much valued Sam's feedback on the projects in this thesis.

I first met Ryan at the department's visit days back in 2020. I distinctly remember getting coffee together at Area Four, where Ryan spoke about all the exciting research projects going on in the lab. While we didn't get a chance to work together when Ryan was at MIT, I'm glad we were able to collaborate on work that formed part of this thesis. I've always been impressed by Ryan's expertise across so many layers of the computing stack, spanning everything from hardware-aware query execution primitives to neural network architectures. Ryan has also been generous with his time in providing research and career advice.

I would also like to thank M. Frans Kaashoek and Charles E. Leiserson for serving on my research qualifying examination committee. Their sharp questions and thoughtful feedback helped me clarify and improve parts of this thesis.

One of the special parts of a PhD is getting to collaborate on research with many people. I'm grateful for the opportunity to have worked with Nesime Tatbul, Dave Cohen, and Vikram Nathan, who were my internship mentors. I also thank Umar Farooq Minhas and Per-Åke Larson for their advice on my first project at MIT, as well as Laurent Bindschaedler, Çağatay Demiralp, and Andreas Kipf for their advice and mentorship over the years.

Another special and rewarding part of a PhD is getting to mentor others. I would like to thank Sophie Zhang, who worked with me as an undergraduate researcher and on her

master's thesis. I'm grateful for her work on a data connector that played an important part in making virtual database engines practical.

I've also been fortunate to get to know many students, postdocs, and visiting researchers in the Data Systems Group and at MIT: Sushrut Borkar, Peter Baile Chen, Jialin Ding, Zhuohan Gu, Wenjia He, Darryl Ho, Dominik Horn, Albert Kim, Ferdi Kossmann, Eugenie Lai, Tianyu Li, Chunwei Liu, Yuze Lou, Markos Markakis, Jason Mohoney, Oscar Moll, James Moore, Parimarjan Negi, Amadou Ngom, Aniruddha Nrusimha, Matthew Perron, Pascal Pfeil, El Kindi Rezig, Matthew Russo, Ibrahim Sabek, Sivaprasad Sudhir, Wenbo Tao, Joana M. F. da Trindade, Kapil Vaidya, Gerardo Vitagliano, Edward Wang, Weiyang Wang, Fabian Wenz, Ziniu Wu, Brit Youngmann, Anton Zabreyko, Anna Zeng, Sylvia Zhang, and Xinjing Zhou. I'm thankful to have had so many amazing colleagues and friends who made graduate school fun through informal outings, daily lunches, and countless conversations.

Before coming to MIT, I was fortunate to work with my master's advisor at the University of Toronto, Gennady Pekhimenko. Gennady took a chance on me when I had just finished my undergraduate degree and guided me through writing my first paper. I would not be where I am today without his early belief in me.

Most importantly, I would like to thank my parents Xiaojie Guo and Rui Yu for their love and support. They have always been there to talk whenever I needed advice or simply wanted to vent. I would not have been able to finish my PhD without them. This thesis is dedicated to them.

Finally, I would like to thank you, the reader, for taking the time to read my thesis. I hope you find it interesting and learn something new along the way.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Modern organizations (e.g., corporations, startups, nonprofits, educational institutions) have diverse, complex, and evolving data needs. For example, consider the business needs of a movie theater company. It might want to sell tickets online, manage its movie showing schedules across multiple theaters, process historical sales data for financial reporting, analyze customer preferences to make movie recommendations, and monitor attendance trends to optimize movie schedules. Each of these *data use cases* has different workload properties and performance needs. Ticket sales comprise short reads and writes with high concurrency. Sales analytics, on the other hand, tends to consist of complex and long-running read-only queries over large amounts of data. Recommendations and related tasks fall somewhere in between, as analysts may run exploratory queries over the data while also writing new results to the underlying database (e.g., cleaned or aggregated data).

To support all of these diverse use cases, organizations usually use multiple database systems [103]. This is because different systems excel at different kinds of workloads, support different feature sets, or have better pricing models for particular use cases (in the case of cloud databases [16, 19]). For instance, our example movie theater company might choose to use PostgreSQL [148] for its ticketing use case and a data warehouse like AWS Redshift [19] for its sales analytics since these two systems make design decisions that let them excel at these respective workloads. PostgreSQL offers transactional support and provides indexes to accelerate point reads and writes [148], making it a natural choice for ticketing. Redshift, in contrast, does without indexes [27] and instead combines a columnar data layout with clever data partitioning [59], among other design decisions, to optimize for large complex analytical queries. Our example company might also leverage a data lake (e.g., querying data on AWS S3 [21] using BigQuery Omni [79]) for their other analytics use cases to take advantage of its serverless pricing model (i.e., paying only for the data stored and scanned). These concrete examples form just a small slice of the choices available to data infrastructure engineers, as the last decade has seen an explosion in the number of specialized relational database systems. At the time of writing, Amazon Web Services and Google Cloud together offer nearly twenty different cloud database systems [24, 25, 78].

Despite the performance benefits from specialization, these multi-system data infrastructures bring three significant practical challenges.

1. **Designing an "optimal" infrastructure is difficult.** The best design (e.g., that minimizes cost while meeting performance constraints) depends on many interconnected factors such as query selectivity, service level objectives (SLOs), and dynamic system load [103]. Consequently, practitioners need intimate knowledge of their workload and the characteristics of their database system options to make good design decisions.

2. **Changing the infrastructure later on (e.g., due to business growth or new use cases) is cumbersome.** Once the data infrastructure is designed, application code often ends up tightly coupled to the specific engines used (e.g., relying on engine-specific APIs or libraries). Swapping out engines (e.g., if a more cost-effective one becomes available) can be a risky undertaking because it may require non-trivial application code changes.

3. **End-users (e.g., data scientists or application developers) have to deal with additional complexity when using the data infrastructure.** They need to identify the exact system or combination of systems to use for their queries. If they lack expertise in the underlying systems and the infrastructure's design, the complexity could lead to a poor user experience and suboptimal use of the infrastructure.

In this thesis, we propose a new unifying framework, *data infrastructure virtualization*, that addresses these challenges while also enabling new optimization opportunities (Figure 1.1). The key idea is to separate a data infrastructure's logical application-facing requirements from the physical engines that could be used to realize them. This is done by declaring a set of *virtual database engines* (VDBEs), which specify the application-facing properties (e.g., query interface and performance) and tables an engine should have, but do not prescribe a specific concrete engine. VDBEs are then realized by mapping them to concrete physical engines that are capable of implementing their declared properties. Multiple VDBEs can be mapped to the same physical engine, just like how multiple virtual machines can run on the same physical server.

Virtual data infrastructures are powerful because they decouple end-user applications from the underlying physical engines, providing *physical design independence* to the data infrastructure. Applications interact directly with a VDBE (e.g., connecting to a VDBE endpoint exposed by a database proxy [143]) and are oblivious to the underlying physical engines. This separation (i) allows for transparent infrastructure changes without modifying application code, which solves the second challenge above, (ii) provides a mechanism to support automated infrastructure designs, and (iii) simplifies the infrastructure for end-users as each VDBE can be used to support a single data use case. Thus in this thesis, we make the following claim about the generality and benefits of data infrastructure virtualization:

Thesis statement

Data infrastructure virtualization provides *physical design independence* for multi-database infrastructures, enabling new methods for (i) automatically designing data infrastructures to optimize performance and cost; and (ii) non-invasively applying workload-specific query execution optimizations.

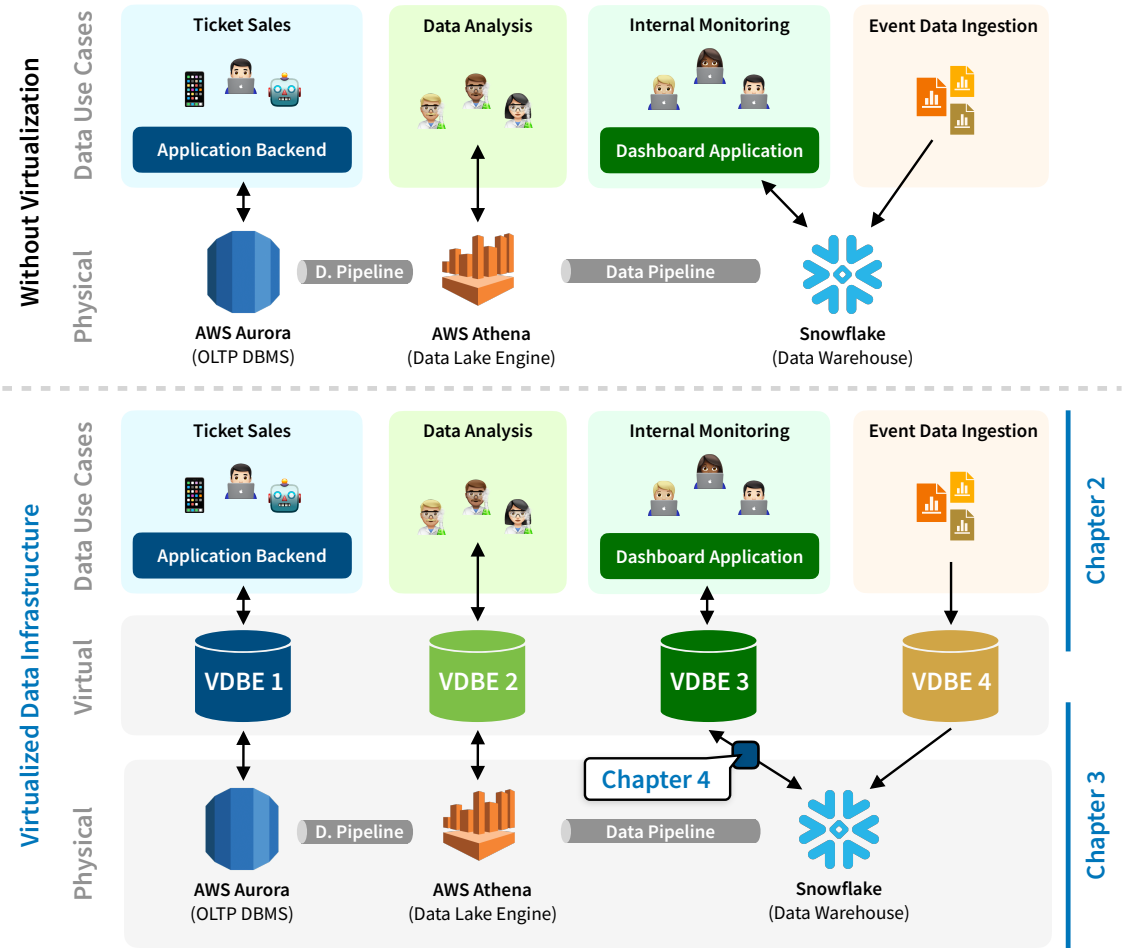


Figure 1.1: Organizations currently design their data infrastructure by manually mapping their data use cases to concrete database engines. In addition to being hard to optimally design (e.g., lowest cost design subject to performance constraints), multi-engine infrastructures are difficult to change (e.g., add or remove engines) because application code often ends up tightly coupled to the chosen physical engines. This thesis introduces *data infrastructure virtualization*. Users declare a set of VDBEs, one per data use case, that describe each virtual engine's application-facing properties (query interface and performance) (Chapter 2). An automated planner can then decide how to best map the VDBEs onto physical database engines to minimize cost under performance constraints or vice versa (Chapter 3). VDBEs also give us the opportunity to control how queries are executed, creating the opportunity to integrate specialized per-query acceleration optimizations (Chapter 4).

We support this thesis statement by making three core contributions: (i) describing the virtual data infrastructure abstraction, its data consistency and transactional isolation semantics, and illustrating its expressiveness on common data infrastructure designs (Chapter 2); (ii) developing a new technique to automatically realize a virtual data infrastructure using physical database engines (Chapter 3); and (iii) presenting a non-invasive framework for specialized query acceleration by leveraging the indirection provided by virtual database engines (Chapter 4). We give a brief overview of each contribution next.

1.1 Virtual Data Infrastructures

A virtual data infrastructure is composed of a set of *virtual database engines* (VDBEs). A VDBE specifies the application-facing properties a database system should provide: its tables, its query interface (dialect and supported features), and constraints on its performance and data freshness relative to other VDBEs. To users, VDBEs appear as concrete database systems, but they can be implemented using any concrete database system capable of providing the VDBE's declared properties.

The key feature of the VDBE abstraction is that VDBEs are defined over shared logical data. This means that the same table can be in multiple VDBEs, with each VDBE exposing a logical snapshot of its tables. If a table appears in multiple VDBEs, it corresponds to a different logical snapshot in each. This property is powerful because it lets data architects design their infrastructure around data use cases without having to first decide how many physical data replicas to maintain or which concrete database engines should store them. For example, our movie theater company from earlier could create one VDBE to handle its ticket sales (representing a traditional "transactional DBMS") and another VDBE for financial reporting over the sales data (representing a traditional "data warehouse"), both with an overlapping set of tables.

Allowing VDBEs to have overlapping table sets is what makes the abstraction powerful, but it also makes its semantics nontrivial. If one VDBE writes to a table that another VDBE also contains, the abstraction must define when that write becomes visible through the second VDBE and how transactions issued through different VDBEs should be isolated from one another. Chapter 2 thus makes the VDBE abstraction precise. We first define VDBEs and demonstrate their expressiveness by presenting VDBE design patterns for common data use cases. We then formalize a VDBE's data consistency and transactional isolation semantics and present one method for realizing these semantics on physical engines. The key idea is to derive a set of logical backing snapshots, each of which provides data for (i.e., backs) one or more VDBEs and can be independently mapped to a physical engine. Finally, we conclude with a sketch of how a virtual data infrastructure could be implemented in practice.

1.2 Automatically Realizing Virtual Infrastructures

A second key benefit of virtualization is that it enables us to automatically design the underlying physical infrastructure. A set of VDBEs imposes a set of constraints over the physical infrastructure design space. Similar to how query planners search for the best physical plan for a query, a data infrastructure planner can search for the best physical infrastructure that realizes a set of VDBEs. This is a challenging problem because we must explore a huge space of possible designs while meeting performance constraints.

We solve this problem using a novel technique we call *blueprint planning*, which is a holistic *cost-based optimization* over the infrastructure design space. Specifically, blueprints are system plans that define a VDBE deployment. They contain (i) the set of engines to include in the infrastructure, (ii) their provisioning configurations (e.g., instance type and number of nodes), (iii) the engine to which each VDBE is mapped, and (iv) a policy for routing queries to the engines. Blueprints allow us to systematically and quantitatively consider all aspects of the infrastructure design problem in a unified search space, analogous to traditional query planning [160]. However, accurately assigning scores to blueprints is significantly harder than query planning. First, the utility of a blueprint is not captured by performance alone, as a good blueprint for a given workload minimizes dollar-based operating costs under a latency-based performance constraint (or vice versa, depending on user-specified goals). Second, accurately predicting a workload's performance (e.g., a query's run time) on a blueprint is difficult due to (i) engines having opaque system implementations and (ii) new constraints in our setting. Specifically, we must make these predictions when a physical query plan is unavailable, preventing us from reusing existing learned models [87, 116, 117, 119, 173, 197]. For example, a candidate blueprint may add an engine into the infrastructure that is not yet running (e.g., starting up a data warehouse) or replicate a table onto a new engine to support a query.

In Chapter 3, we show that these challenges are tractable. In the cloud setting, infrastructure operators can collect performance data over a wealth of workloads and deployments to build learned performance models. Moreover, most query optimizers are deterministic. Thus, we can train a model to predict a query's run time using just its logical properties (e.g., filter selectivities, join templates) since the optimizer will pick similar query plans with comparable run times for similar queries. We leverage these observations to build a graph neural network with a novel query featurization that relies only on such logical query features (Section 3.3.2). Together with other analytical models, we use this model to predict the performance and cost of candidate blueprints on a given workload. We then use these predictions to drive a greedy beam-based search over the blueprint search space to find an optimized infrastructure design. We have implemented our blueprint planner in a virtual data infrastructure runtime called BRAD, enabling it to automatically design infrastructures consisting of three engines that cover a broad range of enterprise needs: (i) a transactional store (Aurora [16]), (ii) a data warehouse (Redshift [19]), and (iii) a data lake query engine (Athena [14]). Our experiments show that BRAD can automatically pick infrastructure designs that meet performance constraints while achieving 1.6×–13× lower

cost than using existing cloud serverless engines. We discuss blueprint planning in more detail in Chapter 3.

1.3 Specialized Query Accelerators

Finally, virtual data infrastructures also open up new opportunities to accelerate query execution through focused specialization. We make the key observation that while relational database systems (RDBMSes) can run any SQL query, they often have lower performance than purpose-built solutions for specific classes of queries [2, 5, 83, 91, 114]. For example, consider a group-by query over a few known groups (e.g., grouping by country). While an RDBMS would likely use a hash map to do the grouping, a faster method could hard-code the expected groups into the query executor. Such workload-specific techniques, which we call *query accelerators*, are currently not widely used in practice because the engineering effort (optimizer and engine changes, potential bugs) does not always justify the isolated performance gains (speedup on a specific query). However, since applications submit queries to VDBEs instead of physical engines, we can interpose on the query submission path and transparently choose how to execute the query, which may include dispatching part of it to a query accelerator.

To that end, Chapter 4 introduces Tailwind: a non-invasive query planner and executor for adding query accelerators to any RDBMS that supports data import and export. Tailwind sits at the VDBE layer. Accelerator builders declare the logical plan fragments their accelerator can compute and provide an implementation against Tailwind's execution API. Offline, given a representative query log and space budget, Tailwind learns the accelerators' performance, selects accelerator instances to minimize predicted query run time, and manages their side information (if any). Online, when a query arrives, Tailwind interposes on the VDBE's query submission path and transparently rewrites queries to use those accelerator instances when predicted to be beneficial. The end result is a system that automatically and *surgically* inserts accelerators into an RDBMS without modifying its internals, helping to close the gap between specialized and general-purpose systems.

Realizing Tailwind presents a significant design challenge: we need a generic way to (**C1**) let users express their accelerator's semantics, (**C2**) interface with their implementation, and (**C3**) automatically model their performance to know when an accelerator should be used. For example, in our known group-by acceleration example, users need a simple but precise way to describe the specific "group by pattern", their implementation needs to know what columns to group by, and Tailwind needs to understand how these pieces influence the accelerator's performance.

To address these challenges, we introduce *abstract logical plans* (ALPs): a new abstraction that accelerator builders use to describe their accelerator's semantics. At its core, an ALP centers on a parameterized regular tree expression [9, 55] over logical query plans that specifies the set of logical plan fragments an accelerator can correctly replace (**C1**). The parameters exist to capture query-specific information that can vary across accelerator uses (e.g., the predicate to negate in domain negation). The same ALP serves two further purposes.

It can be compiled into a tree automaton [55, 63, 177] to efficiently search query plans for matches and extract parameter values (**C2**), and its tree structure and parameters can be used to construct a custom neural network that estimates when the accelerator is beneficial to use (**C3**, Section 4.3.3). For Tailwind, ALPs therefore provide a unified abstraction for an accelerator's applicability, interface to its implementation, and performance modeling. We demonstrate the generality of Tailwind and ALPs using three case studies on distinct query workloads. Across all three cases, Tailwind lets us integrate workload-specific accelerators to speed up queries on Redshift and DuckDB, achieving geomean query run time speedups of up to 1.38×, 1.76×, and 1.28×. We discuss ALPs and Tailwind in more detail in Chapter 4.

Thesis organization. The rest of this thesis is organized as follows. We begin by examining the virtual data infrastructure abstraction in detail in Chapter 2. We then look at blueprint planning in Chapter 3, which is our solution for automatically realizing a virtual data infrastructure using a set of physical database engines. Then, leveraging the indirection offered by a virtual data infrastructure, Chapter 4 describes how we can non-invasively integrate specialized query acceleration mechanisms into database systems. Finally, we review related work in Chapter 5 and conclude with a discussion of future work in Chapter 6.

Chapter 2

Virtual Database Engines

2.1 Introduction

Virtual database engines (VDBEs) are the central abstraction underpinning virtualized data infrastructures. As discussed in Chapter 1, the power of the VDBE abstraction is that the same table can be in multiple VDBEs. This property lets developers declaratively specify different use cases of the same logical data. Ultimately, this distinction reframes data infrastructure design as decomposing high-level business needs into distinct data use cases, instead of low-level decisions about which concrete database engine(s) to use.

To show the expressiveness of the abstraction, we begin by presenting example virtual data infrastructure designs covering common data use cases including hybrid transactional/analytical processing (HTAP), extract-load-transforms (ELTs), and differing freshness requirements, among others.

We then make the VDBE abstraction precise. The main challenge is that VDBEs with overlapping table sets are useful only if their semantics are well-defined: if one VDBE writes to a table that other VDBEs also contain, a virtual data infrastructure must specify the order in which those writes are observed on the other VDBEs. Our key idea is to separate the logical history of a table from the snapshots exposed through individual VDBEs. Each table has a single global history of committed writes, while each VDBE exposes a logical snapshot consisting of prefixes of those table histories. A VDBE's maximum staleness constraint determines how far behind those prefixes may be, and its transactional requirements determine if its transactions must be isolated together with transactions issued through other VDBEs (Section 2.5.2).

Finally, we conclude by showing how to realize VDBEs using physical engines while maintaining their data consistency and transactional isolation semantics. The high-level idea is to group together VDBEs that must be realized over the same logical copy of data in order to preserve the virtual infrastructure's semantics. These groups of VDBEs, called logical backing snapshots, provide the interface between the logical VDBE specification in this chapter and the physical planning problem studied in the next chapter.

Contributions. In summary, this chapter makes the following contributions:

- The virtual database engine and infrastructure abstractions, together with details about their data consistency and transactional isolation semantics.
- Design pattern examples for expressing common data infrastructure designs using virtual database engines.
- A method that takes a virtual data infrastructure and produces a set of logical backing snapshots (table placement groups) that are used to realize VDBEs on physical database systems while meeting the infrastructure's consistency and isolation semantics.

2.2 Virtual Data Infrastructures

A virtual database engine (VDBE) is a logical specification of a database system. A virtual data infrastructure is simply a set of VDBEs. Concretely, a VDBE comprises

1. A set of tables and a per-table flag indicating if the VDBE writes to the table.
2. A query interface (dialect and supported features, e.g., PostgreSQL-compatible SQL).
3. A maximum staleness constraint (explained below).
4. An optional performance constraint (e.g., p90 query latency under 30 seconds).

Crucially, VDBEs in a virtual data infrastructure can have *overlapping table sets*, meaning the same table can be in multiple VDBEs. A VDBE represents a *logical snapshot* of a set of tables. If a table is in multiple VDBEs, it is part of a different logical snapshot in each VDBE. By snapshot, we mean a collection of tables, each at a specific version and only seeing writes from committed transactions. While counterintuitive at first, this is the key property that makes it possible to decouple logical data use cases from their physical realization as it lets each VDBE represent exactly one use case over logical data. We go through examples of how to leverage this property in Sections 2.3 and 2.4, and we define a VDBE's data consistency and isolation semantics precisely in Section 2.5.

Maximum staleness constraint. Queries and transactions running on a VDBE always see a snapshot of the VDBE's tables. A VDBE V's maximum staleness constraint therefore specifies, for every table T in V, the *maximum staleness* that V's logical snapshot is allowed to have with respect to writes made to T on *any* VDBE in the virtual data infrastructure. This staleness can be zero, which is for classical transactional use cases (e.g., accepting e-commerce orders). A non-zero value means bounded staleness, which is common in existing data infrastructures that rely on periodic ETLs [92, 167, 198].

Maximum staleness constraints can be specified as a period of time (e.g., at most 30 seconds stale) or as a relative timepoint (e.g., has all writes made as of 1:00 am today). When giving this constraint as a relative timepoint, users need to provide a replication time budget

Table 2.1: A summary of the properties of a VDBE.

Property	Example for VDBE 1 in Figure 2.1
Table set (with schema and write flags)	`showings` (write), `tickets` (write)
Query interface (dialect and features)	SQL-99, supports transactions
Performance constraint	p90 query latency < 30 milliseconds
Maximum staleness	No staleness permitted

(e.g., has all writes made as of 1:00 am today, available at most 10 minutes afterwards). This budget is necessary because the VDBE with this staleness constraint could be realized on a different physical engine than the VDBE receiving writes to one or more of its tables; the budget specifies how long those writes take to become visible after the timepoint passes.

Performance constraint. One benefit of virtual data infrastructures is that they enable automated infrastructure design: a planner can decide how to realize VDBEs on physical database engines. To make these decisions, the planner needs an optimization objective; a common practical choice is to minimize cost, which is only meaningful under a performance constraint. Without a performance constraint, a planner could trivially choose the lowest cost configuration, such as mapping all VDBEs to an under-provisioned engine, even if the resulting infrastructure would be unusable in practice. Users can therefore specify performance constraints for the queries running on a VDBE, which affect the optimization objectives that guide automated planning (discussed in Chapter 3). If a user is manually designing their infrastructure, they do not need to provide any performance constraints.

Now to help provide some intuition for the VDBE abstraction, let us look at an example of how a company might use VDBEs to model their data use cases.

2.3 Virtual Database Engines at QuickFlix

Consider the data needs of QuickFlix, a fictitious movie theater company. QuickFlix is modernizing their operations and wants to start selling movie tickets online. They also want to run analytics over their sales data (e.g., to select better movie showing times). For simplicity, suppose they model their data using three tables: (i) `showings` for movie showings, (ii) `tickets` for ticket orders, and (iii) `view_hist`, which holds a history of each customer's movie views and is derived from the data in `showings` and `tickets`. QuickFlix needs to access `showings` and `tickets` using transactions (e.g., adding new showings and recording ticket orders). Furthermore, they will only use `view_hist` for their sales analysis.

Thinking in data use cases. With VDBEs, QuickFlix designs their infrastructure by thinking about their *data use cases*. They have three use cases: (i) selling tickets, (ii) running analytics on their sales data, and more subtly (iii) keeping their `view_hist` table up-to-date with new ticket orders. After identifying their use cases, QuickFlix creates a VDBE for each use case (as shown in Figure 2.1 (A)).

To make ticket sales, QuickFlix creates VDBE (1) that contains the `showings` and `tickets`

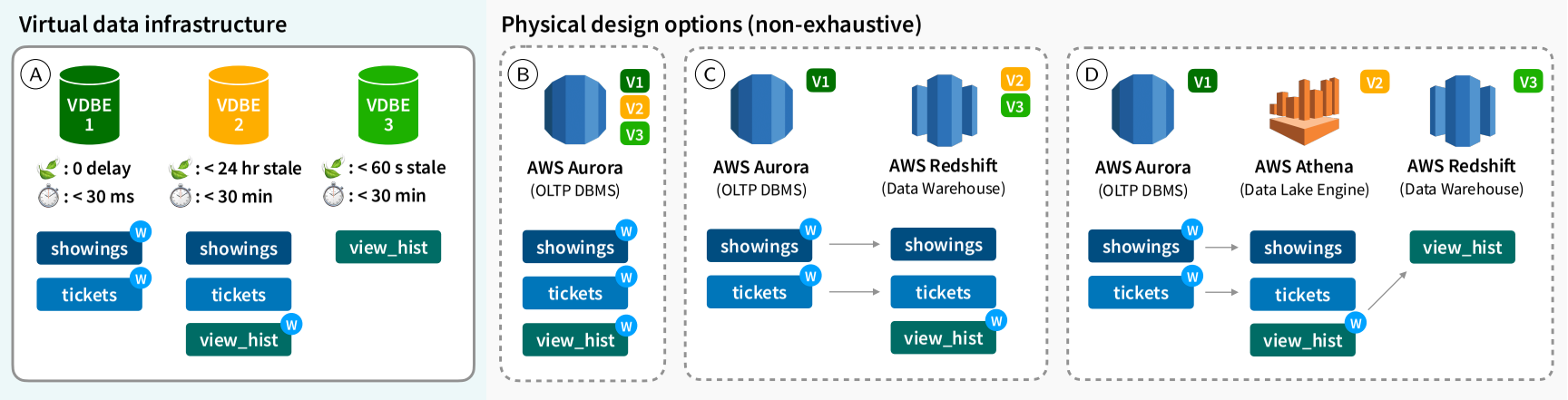


Figure 2.1: QuickFlix's virtual data infrastructure alongside three possible physical designs. The "W" means the engine writes to the table and the gray arrows indicate the direction of physical data replication.

tables, supports transactions, and has a low query latency. For sales analytics, they create VDBE (3), which has the `view_hist` table. They set a performance SLO of 30 minutes since they will run large exploratory queries. To maintain the `view_hist` table, they create VDBE (2), which holds the `showings`, `tickets`, and `view_hist` tables; this VDBE runs queries that transform the data in `showings` and `tickets` into `view_hist` (we discuss data consistency in Section 2.5). They again set a performance SLO of 30 minutes as the transformation queries will take time to run. We summarize these properties in Table 2.1. By using VDBEs, notice that QuickFlix focuses on carving out their high-level data use cases instead of being bogged down with physical implementation details.

2.3.1 Maximum Staleness Constraints

We can now look at QuickFlix's maximum staleness constraints. Recall that the same table can appear in multiple VDBEs (Figure 2.1 (A)); each VDBE represents a logical snapshot of data. VDBE 1 has zero staleness as it is used for ticket sales, which cannot run on stale data. VDBE 2 runs transformations that update `view_hist` using `showings` and `tickets`. Since `view_hist` is used for historical analytics, QuickFlix is fine with it being up to one day stale. This means the transformations can run using versions of `showings` and `tickets` that are up to one day stale. Consequently, QuickFlix sets VDBE 2's maximum staleness to one day. VDBE 3 supports analytics on `view_hist`. QuickFlix wants to use the latest version of `view_hist` once updates complete on VDBE 2, but does not need it to be immediately accessible. So VDBE 3 declares a maximum staleness of one minute.

2.3.2 Query Dialects

Lastly, VDBEs must declare a SQL query dialect. This is an important practical consideration as different physical engines use different dialects, which are typically not fully compatible. Knowing the dialect ensures that VDBEs are mapped to an infrastructure that is capable of supporting the dialects its applications need. For QuickFlix, they set all three VDBEs to SQL-99 as they are not using any engine-specific features.

Importantly, specifying a dialect does not necessarily constrain the VDBE to a single physical engine. When it is possible to automatically translate queries and emulate any missing functionality (e.g., special SQL functions specific to a dialect) [133], we can map a VDBE to a physical engine that implements a different dialect. If the VDBE's dialect is only partially translatable, then we could also realize the VDBE on two physical engines: one having a different dialect that supports the translatable queries, and one for the remaining untranslatable queries. These physical design options are beneficial if they bring better performance or a lower overall cost when compared to using a physical engine that directly uses the declared dialect.

2.3.3 Physical Design Options for QuickFlix

Figures 2.1 (B), (C), and (D) show three possible physical designs for QuickFlix's VDBEs. We could map all three VDBEs onto a single Aurora engine (Figure 2.1 (B)). This design is cost-efficient and provides acceptable performance at low workload scales [200]. Another option is to use two engines: Aurora for transactions on `showings` and `tickets` and Redshift for analytics on `view_hist` (Figure 2.1 (C)). This design is more expensive but provides better performance at higher workload scales [200]. A third option is to use Athena, a serverless engine, for VDBE 2 (Figure 2.1 (D)). This design makes sense if QuickFlix's transformations require much more compute than their analytics. In that case, offloading the transformations to Athena would be less costly than using a larger Redshift cluster for both VDBEs 2 and 3. A data architect (or an automated planner; see Chapter 3) could pick the best design based on the workload and QuickFlix's cost constraints. Notice that even if QuickFlix chooses to manually design their infrastructure, using VDBEs gives them the flexibility to change their physical design later on without having to modify their application code.

2.4 Virtual Data Infrastructure Design Patterns

To show the broad applicability of VDBEs, let us look at more example virtual infrastructure designs for common data use cases. In these examples, we focus on how to place tables into VDBEs and how to set their maximum staleness constraints to meet each data use case.

2.4.1 Classical HTAP

A common pair of data use cases is needing to run transactions and analytics over the same set of tables. For example, a movie theater company might want to write ticket orders to a table and run sales reporting queries over the same tables. Such workloads are typically called HTAP [122, 168], and can be handled using a single database system (at small data scales), a pair of systems connected with a periodic ETL, or a purpose-built HTAP system [57, 69, 146].

This design is straightforward to specify using VDBEs. Users simply create one VDBE for the transactional use case and one VDBE for the analytical use case, both with the same set of tables. The transactional VDBE should have a maximum staleness of zero to ensure that it always operates on the latest version of the data. The analytical VDBE's maximum staleness depends on the use case. For example, if the analytical workload is historical sales reporting and only needs to process data from the previous day, it can have a maximum staleness of one day. Users can also create more than one analytical VDBE, each with different maximum staleness requirements, if they have multiple use cases with different freshness needs (e.g., quarterly sales reporting versus computing daily sales summaries).

Figure 2.2 shows this virtual infrastructure. The "W" next to a table indicates that the VDBE makes writes to the table. Notice that there are multiple physical ways to realize the

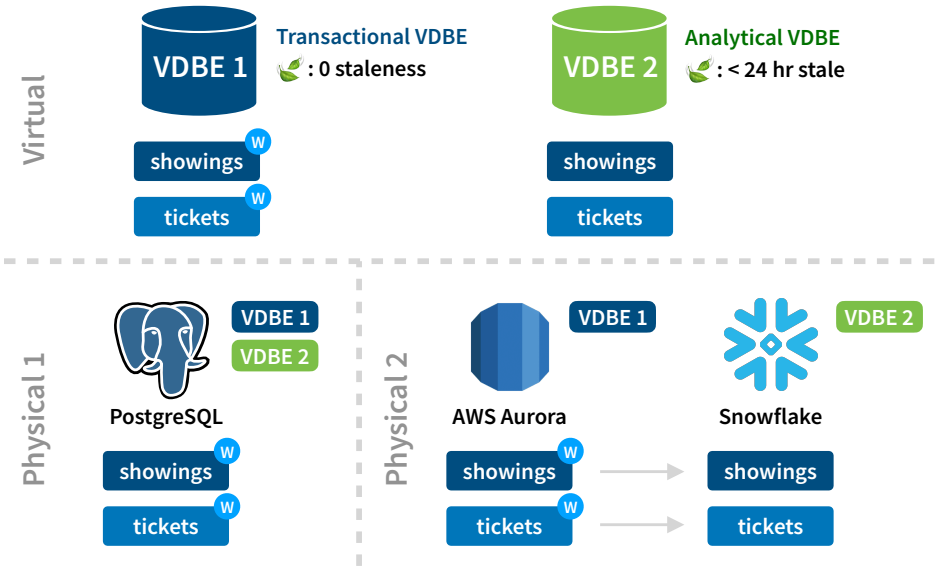


Figure 2.2: An example virtual data infrastructure design for HTAP, along with two possible physical realizations. We can map both VDBEs onto the same engine or to distinct engines connected with a data pipeline. The gray arrows indicate the direction of data replication.

virtual infrastructure. We can map both VDBEs to the same system. We can also map each to a distinct database system with the systems connected by an ETL (e.g., running daily).

2.4.2 Distinct Data Access

Some organizations may have multiple teams that need access to the same data, usually for different use cases. For example, they could have a team of (i) data scientists that run ad-hoc data analysis queries, (ii) financial analysts tasked with running the same set of queries every quarter for financial reporting, and (iii) engineers that run an internal dashboard for near-real-time sales visualization.

With virtual data infrastructures, the organization can create one VDBE for each team (each data use case), as shown in Figure 2.3. The key advantage of this design is that the VDBEs can be mapped to the same or different physical engines, depending on the performance and cost needs of the organization. For example, the data analysis and financial reporting queries might interfere with the internal dashboard use case. Consequently, we could map the internal dashboarding VDBE to its own distinct physical engine, while mapping the first two VDBEs to the same physical engine.

2.4.3 Freshness-Sensitive Reports

VDBEs also make it easier to specify different freshness constraints for different uses of the same underlying data. For an example of how this requirement might come up, consider an organization with a data warehouse containing many tables. Suppose that most of their

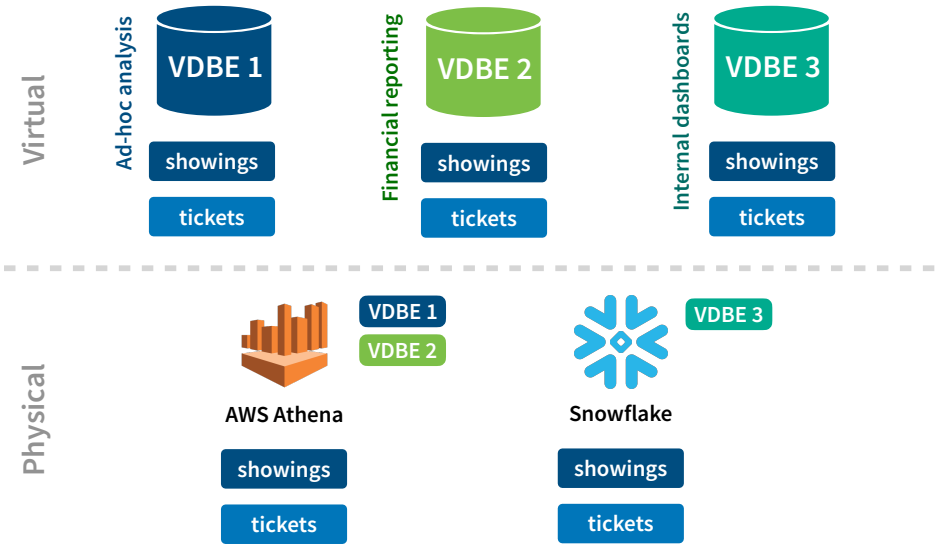


Figure 2.3: An example virtual data infrastructure showing distinct data use cases over the same data, along with one possible physical realization. Here, we realize the ad-hoc analysis and financial reporting VDBEs using the same physical engine (AWS Athena) and map the internal dashboarding VDBE onto its own physical engine (Snowflake).

workloads can tolerate data that is up to one day stale, but a daily CEO report depends on a small subset of tables that must reflect data from the last hour.

The organization can express these requirements using separate VDBEs, as shown in Figure 2.4. Here, we have four distinct use cases. We have a ticketing VDBE meant to support ticket sales to customers (VDBE 1). We also have a VDBE to ingest event data (e.g., clicks, pageviews) on the organization's website (VDBE 2). This data needs to be queryable in a data warehouse, which can be up to one day stale (VDBE 4). Finally, to support having a fresh CEO report, we create a VDBE that has a maximum staleness of one hour (VDBE 3). Here the report is on ticket sales, so we only need to include the tickets table.

Figure 2.4 also shows one possible physical realization. We use a transactional DBMS for the ticketing VDBE (PostgreSQL). Since we need high freshness, we spin up a separate database engine (Redshift) for the CEO reports VDBE. We can keep it fresh by periodically copying over the new writes from PostgreSQL. Finally, we create a third physical engine (Snowflake) to hold all the tables for the data warehouse VDBE. Notice that we map the event data VDBE to Snowflake too, as multiple VDBEs can be mapped to the same physical engine. We keep this engine fresh also by periodically copying over the new writes from PostgreSQL. Crucially, this periodic copying does not need to run as frequently as it does for Redshift, given the more relaxed staleness constraint.

2.4.4 Extract-Load-Transforms (ELTs)

Some data infrastructures run transformations as ELTs [29, 166], meaning tables are first replicated into a data warehouse or data lake and then transformed using DML statements.

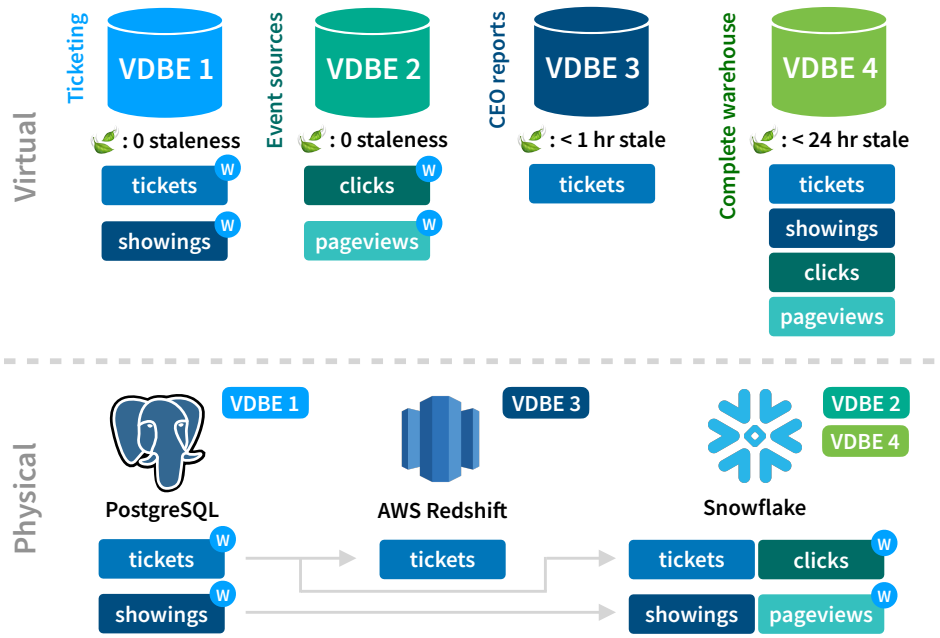


Figure 2.4: An example virtual data infrastructure showing VDBEs with different staleness constraints for different business needs. We show one possible physical realization. Notice that we can map the event sources and data warehouse VDBEs to the same physical engine. The gray arrows indicate the direction of data replication.

Figure 2.5 shows how an ELT architecture can be expressed with VDBEs, representing a movie theater making ticket sales and running analytics on their view history, which is a table derived from a transformation on their ticket sales tables. The key idea is to declare a VDBE for each distinct use case: one for the ticket sales, one for the analytics, and crucially, one to run the transformations. VDBE 1 represents the ticket sales, where ticket orders are written into the database transactionally. VDBE 3 represents the analytics use case over the `view_hist` table, which contains data derived from the `showings` and `tickets` tables. To handle the transformations, we declare VDBE 2, which contains the `showings`, `tickets`, and `view_hist` tables. It reads from `showings` and `tickets` and writes to `view_hist`. VDBE 2 has a maximum staleness of 24 hours, meaning each transformation reads ticket data that is at most one day old. The analytics VDBE (VDBE 3) has a maximum staleness of 10 minutes to allow VDBE 2 to be realized on a separate physical engine. If it had a maximum staleness of zero, VDBEs 2 and 3 would be co-located on the same engine (see Sections 2.5 and 2.6).

Figure 2.5 also shows one possible physical realization. Although we mapped VDBEs 2 and 3 onto the same physical engine (Snowflake), we could have also mapped them to distinct engines. That design would make sense when the transformation we need to run has better performance on a different engine, or is more cost-efficient on a different engine.

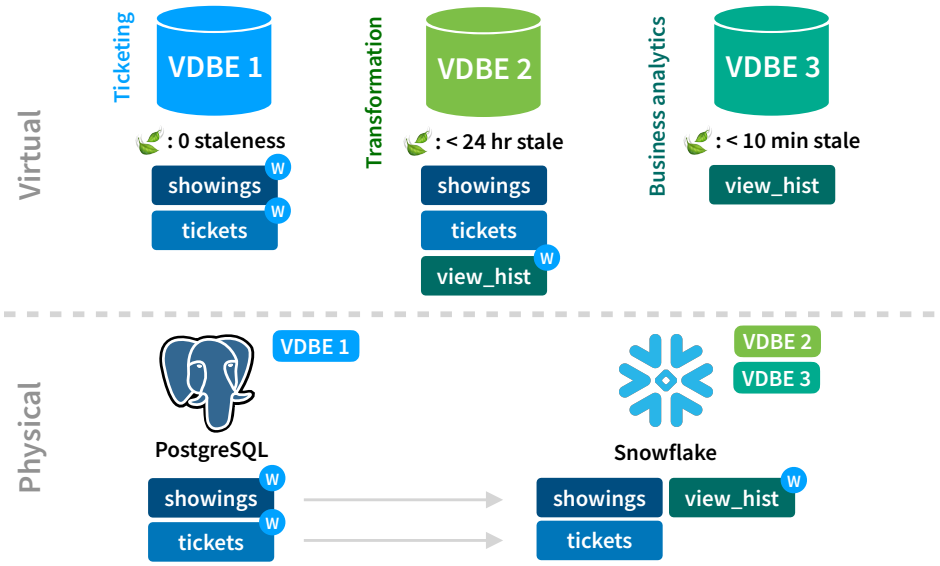


Figure 2.5: An example virtual data infrastructure showing how to express extract-load-transforms. Users declare one VDBE for each data use case, of which running transforms is one. The transformation VDBE can be mapped to the same physical engine as the business analytics VDBE (as shown), or can be mapped to its own physical engine. The gray arrows indicate the direction of data replication.

2.4.5 Data Lakehouses

Data lakehouses and fabrics [121] are increasingly common data infrastructure designs [34]. At a high level, the key idea is to store data in an object store (e.g., S3 [21]) and access it using a variety of front ends depending on the application use case. This architectural design is also expressible using VDBEs. Similar to how a data lakehouse has different front-end database engines for accessing the data, one creates one VDBE for each use case. The VDBEs' data can then be mapped to the same underlying object store.

2.5 Consistency and Isolation Semantics

We now formalize the data consistency and transactional isolation semantics of VDBEs. Since VDBEs in a virtual data infrastructure can have overlapping table sets, these semantics specify when writes in one VDBE become visible in another and how transactions over those tables are isolated.

2.5.1 Data Consistency

Table write history. For each table T in a virtual data infrastructure, all committed writes to T define a single total write order, which we call T's *write history*. The write history is global and independent of VDBEs: if two VDBEs both contain T, they observe writes to T according

to the same commit order. We identify a version of T with a prefix of T's write history, or equivalently with the last committed write included in that prefix. Thus, making a table version visible to a VDBE means exposing some prefix of the table's committed writes.

VDBE snapshot. A VDBE exposes a logical snapshot over its table set. Since each table has a single write history, the snapshot corresponds to one prefix of each table's committed write history. As mentioned earlier, VDBEs have maximum staleness constraints. Thus at physical time τ, a VDBE with maximum staleness Δ must expose every write to its tables that was committed before $\tau - \Delta$.[1] Informally, this means each table cannot be stale beyond the VDBE's maximum staleness constraint.

For example, suppose a VDBE contains tables A and B, exposes version A_2 and B_3, and has a maximum staleness of 1 hour. The data consistency requirement is that A_2 is a prefix of A's write history, B_3 is a prefix of B's write history, and any newer unexposed versions of A or B must have been committed in the physical time range $(\tau - 1 \text{ hour}, \tau]$, where τ represents the current physical time.

Atomic transaction visibility. If a transaction commits writes to multiple tables on a VDBE, then any VDBE containing more than one of those tables must observe the writes atomically over the subset of tables it contains.

For example, suppose we have VDBEs V_1 and V_2 and we have $V_1 = \{A^w, B^w\}$ and $V_2 = \{A, B, C\}$ as the table sets. Here, T^w indicates the VDBE writes to table T. If a transaction modifies A and B together on V_1, then V_2 must see both writes to A and B or neither; it cannot see the write to one table without the other.

2.5.2 Transactional Isolation

The data consistency semantics above describe how committed writes become visible through VDBEs. Transactional isolation is a separate concern, as it governs how concurrent transactions issued to the same or different VDBEs interact. When declaring a VDBE, users specify whether the VDBE supports transactions as part of its query interface (Section 2.2). A virtual data infrastructure provides transactional isolation only among VDBEs that declare transactional support.

Isolation scopes. Virtual data infrastructures provide transactional isolation within *isolation scopes*. Informally, an isolation scope is a set of transactional VDBEs that have cyclic read-write dependencies. We define these scopes using a directed dependence graph over all VDBEs in the virtual infrastructure. Each VDBE is a node. We add a directed edge $(V_i \rightarrow V_j)$ if there exists a table T such that T is in both V_i and V_j, $V_i \neq V_j$, and V_i declares that it writes to T. The edge means that transactions issued through V_j may observe writes made by transactions issued through V_i.

An isolation scope is a strongly connected component of this graph whose VDBEs all declare transactional support. A strongly connected component is a maximal set of nodes in

[1] This is for maximum staleness constraints specified as a period of time. For maximum staleness constraints specified as a relative timepoint, it is straightforward to derive a different time bound.

which every node can reach every other node by following directed edges. Thus, isolation scopes capture precisely the groups of VDBEs whose read-write dependencies are cyclic.

Within each isolation scope, the virtual data infrastructure provides serializable isolation: transactions issued through VDBEs in the scope must behave as if they execute in some serial order. Thus, the VDBEs in an isolation scope must be realized under a common transaction manager that can provide serializable isolation (e.g., in the same physical database engine). We discuss how VDBE snapshots can be physically realized to make these guarantees in Section 2.6. Exploring weaker isolation levels [4] is beyond the scope of the thesis.

Well-formedness. We construct the dependence graph mentioned above over all VDBEs, including VDBEs that do not declare transactional support. This graph allows us to detect cyclic read-write dependencies that involve VDBEs without declared transactional support. If a multi-VDBE strongly connected component contains any VDBE that does not declare transactional support, then the virtual infrastructure is considered not well-formed. In this case, we reject the virtual infrastructure and ask the user to either mark the relevant VDBE(s) as transactional or revise their table sets and write declarations.

Isolation boundaries. Virtual data infrastructures do not provide isolation guarantees across distinct isolation scopes. When VDBEs in different isolation scopes have overlapping tables, the visibility of committed writes is governed by the data consistency semantics described in Section 2.5.1. In other words, virtual data infrastructures do not guarantee that transactions from different isolation scopes are part of one common serial order.

2.6 Deriving Logical Backing Snapshots

A virtual data infrastructure's consistency and isolation semantics constrain how VDBEs can be mapped to physical database engines. For example, an arbitrary assignment of VDBEs to physical engines could fail to provide a single write order for shared tables.

In this section, we describe one method for deriving VDBE-to-engine mapping constraints that preserve these semantics. The key idea is to introduce an intermediate representation called a *logical backing snapshot*. A logical backing snapshot is a logical snapshot that supplies data for (i.e., *backs*) one or more VDBEs and has its own maximum staleness constraint. It can be viewed as a "higher-order VDBE" that is composed of one or more user-defined VDBEs. Each VDBE is backed by exactly one logical backing snapshot, but a single logical backing snapshot may back multiple VDBEs. Crucially, logical backing snapshots are constructed so that a virtual data infrastructure can be realized by assigning the backing snapshots to physical database engines while ensuring that writes can be replicated between dependent backing snapshots in time to satisfy their staleness constraints. This assignment is not necessarily one-to-one: multiple backing snapshots can be co-located on the same engine and backing snapshots can themselves also be merged. The assignment decisions should be made based on cost and performance (Chapter 3).

The rest of this section describes how to derive these backing snapshots. We first describe how VDBE snapshots can be merged into larger logical snapshots, then identify cases where

we merge, and finally give a graph-based derivation.

2.6.1 Merging VDBE Snapshots

The basic operation in our derivation is merging the logical snapshots of multiple VDBEs into a single backing snapshot. A VDBE represents an application-facing contract: clients must be able to access its tables, and the VDBE must provide data within its declared staleness constraint. A key observation is that this contract does not specify *how* the VDBE must be realized, nor does it require the VDBE to be mapped to a distinct copy of the data. A single logical backing snapshot can therefore supply data for multiple VDBEs as long as it satisfies all of their application-facing contracts. Thus given two VDBEs, we can merge their logical snapshots into one backing snapshot by taking the union of their table sets and using the more restrictive maximum staleness constraint. The resulting backing snapshot therefore satisfies both VDBEs' data and staleness contracts by construction.

2.6.2 Merge Rules for Avoiding External Coordination

We merge VDBE snapshots in the following three cases. Note that these cases are not fundamental requirements of the abstraction. They represent scenarios where mapping VDBEs to separate physical engines would require external coordination to preserve the virtual data infrastructure's consistency or isolation semantics. By merging these VDBEs into the same logical backing snapshot, we can realize them together on a single physical engine and avoid that coordination.

1. **Single writer engine.** To ensure each table has a single total write order, we require each table to have exactly one writer engine. This means if two or more VDBEs write to the same table T, they are merged into the same logical backing snapshot.

2. **Single-copy isolation scopes.** All VDBEs in a read-write dependency cycle are merged into the same logical backing snapshot. This backing snapshot is then mapped to one physical engine that must provide serializable isolation.

3. **Zero staleness merging.** As shown in Section 2.4, users can declare a VDBE to have a maximum staleness of zero, meaning that it must never be stale. To meet this constraint, we merge the VDBE with the VDBEs that write to its tables, ensuring that they belong to the same backing snapshot. This merging ensures they are realized together on the same physical engine.

These rules are conservative. We could support multiple writer engines, but doing so would require external coordination to ensure a single total write order. Similarly, we could map the VDBEs in a read-write dependency cycle to different engines, but doing so would require external coordination to provide transactional isolation. A zero-staleness VDBE could also be realized on a separate engine, but again with external coordination that ensures it never observes stale data. We use these merge rules because they enforce the virtual data

infrastructure's consistency and isolation semantics without introducing expensive external coordination.

2.6.3 Derivation

We derive logical backing snapshots using a dependency graph over VDBEs, similar to how we define isolation scopes (Section 2.5.2). Each VDBE is a node. We add a directed edge ($V_i \rightarrow V_j$) if there exists a table T such that T is in both V_i and V_j, $V_i \neq V_j$, and V_i declares that it writes to T. The edge means that clients accessing V_j may observe writes made by clients accessing V_i. In addition, if a VDBE V_o declares a maximum staleness constraint of zero, we add an edge $V_o \rightarrow V_i$ for all VDBEs V_i that write to a table in V_o. This ensures V_o is part of a strongly connected component containing all VDBEs that write to its tables.

The logical backing snapshots are the strongly connected components of this graph. The strongly connected components are computable in linear time with respect to the graph's size [175]. Each backing snapshot contains the union of the tables declared by the VDBEs in the component and is assigned the tightest maximum staleness constraint required by those VDBEs. Each VDBE in the component is then backed by this logical snapshot.

Sketch of correctness. Together with replication between backing snapshots that preserves transaction atomicity (Section 2.7), this backing snapshot construction satisfies the virtual data infrastructure's consistency and isolation semantics by construction. If two VDBEs declare that they write the same table, the graph contains edges in both directions between them, so they will belong to the same strongly connected component. Therefore, all writes to a table go through one backing snapshot, giving the table a single write order. If a set of transactional VDBEs has cyclic read-write dependencies, they also belong to the same strongly connected component. Therefore, transactions in that dependency cycle execute within one backing snapshot and can be serialized by a single database engine. If a VDBE has a maximum staleness of zero, our edge construction ensures that it will be in the same strongly connected component as the VDBEs that write to its tables, and is therefore merged into the same backing snapshot. Finally, each VDBE is backed by a logical snapshot that contains all of its declared tables and has a maximum staleness constraint at least as strong as the VDBE requires. Thus, merging VDBEs into backing snapshots does not violate the user-visible VDBE constraints.

2.7 Implementation Sketch

We can implement a virtual data infrastructure using a database proxy [23, 143, 201]. The proxy stores a mapping from each VDBE to the logical backing snapshot that implements it, and from each backing snapshot to the physical engine that hosts it. A data architect can choose this mapping manually, or a planner can select it automatically (Chapter 3). The proxy exposes one endpoint per VDBE. Clients connect to an endpoint and issue queries as they would to physical database instances. Internally, the proxy consults its mapping and

forwards the query to the relevant physical engine for execution.

Since multiple backing snapshots, potentially with overlapping tables, can be mapped to the same physical engine, the implementation must avoid physical table name collisions. One simple approach is to give each physical table a unique identifier, such as a backing-snapshot-specific prefix. The proxy then rewrites incoming queries by replacing each logical table reference with the corresponding physical table name. This rewriting is straightforward to implement using a SQL parser. If the underlying database system supports table namespacing mechanisms such as databases or schemas, they can also be used.

In addition to forwarding queries, the proxy initiates periodic background replication between backing snapshots related by data dependencies so that the infrastructure satisfies its staleness constraints. Replication between backing snapshots must also ensure that committed writes from a transaction to multiple tables become visible atomically in downstream backing snapshots. The replication relationships follow from the VDBE dependency graph described in Section 2.6. Recall that logical backing snapshots are the strongly connected components of this graph. If we contract each strongly connected component into a single node, each representing one backing snapshot, the resulting graph is a directed acyclic graph. The edges determine the direction of replication between backing snapshots.

A full implementation and evaluation of such a virtual data infrastructure database proxy is beyond the scope of this thesis. However, in Chapter 3, we study the implementation and overhead of a simple version of this proxy.

2.8 Conclusion

This chapter introduced virtual database engines (VDBEs), the central abstraction in this thesis. The key twist in the abstraction is that VDBEs can have overlapping table sets. If the same table appears in multiple VDBEs, it simply corresponds to a different logical snapshot in each VDBE. This flexibility lets data architects design data infrastructure around use cases rather than physical systems. Each VDBE can represent the data, freshness, and isolation requirements of a particular use case over shared logical data. Virtual data infrastructures ultimately shift the focus of data infrastructure design from choosing physical engines and manually specifying replication relationships to decomposing application needs into distinct data use cases.

Chapter 3

Blueprint Planning

3.1 Introduction

Once we have a set of virtual database engines, the next step is to realize them on physical engines. Since virtual database engines are logical specifications of the properties a database system should provide, we are free to map them to any set of physical database systems as long as they are capable of providing the VDBE's declared properties. This flexibility creates an opportunity for automated infrastructure design: a planner can search for a realization that minimizes cost subject to performance constraints, or conversely that maximizes performance subject to a budget. This chapter explores how to make such decisions automatically.

Automatically realizing VDBEs on physical engines is challenging because the planner must search a large design space while preserving the VDBEs' declared semantics and meeting their performance constraints. Prior work showed that an optimal infrastructure depends on many interconnected factors, including query selectivity, service level objectives (SLOs), and dynamic system load [103]. Moreover, simple designs based on conventional wisdom are insufficient as they can miss out on significant performance and cost savings (Section 3.2.1).

We solve this problem using a novel technique we call *blueprint planning*, which is a holistic *cost-based optimization* over the infrastructure design space. Blueprints are system plans that define a virtual data infrastructure deployment. They contain (i) the set of engines to include in the infrastructure, (ii) their provisioning configurations (e.g., instance type and number of nodes), (iii) a mapping of VDBEs to the physical engines, and (iv) a policy for routing queries to the engines. Blueprints allow us to systematically and quantitatively consider all aspects of the infrastructure design problem in a unified search space, analogous to traditional query planning [160]. However, accurately assigning scores to blueprints is significantly harder than query planning. First, the utility of a blueprint is not captured by performance alone, as a good blueprint for a given workload minimizes dollar-based operating costs under a latency-based performance constraint (or vice versa, depending on user-specified goals). Second, accurately predicting a workload's performance (e.g., a query's run time) on a blueprint is difficult due to (i) engines having opaque system implementations and (ii) new constraints in our setting. Specifically, we must make these predictions when a

physical query plan is unavailable, preventing us from reusing existing learned models [87, 116, 117, 119, 173, 197]. For example, a candidate blueprint may add an engine into the infrastructure that is not yet running (e.g., starting up a data warehouse) or replicate a table onto a new engine to support a query.

In this chapter, we show that these challenges are tractable. In the cloud setting, infrastructure operators can collect performance data over a wide range of workloads and deployments to build learned performance models. Moreover, most query optimizers are deterministic. Thus, we can train a model to predict a query's run time using just its logical properties (e.g., filter selectivities, join templates) since the optimizer will pick similar query plans with comparable run times for similar queries. We leverage these observations to build a graph neural network with a novel query featurization that relies only on such logical query features (Section 3.3.2). Together with other analytical models, we use this model to predict the performance and cost of candidate blueprints on a given workload. We then use these predictions to drive a greedy beam-based search over the blueprint search space to find an optimized infrastructure design. We have implemented our blueprint planner in a system called BRAD, which is a virtual data infrastructure planner and runtime. Blueprint planning enables BRAD to automatically design infrastructures consisting of three engines that cover a broad range of enterprise needs: (i) a transactional store (Aurora [16]), (ii) a data warehouse (Redshift [19]), and (iii) a data lake query engine (Athena [14]).

While there is a wealth of prior work in automatically optimizing single systems [113, 127, 135–137, 139–141, 157, 184], and in managing existing multi-engine deployments [11, 37, 41, 42, 67, 73, 90, 93, 150, 161, 188, 203], BRAD holistically automates and optimizes the design and operation of multi-engine infrastructures. Doing so involves reasoning about cost and performance across engines and hypothetical deployments, which, to our knowledge, has not been studied.

We evaluate BRAD by having it automatically optimize a data infrastructure for cost under a performance constraint. We use a workload with both transactions and diverse analytics running on an adapted version of the IMDb dataset [108]. Overall, we show that BRAD is able to react to changing workloads and select designs that achieve performance targets in diverse deployment scenarios. When compared to a baseline that naïvely auto-scales transactional and analytical systems, BRAD achieves 1.6×–13× lower cost due to its ability to route queries between engines and precisely scale to the resource needs of a workload, instead of reacting passively to increased system load.

Contributions. In summary, we make the following contributions:

- We introduce *blueprint planning*: a new framework for automatically realizing VDBEs on cloud database engines by leveraging cost-based optimization.
- We present a practical blueprint planning solution. We leverage a graph neural network with a novel logical query featurization that generalizes to common gradual workload changes.
- We present the design, implementation, and evaluation of BRAD: a virtual data infrastructure planner and runtime that leverages blueprint planning.

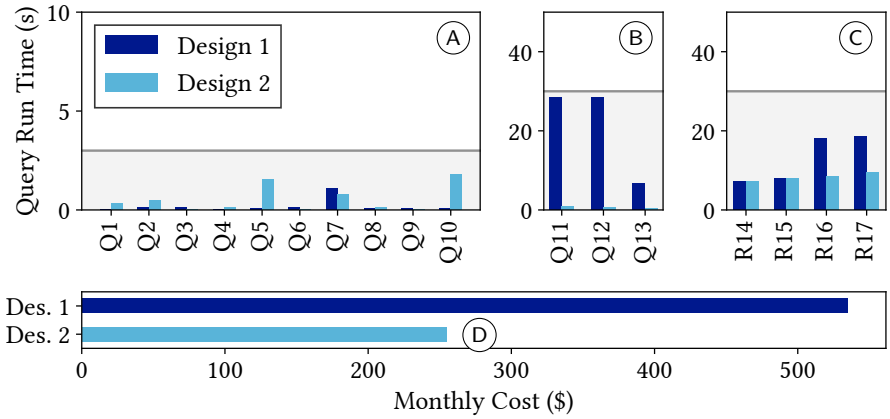


Figure 3.1: Query performance and operating costs of the same workload on two data infrastructure designs.

3.2 Conquering the Complex Cloud

We first illustrate the subtle challenges in cloud infrastructure design and contrast this experience with using BRAD.

3.2.1 When Conventional Wisdom Falls Short

Consider the data processing needs of a movie theater chain. Under conventional wisdom, they should run an OLTP engine (e.g., Aurora) for their transactions and a data warehouse (e.g., Redshift) for their analytics. To show the downsides of this approach, we run a synthetic workload based on a 160 GB version of the IMDb dataset [108] comprising transactions, repeating dashboarding queries (A) (B), and periodic reporting queries (C). We run Aurora with one db.t4g.medium instance and two Redshift dc2.large nodes (Design 1). Figure 3.1 shows the analytical query latencies. We aim to keep some queries under 3 s (A) and others under 30 s (B) (C).

At first glance, Design 1 appears reasonable. However, consider an alternative design with just two Aurora db.t4g.medium instances: a primary and replica (Design 2). We can run a subset of the queries (A) (B) on the Aurora replica and offload the reporting queries (C) onto Athena (a serverless data lake engine). As shown in Figure 3.1, Design 2 saves 2× on cost (D), meets the performance targets, and even improves query latency on some queries (up to 48× (B) and 2.1× (C)). Transaction latency is unaffected on both designs because they run on the unchanged Aurora db.t4g.medium primary instance.

Design 2 performs better because some queries (B) have predicates on indexed columns, which Aurora can leverage. Redshift does not support indexes and must use table scans. The reporting queries (C) run infrequently, once every four hours, so they can be offloaded to Athena (a serverless engine) instead of incurring a high cost on a provisioned but under-utilized Redshift cluster. Athena's serverless burst capability enables the up to 2× decrease

in query latency (C). These queries cannot meet the performance targets on Aurora; they would run for over 150 seconds each.

This example shows that an effective design strongly depends on the specifics of the workload and engines, rather than high-level guiding principles (e.g., run transactions on an OLTP engine and analytics on a data warehouse). Here, the conventional wisdom design is about twice as expensive and an order of magnitude slower on some queries than Design 2. An engineer would need an intimate understanding of the engines and workloads to find such a design, possibly spending a lot of time doing so. Moreover, because the best design is workload-dependent, it can change in response to workload shifts, forcing the engineer to redo their work. These repeated design endeavors are unscalable and difficult to get right, underscoring the need for a principled and automated alternative.

3.2.2 BRAD to the Rescue

With BRAD, users no longer manually design and operate their infrastructures. Instead, the virtual data infrastructure managed by BRAD has a user-specified design goal (e.g., minimize cost and keep query latency under 30 seconds), and users simply submit their queries and transactions directly to VDBE endpoints. BRAD uses this design goal to optimize the infrastructure to best run the user's workload—what we call *blueprint planning*. In this chapter, we focus on the models, algorithms, and mechanisms used in BRAD's blueprint planner (Section 3.3). That said, virtualization comes with many more challenges. In the remainder of this section, we briefly outline how BRAD tackles them. We leave a more detailed exploration of this topic for future work.

Data consistency and freshness. In this chapter, we assume an infrastructure with two VDBEs: one for transactional read/write queries, and one for read-only queries. The transactional VDBE is mapped to Aurora and all transactions always run on the latest snapshot of data (i.e., the VDBE has a maximum staleness of zero). BRAD then syncs table replicas (if any) to meet the maximum staleness constraint of the read-only VDBE. This approach provides similar freshness to existing solutions [96]. We leave the automatic realization of virtual infrastructures containing more than these two VDBEs to future work (Section 6.2).

Transformations. Modern data infrastructure designs typically use ELTs [29, 166], meaning that tables are first replicated into a data warehouse and then transformed inside the warehouse using DML statements. BRAD supports this model, as it already syncs table replicas across engines; these transformations would thus run as regular DML statements that modify the logical tables in a VDBE. Running these transformations on a schedule is orthogonal to BRAD and can be handled using an external tool.

SQL dialects and semantics. Different database engines can have different SQL dialects and semantics, meaning the same SQL statement may not be executable on every engine. Currently, we assume that the dialects issued to the VDBEs are supported on at least one of Aurora, Redshift, and Athena. We also assume that BRAD can detect the subset of its engines that can correctly run a given SQL query; BRAD will ensure it only routes the query

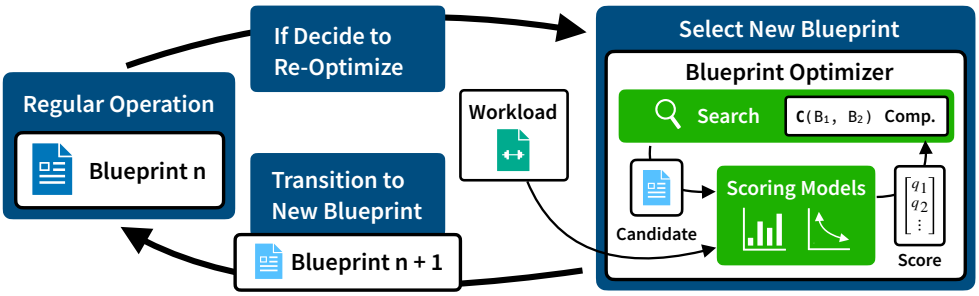


Figure 3.2: BRAD's blueprint planning life cycle.

to those eligible engines (see Section 3.4.3). SQL dialect translation techniques [31] may enable BRAD to expand a query's set of eligible engines; we leave this to future work.

3.3 BRAD's Blueprint Planner: Key Ideas

We now describe the key ideas behind BRAD's blueprint planner. We continue with further implementation details in Section 3.4.

3.3.1 The Blueprint Planning Life Cycle

BRAD automatically designs and operates data infrastructures using a blueprint planning *life cycle*, which we depict in Figure 3.2. The core idea is to select the "best" blueprint for the user's workload, operate the infrastructure according to the blueprint, and then trigger re-optimization if the workload changes (Section 3.4.6). Concretely in BRAD, blueprints are infrastructure plans that contain

- The engines to include in the underlying data infrastructure.
- The provisioning configuration to use for each engine when applicable (e.g., instance type, the number of nodes to use).
- The placement of data tables and replicas on the engines.
- A policy for routing queries to the engines in the infrastructure.

Below, B is an example blueprint describing an infrastructure comprising Aurora (provisioned with one db.r6g.xlarge instance) and Athena. Table T_1 is placed on Aurora, and T_2 is replicated on Aurora and Athena. The routing policy consists of concrete query assignments chosen during blueprint optimization (query q_1 to Aurora and q_2 to Athena) (see Section 3.3.3) and an online policy $P(q)$ that selects an engine for a given query q (see Section 3.3.4).

$$B = \begin{cases} \{\text{Aurora, Athena}\} & \textbf{Engines} \\ \{(\text{Aurora, db.r6g.xlarge, 1})\} & \textbf{Provisioning} \\ \{T_1 \rightarrow \text{Aurora}, T_2 \rightarrow \text{Aurora}, T_2 \rightarrow \text{Athena}\} & \textbf{Placement} \\ \{q_1 \rightarrow \text{Aurora}, q_2 \rightarrow \text{Athena}, P(q)\} & \textbf{Routing} \end{cases}$$

To automatically find an optimized blueprint, BRAD needs a mechanism to (i) quantify the utility of (i.e., assign a "score" to) candidate blueprints on the user's workload (Section 3.3.2), and (ii) systematically search over the blueprint design space (Section 3.3.3). Here, a workload is a representative (but not necessarily exhaustive) list of expected queries and DML statements along with dataset statistics (e.g., its size). Concretely, BRAD obtains a user's workload by logging their transactions and queries (see Section 3.4.1).

Scoring a blueprint is challenging because multiple factors influence a blueprint's utility (e.g., performance, cost), and different users may have different design goals (e.g., maximizing performance vs. minimizing cost). Consequently, BRAD assigns each candidate a *vector score* comprising three components:

1. **Workload Performance.** BRAD predicts the run time of the queries and transactions in the workload on the blueprint.

2. **Operating Cost.** The monetary cost of operating the data infrastructure and routing policy specified by the blueprint.

3. **Transitions.** The time and monetary cost of *transitioning* the underlying infrastructure to the candidate blueprint.

For example, a vector score would look like $[q_1\ q_2\ \ldots\ c\ t_T\ c_T]^\intercal$ where the q_is are predicted query and transaction latencies, c is the operating cost, and t_T and c_T are the transition time and cost. BRAD uses a set of learned models to assign values to all three components, which we discuss next in Section 3.3.2.

Users express their "design goals" to BRAD by providing a comparator function that ranks the vector scores (Section 3.4.5), analogous to the comparators used in sort routines [56]. For example, one such goal could be to design an infrastructure that minimizes cost while maintaining a performance constraint (e.g., a latency SLO, see Section 3.4.5). The comparator would therefore rank blueprints by their operating cost while treating blueprints that are predicted to not meet the latency constraint as having an infinite cost.

3.3.2 Blueprint Scoring

In the blueprint's score vector, the main challenge is predicting the performance of the workload on the blueprint. For BRAD, this means estimating the latencies of the queries in the workload on each of BRAD's engines while taking into account their provisioning and load. We discuss scoring in more depth in Section 3.4.2.

Query Run Times

Prior work has proposed run time prediction methods for use in query optimization [173], workload scheduling [187], resource management [157], and maintaining SLOs [53]. These methods require the query's physical execution plan as input. For example, DBMS cost models and traditional predictors use hand-derived heuristics to understand the cost of each physical operator [10, 68, 110, 193]. Advanced methods featurize the physical query plans and train deep learning models to predict their run time [87, 116, 117, 119, 173, 197]. BRAD cannot directly use these methods because it cannot always get a physical plan. For example, BRAD may need to predict the run time of a query on an engine that is not running or does not have the relevant data loaded (e.g., to decide whether to start Redshift and/or move a table there). BRAD must also account for the effects of provisioning and system load.

BRAD addresses these challenges using a graph neural network (GNN) and two analytical models. We design a new GNN that predicts a query's run time using the query's SQL as input (i.e., relying only on logical features) for an engine on a fixed provisioning in the absence of concurrent queries. Our GNN's novelty is that it featurizes a query based on its SQL text and data properties. In contrast, existing models featurize queries using their physical query plans. We then use two analytical models based on Amdahl's law [30] and queuing theory [85] to adjust this model's estimates for different provisionings and system loads, respectively. We take this approach because making such predictions with a single model is expensive and hard to realize due to the need for diverse run time observations across various query types, provisionings, and system loads. We describe the details of our analytical models in Section 3.4.2; they provide acceptable accuracy and enable BRAD to find effective blueprints (Section 3.5.2).

GNN model and query featurization. We use a GNN model with a novel query featurization that depends only on logical query properties (e.g., the join template, join/filter selectivity) and dataset statistics (e.g., estimated join selectivity). Our design is based on the key observation that most query optimizers are deterministic: they will choose similar query plans with similar run times for queries with similar features and statistics. Thus, we identify these features and then model them with a novel graph structure. As a result, even without physical plans, our model learns the optimizer's behavior and makes accurate predictions for queries similar to the training queries in our featurization space (Section 3.5.3). We use the same approach to predict the amount of data a query scans (to estimate Athena's query cost). We describe the featurization and graph structure in more detail in Section 3.4.2.

Model Bootstrapping

BRAD is designed to be gradually deployed onto an existing infrastructure running one or more of our component engines. When first deployed, BRAD observes the running workload and gathers performance data (e.g., query run times) for each engine in a brief "bootstrapping phase." BRAD then uses this data to train its models. Once complete, BRAD begins to actively optimize the infrastructure using its blueprint planner. Avoiding a bootstrapping phase would require having performance models that are fully transferable across workloads and

datasets, which we leave to future work.

3.3.3 Blueprint Search

Exhaustively searching the entire design space is intractable for most workloads because it is exponentially large with respect to the number of tables and queries. Given this challenge, BRAD must use an efficient search algorithm that finds optimized blueprints without visiting the entire search space.

BRAD uses a greedy beam-based search [115] over the blueprint's routing policy (i.e., query-to-engine mapping), which directly impacts the workload's performance. Blueprints are optimized for a workload which contains a representative list of expected queries. The idea is to incrementally expand a set of top-k blueprints (the "beam") by examining queries in the workload one-by-one. For each blueprint in the current top-k, the planner takes the next query and assigns it to each of the three engines, creating three new candidate blueprints. It then places tables according to these routing decisions (e.g., if a query accessing table T is routed to E, then a copy of T is placed on E). After each step, the planner only keeps the top-k blueprints, and it repeats until all the queries have been assigned. BRAD runs this beam search for each provisioning "near" the current one (in computational resources) and returns the best-scoring blueprint. BRAD uses a beam of size 100 (i.e., $k = 100$). We analyze and discuss our search algorithm in more detail in Section 3.4.4.

3.3.4 Operating Blueprints: Query Routing

Once BRAD has chosen a blueprint, the final step is to use it to serve the workload. To route queries, BRAD first consults the query-to-engine assignment in its blueprint. If the query is in the assignment, BRAD uses this pre-planned routing decision. Otherwise, it uses an online routing policy. The key challenge in designing this policy is that it must make intelligent routing decisions without imposing an undue overhead (i.e., doing so within tens of milliseconds). This efficiency constraint precludes the use of computationally expensive models, such as the query run time model we use for blueprint scoring. To address this challenge, we make two key observations. First, in tasks like run time prediction, prior work showed that classification is easier than regression in terms of model efficiency and accuracy [72, 206]. Thus, we can cast this routing problem as a classification problem and leverage a lightweight classifier. Second, this online routing policy can be trained during blueprint planning, which runs off of the critical path. This workflow gives us the opportunity to bootstrap the online routing policy using a more sophisticated but computationally expensive model, e.g., our query run time model.

We leverage these observations to design BRAD's online routing policy. BRAD's online policy is a decision forest that, for a given query, produces a ranking of the engines (most preferred routing to least). BRAD routes the query to the highest-ranked engine that has all the tables the query accesses. As input, the forest takes the estimated scan cardinality of each table that the query accesses; these cardinalities can be computed using an off-the-shelf

cardinality estimator. The forest is trained as the final step in blueprint optimization using run times that our query run time model predicts (the engine with the lowest predicted run time is the most preferred). Inference over the decision forest is fast and does not impose an undue overhead on the queries. We empirically evaluate the effectiveness and overhead of our routing policy in Section 3.5.4. We discuss additional practical details for query routing in Section 3.4.3.

3.4 BRAD's Blueprint Planner: Details

In this section we outline the implementation details behind BRAD and provide additional details about BRAD's blueprint planner.

3.4.1 Realizing the Blueprint Planning Life Cycle

Figure 3.3 depicts BRAD's system architecture, which implements the blueprint planning life cycle using two components: (i) a front-end server responsible for interfacing with clients and operating the blueprint, and (ii) a blueprint planner that monitors the workload and selects new blueprints when appropriate. We describe the architecture by walking through the blueprint planning life cycle.

The life cycle begins at BRAD's front-end server (Figure 3.3 (A)). Users submit SQL queries to the server, which get routed to a suitable engine for execution (Section 3.3.4). Crucially, to keep track of the executing workload, the front end (i) logs the queries it receives (B), and (ii) collects metrics about the workload (e.g., transaction latency, query latency). The blueprint planner monitors these metrics (C), alongside others it retrieves from the underlying engines (e.g., CPU utilization) and triggers blueprint optimization when they exceed or fall below specified thresholds (D) (Section 3.4.6).

When starting blueprint optimization, BRAD first extracts the queries that ran during its planning window (a sliding window of a configurable length (E)) from its workload log. These queries, along with dataset statistics (e.g., the sizes of the tables), are passed to the optimizer and represent the workload that BRAD uses when scoring candidate blueprints (F). BRAD's optimizer then searches over valid blueprints (Section 3.3.3), scores them (Section 3.4.2), and returns the best-scoring blueprint (G) (Section 3.4.5). The planner then transitions the infrastructure to the chosen blueprint and passes it to the front end (H). The front end uses this blueprint until the next one is chosen, completing the blueprint planning life cycle.

3.4.2 Additional Blueprint Scoring Details

Query Run Time and Data Scanned

As discussed in Section 3.3.2, BRAD uses a graph neural network with a novel query featurization to predict a query's run time and the amount of data it scans. We now describe

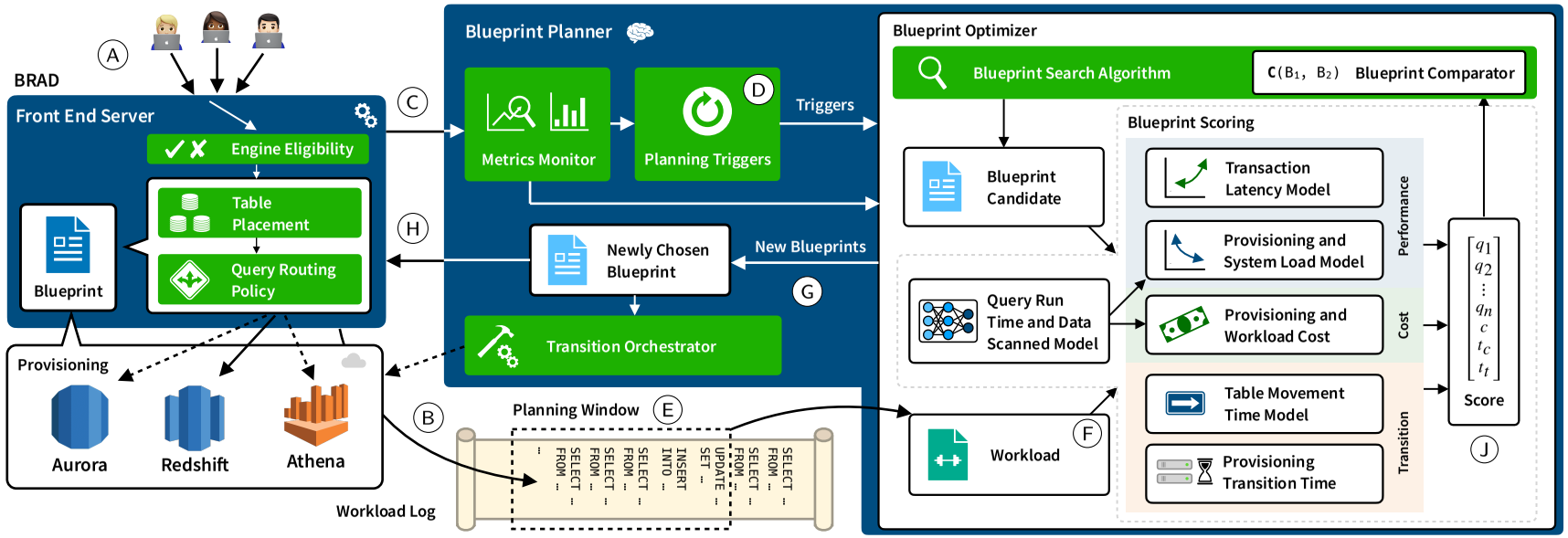


Figure 3.3: A detailed view of BRAD's architecture and its end-to-end blueprint planning life cycle.

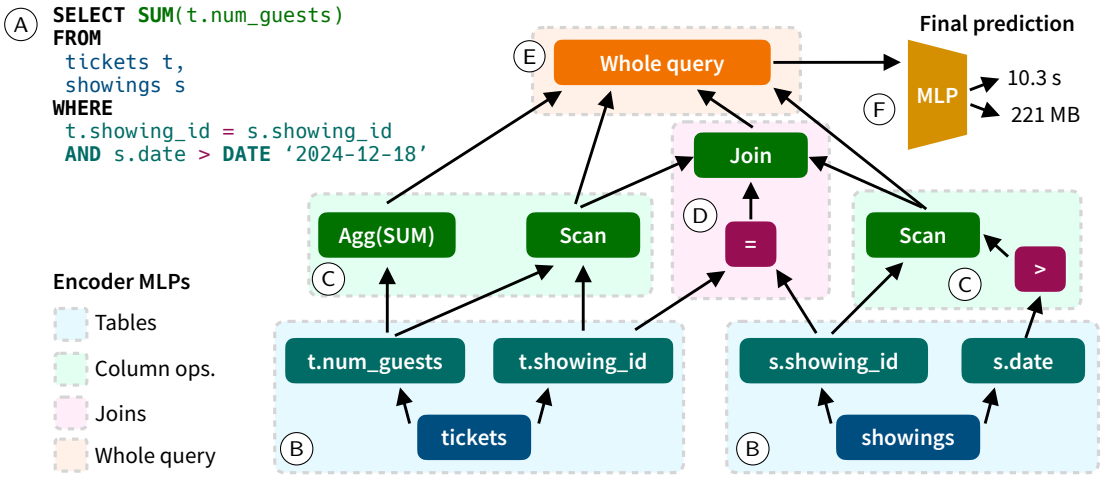


Figure 3.4: An example query featurization for our GNN model, which predicts a query's run time and the amount of data it scans. Table 3.1 lists the features we use.

Table 3.1: The features our model uses for each node type.

Node type	Features
Table	Number of rows, number of columns, storage size, storage format
Column	Width (bytes), data type, null fraction, number of distinct values, is primary key, is foreign key, is partition key, is sorted, index type
Predicate	Operator type (e.g., AND, OR, <, =, etc.), literal complexity
Operation	Operator type (e.g., scan, join, aggregate), width (bytes), estimated selectivity, join type (e.g., inner, left), aggregation type (e.g., SUM, AVG)
Whole query	Number of tables, columns, predicates, and operations

the featurization and model in more detail.

Query featurization. As discussed, existing run time predictors require the query's physical execution plan as input, which is not always available in BRAD's setting (Section 3.3.2). Thus, we design a new graph featurization approach to encode information, such as filters, joins, and group-bys, purely from a query's SQL. This logical featurization differentiates our GNN from existing models [87, 116, 117, 119, 173, 197]. Figure 3.4 shows an example procedure for constructing such a query feature graph. It has five types of nodes: table, column, predicate, operation, and the whole query, each encoded using the features in Table 3.1. The feature graph's edges represent the dependencies between these nodes. We parse the query's SQL to extract the tables, columns, predicates, and logical operations it involves and construct the feature graph in four steps.

First, we extract the tables and columns involved in the query. Consider the query in Figure 3.4 (A). It accesses the `tickets` table (columns `num_guests` and `showing_id`) and `showings` table (columns `showing_id` and `date`). Consequently, we create a table node for

each table and a column node connecting to a table node for each column (Figure 3.4 (B)). Second, we extract the operations on a single table, such as "scan" and "aggregate" (green background in Figure 3.4 (C)). We create a predicate node for each filter predicate and connect the column nodes involved in this predicate to it. We extract the features of the predicate nodes using an approach similar to recent work [87, 173]. We connect these predicate nodes to their parent operation node. For operations without a predicate, such as "aggregate", we just connect them to the relevant column nodes. Third, we parse the operations involving multiple tables, e.g., "join" (purple background in Figure 3.4 (D)). The children of each join operation node are the relevant scan operation nodes and the join predicate node extracted the same way as a filter predicate. Finally, as highlighted in orange in Figure 3.4 (E), all the operation nodes are connected to a top-level embedding node, which aggregates the overall information.

This generic query graph representation does not encode any information about a physical plan (e.g., join order or physical operators). The features we use can all be independently derived without relying on BRAD's underlying engines. We use a value of -1 for the aforementioned node features if the feature is not applicable.

Selectivity estimates. Our model also uses an estimate of the selectivity of each operation node. This is because the selectivity influences the chosen physical plan, and thus the query's run time as well. BRAD collects simple statistics about each table (e.g., histograms) using an analysis pass over the underlying data. BRAD then uses the Selinger method [160] to make these estimates because of its simplicity and negligible overhead. The choice of estimator is orthogonal to our model; other methods are also applicable [130, 196].

Graph neural network. Inspired by the zero-shot run time predictor [87], we use a graph neural network to model interactions between nodes and to propagate information through the feature graph. We construct one multi-layer perceptron (MLP) node encoder for each level that embeds the node features into a fixed-length vector. Then, we create another MLP for message passing through edges [74]. At each internal node, we sum its children's embedded vectors, concatenate them with its own vector, and feed the resulting vector to the MLP to get a new vector. The message passing stops at the last level when the "embedding node" has aggregated all of the query's information into a single vector. This vector is fed to an MLP (Figure 3.4 (F)) to predict the query's run time and amount of data scanned. The entire model is trained end-to-end.

Discussion. By using logical features and selectivity estimates, our model implicitly learns a query optimizer's behavior and makes accurate predictions for queries similar to our training queries (see Section 3.5.3). Our model may not work well in some cases where the testing queries are *significantly* different from our training queries. For example, suppose the model was only trained on short-running queries joining small tables and the user then submits a long-running query joining large tables. Then, the engine could possibly choose a join operator/order that it has never chosen before, and our model (and possibly all existing run time models) would be unlikely to correctly predict that query's run time. However, recent work shows that in practical workload traces, a large portion of queries are repeating

and "similar" to prior queries [195]. Thus, encountering a query that greatly impacts the selected plan in a way that was not captured in the training data should be rare.

Provisioning and System Load

Next, we describe the two analytical models we use to adjust our GNN's run time estimates for different provisionings and system load.

A query's run time consists of two components: the time spent running and the time it queues due to system load. Thus, BRAD models a query's complete run time R as $R(G, \rho) = P(G) + W(\rho)$, where G is the run time given by our GNN, $P(\cdot)$ is a model that accounts for the engine's provisioning, and $W(\cdot)$ is the time spent queuing depending on the system's utilization (load) ρ.

Compute resources. BRAD uses $P(G) = \left(c_1(b/d) + c_2\right) G$, which we derive from Amdahl's law [30]. We model the query's run time as having two parts: (i) one that will decrease (or increase) if the engine's provisioning is changed to have more (or less) compute resources, and (ii) a fixed part that will not change even with more resources. Here $c_1 G$ represents part (i) and is multiplied by b/d, which is the ratio between the resources available on the "base" and "destination" provisionings. The base is the provisioning on which the graph neural network was trained. The destination is the engine's provisioning in the candidate blueprint. The $c_2 G$ term is part (ii). BRAD uses the total number of vCPUs in the provisioning to represent the available compute. We learn one set of constants c_1 and c_2 for each engine (Aurora and Redshift) using least squares linear regression [39] on the query workload. This approach assumes that provisioning changes do not cause significant query plan changes that would affect the run time. Empirically, we find this model to be sufficient for blueprint planning (Section 3.5.2).

System load. BRAD models $W(\rho_r)$ using queuing theory, approximating an engine as an M/M/1 system [85]. We use $W(\rho_r) = -K\left(1-\rho_r\right)^{-1} \log\left(\rho_r^{-1}(1-q)\right)$ which models the q-th percentile queuing time on a system with utilization ρ_r [3]. K represents the mean processing time, which we estimate as the average $P(G)$ for all queries assigned to the engine. We approximate an engine's utilization using its measured CPU utilization. In our experiments, BRAD optimizes for a p90 latency constraint, so we use $q = 0.9$. We use this model as it provides a simple closed-form expression for wait time. Not all engines are M/M/1 systems; for example, the query arrival distribution may not be exponential and/or the engine may process queries in parallel. However, we empirically find that this simple model is sufficient for blueprint planning (Section 3.5.2).

Adjusting ρ_r. $W(\rho_r)$ relies on a representative ρ_r. BRAD cannot directly use CPU utilization because the candidate blueprint may use a different query routing from the current blueprint, thereby imposing different loads. Instead, we assume that a query's "load" is proportional to its run time. BRAD thus scales its observed CPU utilization by a factor: the sum of the run times of the queries routed to the engine in the candidate blueprint divided by the sum of the run times that actually ran on the engine in the last planning window. If no queries ran

on the engine in the previous planning window (e.g., the engine was paused), BRAD scales the query's predicted run times to a load value using a learned proportionality constant.

Transaction Latency

BRAD estimates transactional latency on a candidate blueprint to ensure it provisions Aurora appropriately for the transactional load it experiences. In general, estimating a transaction's run time is a hard problem due to the many factors that can influence its latency (e.g., lock contention, buffer pool state, etc.) [128]. We make a simplifying assumption that the transactional workload is uncontended and consists of TPC-C-like [179] indexed point reads and writes made interactively over the network. For this setting, we use an analytical function that models a transaction's latency as a function of system utilization: $R(\rho_t) = a/(M-\rho_t)+b$. Here, $R(\rho_t)$ is the overall transactional latency. a, b, and M are workload-specific learned constants. $\rho_t \in [0,1]$ is the system utilization; we require that $M > \rho_t$. We use CPU utilization as a simple proxy metric for ρ_t. This model captures that transaction latency increases rapidly as ρ_t approaches M, like it would on an overloaded system [85]. We derive this model empirically; it is loosely based on the queuing theory equations for an M/M/1 system [85].

Adjusting ρ_t. The candidate blueprint's ρ_t may not be the same as the measured ρ_t on the current blueprint (e.g., due to a changed provisioning and/or query routing). BRAD applies two scaling factors to compensate. First, it multiplies ρ_t by the ratio of vCPUs between the current blueprint's provisioning and the candidate blueprint's provisioning. Second, it applies the same query load scaling factor mentioned in Section 3.4.2 to account for query movement.

Operating Cost and Transitions

A blueprint's cost comprises provisioning costs, Athena query costs, and storage costs. For Aurora and Redshift, BRAD uses AWS' on-demand instance pricing [17, 20]. Currently BRAD only considers Aurora I/O optimized instances, which do not bill I/O usage [13]. Since Athena bills by the amount of data scanned, BRAD uses its data scanned predictions (Section 3.4.2) along with Athena's scan pricing [15]. For storage costs, BRAD models a table's size as $k|T|$ where $|T|$ is the number of rows in the table and k is an engine and table-specific constant. To compare blueprints, BRAD normalizes costs to be in $/hour.

Table movement. BRAD currently moves tables via S3 (i.e., export to S3 and import from S3). It estimates the movement time as $S/k_e + S/k_i$, where S is the physical size of the table and k_e and k_i are empirically measured export and import rates. These rates are engine-specific but independent of the engine provisioning, which we confirmed empirically. AWS does not charge for S3 transfers between AWS services in the same region, so BRAD does not incur movement costs.

Aurora provisioning time. BRAD estimates Aurora's provisioning time as the number of instance changes multiplied by a fixed amount of time (we empirically measured 5 minutes).

Removing replicas is considered to take no time since BRAD does not need to wait for the completion of removal to start using the next blueprint.

Redshift provisioning time. The time it takes to complete a Redshift provisioning change depends on whether it can be done using an elastic resize or not [26]. For elastic resizes, BRAD uses AWS' published estimate of 15 minutes [26]. BRAD estimates classic resizes to take $|R|/k$ where $|R|$ is the physical size of the data in the Redshift cluster. We empirically observed k to be approximately 18 megabytes per second. Pausing Redshift is also considered to take no time for the same reason as Aurora replica removals.

3.4.3 Additional Query Routing Considerations

Upon receiving a query, BRAD selects a suitable engine in two steps: (i) determine the set of engines that are *able* to execute the query, and then (ii) select the most suitable (e.g., best performing) engine from this set. In this section, we describe step (i). For (ii), BRAD uses the online routing policy described in Section 3.3.4.

In BRAD, an engine's eligibility to run a query depends on the *table placement* and its *functionality support*. Table placement is the set of engines that hold a copy of a table and is governed by the blueprint; BRAD currently only routes a query to an engine if it has a copy of every table the query accesses. BRAD parses the SQL query to extract the tables it references and compares them against the blueprint's table placement. During blueprint planning, BRAD ensures that all tables are placed together on at least one engine so that it can always run a query that joins any subset of tables.

A query may also use specialized functionality only available on a subset of the engines (e.g., vector similarity search [109, 144]). By taking functionality into account, BRAD ensures that it only selects engines that can run the query. Automatically determining the "specialized functionality" that a query uses is something we leave to future work. BRAD currently uses keyword matching against pre-specified keyword lists to determine if a query uses such functionality (e.g., the presence of the <=> operator would imply that the query uses vector similarity search).

3.4.4 Additional Blueprint Search Details

Algorithm 1 contains the pseudocode for BRAD's greedy beam blueprint search algorithm, which we outline in Section 3.3.3. The intuition for using a top-k beam search instead of a naïve greedy search lies in the nature of the search space. At the beginning of the search, assigning queries to some engines may be better (e.g., prefer Athena, which is pay-per-query, instead of Redshift where you pay for provisioning). But after assigning more queries, this trade-off changes (e.g., there are enough queries to justify running Redshift). Keeping a set of promising candidates helps BRAD balance these changing trade-offs. We search over nearby provisionings because BRAD handles gradual workload changes; the next best provisioning is likely to be near the current provisioning.

Discussion. Beam search works well empirically in our setting for two reasons. First, BRAD uses a large beam ($k = 100$), which helps prevent some promising candidates from being eliminated too early. Second, the queries in our workload have a skewed arrival frequency (i.e., some queries arrive more frequently than others). This property was also observed by our industrial partners in their real-world workloads [195]. As a result, BRAD processes queries in decreasing order of arrival frequency. Along with using a large beam, this processing strategy makes it more likely for "important" (i.e., frequently occurring) queries to be assigned to the best engine.

Analysis. Let m be the number of engines considered, q be the number of queries in the workload, and p be the number of distinct provisionings considered. This algorithm considers $O(kmqp)$ candidate blueprints. Currently, BRAD has $m = 3$ and uses $k = 100$. We evaluate our algorithm empirically in Section 3.5.5.

3.4.5 Blueprint Comparator: Minimizing Cost

As discussed in Section 3.3.1, end-users need to define a comparator function, which imposes an ordering on blueprint vector scores. This comparator is how users convey their infrastructure design goals to BRAD. A common goal is to minimize an infrastructure's operating costs while maintaining a service level objective (SLO) (e.g., p90 query latency should be under 30 seconds). We use this design goal when evaluating BRAD in Section 3.5. We now describe how this goal is implemented as a comparator.

Given two blueprints (B_1, B_2), the general idea is to map their vector scores to scalar costs (W_1, W_2); the candidate with the lower cost is considered better. A simple mapping would be to use the blueprint's operating cost when the predicted query latency falls under the desired latency constraint and an infinite cost otherwise (to indicate infeasibility). However, this mapping does not consider the time and cost of transitioning to the candidate. Instead, our approach is to weigh the cost of operating each blueprint using the transition time T_T and a user-defined "benefit period" T_B. This period represents how long the user expects the workload to "benefit" from the new blueprint. Concretely, we use

$$W = P^{\gamma} C_0 T_T + C_T + C T_B \qquad P = 1 + \max\left(t/t_{\text{SLO}}, q/q_{\text{SLO}}\right)$$

C_0 represents the current blueprint's operating cost, C is the candidate blueprint's predicted operating cost and C_T is the transition cost. $P \in [1, \infty)$ is a penalty multiplier that grows as the current blueprint approaches and exceeds the performance constraints (e.g., because the workload changes). γ is a user-chosen weight (we use $\gamma = 2$). t and q represent the transaction and analytical latency measured on the current blueprint; t_{SLO} and q_{SLO} represent the user-specified performance constraints for these values. Users can declare multiple such constraints (e.g., for different queries).

When the current blueprint exceeds the performance constraints, the first term in the equation on the left will dominate. Thus BRAD will prefer candidate blueprints that are faster to transition to. Otherwise, BRAD weighs the operating costs by the time spent transitioning

Algorithm 1 BRAD's greedy beam blueprint search algorithm.

Input: B_0: Current blueprint, W: Workload,
Score$(\cdot,\cdot,\cdot)$: Blueprint scoring function
Output: B^*: Best found blueprint

```
function DoSearch(P: Provisioning)
    Sort queries in W in decreasing order of arrival probability and then largest predicted speedup
    across engines
    ▷ Initial blueprint with provisioning P and no routed queries ◁
    T ← [(B(P, ∅), Score(B(P, ∅), B_0, W))]
    for all queries q in W do
        T' ← [ ]
        for all blueprints B in T do
            for all engines e in {Aurora, Athena, Redshift} do
                B' ← (B ∪ {q → e})                          ▷ Route query q to e in B'
                if B' is valid then
                    T' ← T' add (B', Score(B', B_0, W))
        ▷ Note that T' is implemented as a top-k heap. ◁
        T ← T' truncated to the top-k candidates
    return Best candidate in T
B* ← None
for all provisionings P near the provisioning in B_0 do
    B ← DoSearch(P)
    if B is better than B* then
        B* ← B
output B*
```

versus running the new blueprint. This means BRAD will still consider blueprints requiring very expensive transitions (high T_T) but will only select them if their benefit is large enough (low C during T_B). If a candidate blueprint has a predicted transactional or analytical latency greater than t_{SLO} or q_{SLO}, we just assign an infinite cost. If all of the candidates are infeasible, BRAD will ask the user to change their constraints.

3.4.6 Triggering Blueprint Optimization

BRAD periodically checks a set of *triggers* to decide when to initiate blueprint optimization. BRAD initiates blueprint optimization if one of them fires. Concretely, BRAD uses the following triggers:

- **Aurora / Redshift CPU utilization.** BRAD triggers blueprint optimization if they consistently violate preset thresholds (e.g., exceeding 85% or falling below 15% for 10 minutes or more).
- **Transaction and query latency.** When optimizing for cost under a performance SLO, BRAD will trigger re-optimization if these latencies consistently exceed the user's SLOs.

- **Recent provisioning change.** If BRAD selects a new blueprint with a provisioning change, it will trigger re-optimization after a fixed period of time to ensure performance is as expected. This is because BRAD makes conservative provisioning decisions to avoid selecting blueprints that will violate the user's SLOs. Re-optimizing after the new blueprint takes effect gives BRAD an opportunity to revisit its choice after observing the workload on the new blueprint.

3.4.7 Discussion

Blueprint planning practicality. Our blueprint planning framework has three practical benefits. First, blueprints and their scores are *human-interpretable*, making it easy for data engineers to inspect BRAD-chosen designs. Second, blueprints provide a useful abstraction for realizing cloud infrastructure designs. Conceptually, they can be "compiled down" into infrastructure-as-code manifests (e.g., CloudFormation [22]) for deployment. Finally, blueprints are generalizable to other cloud infrastructure design problems that involve cost/performance-based resource provisioning, task scheduling, and adaptation under changing conditions. Some example use cases include selecting resource configurations for Ray [126] programs, designing long-lived infrastructures used by Sky intercloud brokers [47], or assembling a model serving service [155].

Adding engines to BRAD. BRAD can support additional engines beyond Aurora, Redshift, and Athena. For an engine to be included in BRAD, it must (i) support relational data, (ii) have a SQL-based query interface, (iii) expose system metrics (e.g., CPU utilization), (iv) use a deterministic query planner, and (v) have deterministic operational costs. Practically, the engine should also have management APIs that allow BRAD to programmatically alter its allocated resources (i.e., provisioning) to deploy blueprints. By (iv), we mean that the query planner must pick the same physical plan for the same query if it has the same dataset statistics (e.g., estimated scan selectivity) (see Section 3.4.2). Finally, by (v), we mean that the cost of running the engine in the cloud must be a deterministic function of the engine's physical configuration (e.g., its provisioning and the size of its data) and the user's workload. For example, the engine's operating cost cannot depend on external factors that BRAD cannot directly observe (e.g., resource demands from other cloud users).

3.5 Evaluation

In our evaluation, we seek to answer the following questions:

- How effective is BRAD at optimizing a data infrastructure for cost when compared to serverless autoscaling systems? (Section 3.5.2)
- How accurate are the models that BRAD uses to score its candidate blueprints and how well do they generalize? (Section 3.5.3)

- How effective is BRAD's query routing and how much overhead does BRAD add to query execution? (Section 3.5.4)
- How effective is BRAD's blueprint search algorithm? (Section 3.5.5)
- How sensitive is BRAD's blueprint planner to model errors? (Section 3.5.6)

Across five workload scenarios, we find that BRAD selects designs that meet performance targets while outperforming a serverless Aurora and Redshift infrastructure and a serverless HTAP system (where comparable) on cost by up to 13× and 4.6× respectively.

Overall implementation. We implemented BRAD in Python using approximately 30k lines of code. Although BRAD currently uses AWS services, the concepts underlying blueprint planning and our scoring models are general and applicable to other cloud providers.

3.5.1 Workload and Experimental Setup

We evaluate BRAD on a new workload that models the data processing needs of a fictitious movie theater company called *QuickFlix*.

Why create a new workload? BRAD automates the design of multi-engine cloud data infrastructures serving diverse transactional and analytical workloads. Thus, we need a realistic and diverse workload that warrants multiple specialized engines. To our knowledge, no such public workloads exist. The TPC [178, 180] and HATtrick benchmarks [122] are entirely synthetic. IMDb JOB [108] and STATS CEB [84] use real-world datasets and queries, but only contain OLAP queries as they are for evaluating query optimizers. Snowset [186] has statistics about real OLAP workloads, but no data or queries. Our workload addresses these limitations: it (i) contains transactions and diverse analytical queries, (ii) adapts a real-world dataset, and (iii) mimics Snowset statistics where possible.

Dataset. We use an adapted version of the IMDb dataset [108], which is based on real-world data. As the original IMDb dataset is small (3 GB), we create a larger dataset by replicating each tuple in the dataset's major tables 30 times. Then, we assign new values for each replicated primary key and re-assign these values to their corresponding foreign keys. This approach preserves the dataset's attribute correlations, skew, and join-key distributions. We additionally add three synthetic tables representing movie theaters, movie showings, and ticket orders to capture the company's transactional needs. The final uncompressed dataset is 160 GB.

Analytical queries. Our workload consists of two classes of analytical queries: (i) recurring queries (e.g., used for QuickFlix's dashboards and interactive internal tools), and (ii) complex ad-hoc analytical queries (e.g., representing exploratory data analysis). The recurring queries consist of single table scans and two table inner joins, both with predicates. The complex ad-hoc queries are randomly generated to resemble the IMDb JOB queries. They span hundreds of distinct templates that join 4 to 15 tables with complex filter predicates. Of the

unique queries in our workload, 80% are recurring and 20% are ad-hoc; we chose this split to match what our industry partners have observed in their production workloads.

Transactions. We use 3 transaction types: (i) Purchase, (ii) AddShowing, and (iii) UpdateMovie. Purchase looks up a theater, selects a showing, inserts a ticket order, and updates the showing's seat count. AddShowing looks up a theater and movie and inserts a new showing record. UpdateMovie selects a movie from the "title" table and edits the corresponding note column in the "movie_info" and "aka_title" tables. We run these transactions with a breakdown of 70% Purchase, 20% AddShowing, and 10% UpdateMovie.

Baselines. We compare BRAD against (i) an infrastructure using serverless Aurora for all transactions and serverless Redshift for analytics, and (ii) System H, a popular open-source serverless HTAP database. Note that these comparisons are not perfectly fair as these systems provide different guarantees. We select them because they represent existing industry-standard infrastructure solutions that provide the same hands-off autoscaling experience.

Metrics. We record three metrics: (i) transaction latency, (ii) analytical query latency, and (iii) monthly operating cost. In our experiments, BRAD optimizes a data infrastructure to minimize operating cost while ensuring that p90 transaction latency remains under 30 milliseconds and p90 query latency remains under 30 seconds.

Operating cost calculations. We compute BRAD's operating cost using the on-demand instance hourly cost scaled to 30 days. For queries running on Athena, we compute their cost using the reported bytes scanned. We project this value into a monthly cost by assuming that the query repeats at the same observed rate. We include storage costs for the tables placed on Aurora and Athena (S3). For serverless Aurora and Redshift, we use the ACU and RPU values reported by AWS during the workload and scale them to monthly costs. For System H, we use the cost reported by the vendor.

Considered instance types. For Aurora, BRAD currently only considers Graviton-based instances [12] and I/O optimized cluster configurations (i.e., I/O costs are included in the hourly provisioning cost) [13]. We leave the consideration of different instance hardware types (e.g., r6g vs. r6i instances) to future work. For Redshift, BRAD considers the dc2 and ra3 family of instance types.

3.5.2 Optimizing a Data Infrastructure

We have BRAD optimize a data infrastructure for cost under a performance constraint in five scenarios faced by QuickFlix:

1. Scaling down an over-provisioned infrastructure.
2. Scaling engines due to increased load.
3. Maintaining support for specialized functionality.

4. Adjusting to user-changed constraints.

5. Workload intensity variations during a day.

Scaling Down an Over-Provisioned Infrastructure

QuickFlix has been struggling with their data infrastructure. After learning about BRAD, they adopt it to free up their data engineers. QuickFlix deploys BRAD on their infrastructure, consisting of two Aurora db.r6g.xlarge instances (primary and read replica) and two dc2.large Redshift nodes. Following conventional wisdom, they use Redshift for analytical queries and Aurora for transactions. Figure 3.5 shows workload performance and the monthly operating cost over time. The shaded area indicates QuickFlix's performance constraints: 30 ms p90 transaction latency and 30 s p90 analytics latency.

Soon after starting, BRAD triggers blueprint optimization (A) because it detects a low Redshift CPU utilization. BRAD removes the Aurora read replica, shifts the entire analytical workload onto Aurora, and pauses Redshift. BRAD makes these changes to reduce cost (B), as it correctly predicts that Aurora is sufficient to handle the workload. After observing the workload on this new blueprint, BRAD then correctly predicts that Aurora can support the workload with a smaller (cheaper) instance type and thus downscales Aurora to two db.t4g.medium instances to further reduce cost (D). The momentary spike in p90 transactional latency is due to the Aurora primary failover that occurs when changing instance types (C). The chosen blueprint meets QuickFlix's performance constraints (E) (F). From these results, we draw the following two conclusions.

BRAD reduces operating cost by 6.0× over its starting provisioning and by 4.6× over System H, the next best baseline. The serverless Aurora and Redshift baseline is 13× more expensive than BRAD's chosen design because serverless Redshift has a large minimum size, making it cost-ineffective on this workload. Although System H is only 4.6× more expensive than BRAD, its p90 transaction latency is nearly 100× higher than the other baseline. We hypothesize that this is due to System H's internal replication on writes.

BRAD shifts workloads across engines to reduce cost, differentiating it from static multi-engine autoscaling infrastructures. BRAD correctly predicts that Aurora can support the analytical workload, enabling it to pause Redshift to reduce cost. This decision would never be considered in static autoscaling infrastructures, such as our serverless Aurora and Redshift baseline, since they only scale to respond to system load while keeping the workload assignment fixed (i.e., analytics always run on Redshift).

Scaling Engines Due to Increased Load

As QuickFlix grows, their transactional load increases. Figure 3.6 shows how BRAD handles this change; we plot the same metrics as before and include the number of transactional clients over time (A). We begin with the same blueprint as the end of the previous scenario. After a few minutes, BRAD notices that the transaction p90 latency is exceeding QuickFlix's 30 ms ceiling (B) and triggers blueprint optimization. BRAD chooses to scale up Aurora

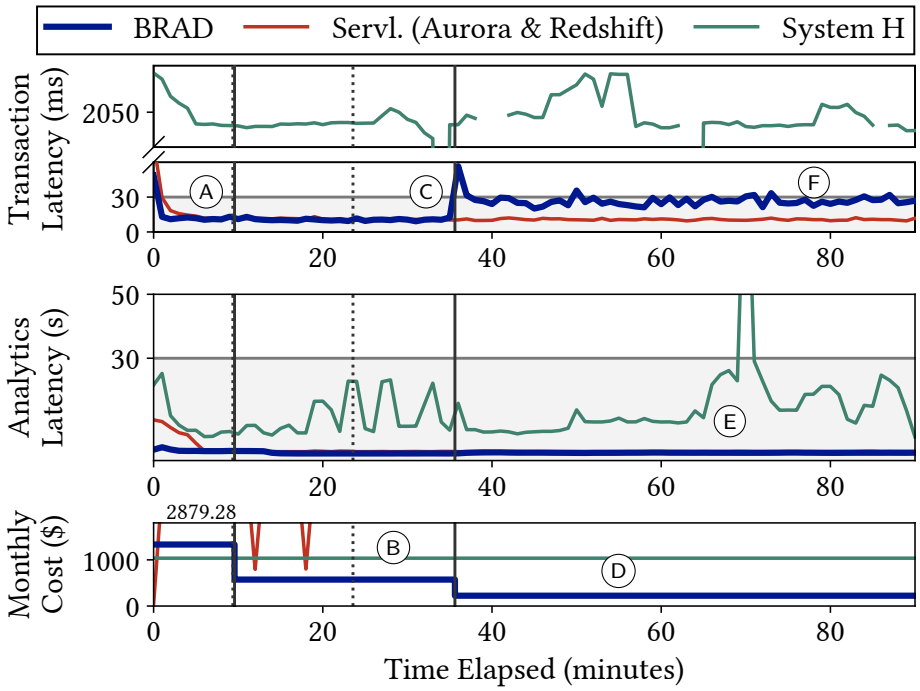


Figure 3.5: BRAD reduces cost while maintaining p90 latency constraints (shaded region). The dotted (solid) vertical lines indicate when a new blueprint is chosen (takes effect).

to a single db.r6g.xlarge instance as it correctly predicts that a single Aurora instance can support both the increased transactional load and the running analytical queries. The spike in p90 transactional latency (C) is when the Aurora primary failover occurs. On the new blueprint, the transactional p90 latency falls under the latency ceiling (E); the analytical queries also continue to complete under the 30 s ceiling (F). Again, the increased System H analytics latency might be due to a combination of its internal physical autoscaling and storage writes.

This shows that BRAD reacts to transactional load to maintain latency constraints. The blueprint that BRAD selects is 2.6× cheaper than the serverless Aurora and Redshift baseline (D) but up to 1.1× more expensive than System H. Serverless Aurora and Redshift is more expensive due to Redshift's large minimum size.

Maintaining Support for Specialized Functionality

To increase engagement, QuickFlix decides to deploy a new feature that recommends movies similar to the ones shown in their theaters. To make recommendations, they use *vector similarity search* [109, 144] queries that find movies with title embeddings that are closest to a given movie's title embedding. QuickFlix deploys this feature on their existing infrastructure; since similarity search is only supported on Aurora, they place the relevant tables on Aurora and run all other queries that access these tables on Aurora as well. They use two Aurora db.r6g.2xlarge instances and two dc2.large Redshift nodes.

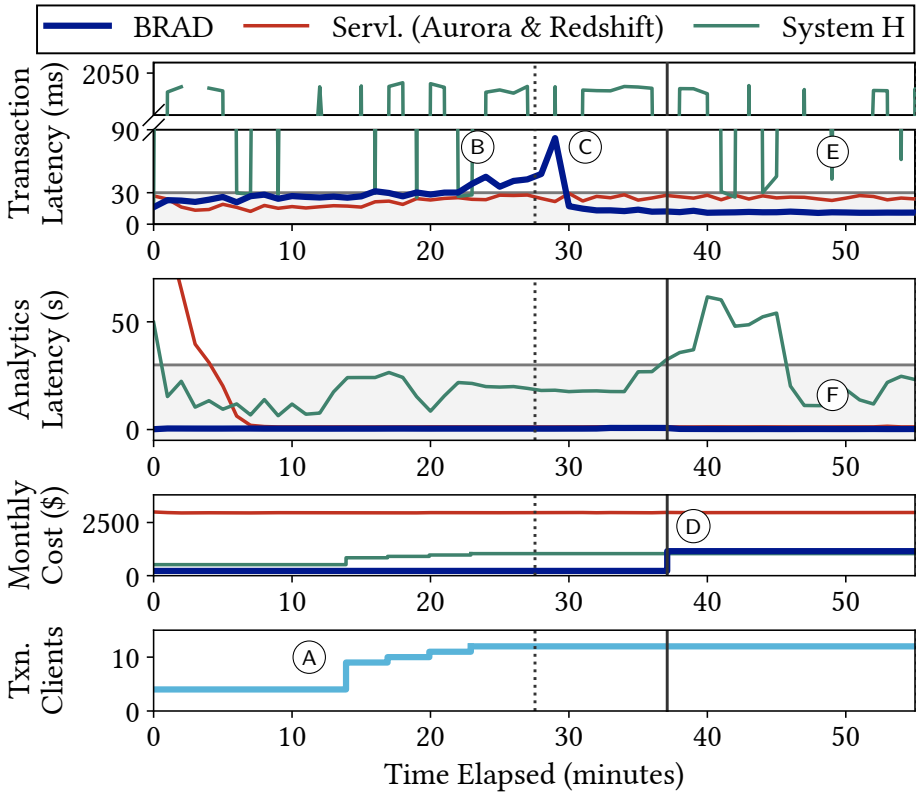


Figure 3.6: As transactional load increases, BRAD switches to a larger Aurora instance and also removes the read replica.

Figure 3.7 shows performance and infrastructure cost over time. The dashed lines are from before BRAD deploys its first optimized blueprint. Crucially, System H *cannot* run this workload because it does not support vector similarity search (A). BRAD notices that the analytical p90 latency exceeds QuickFlix's constraint of 30 s (B) and initiates blueprint optimization. BRAD selects a blueprint that shifts the non-vector similarity search queries onto Redshift (replicating the tables they access onto Redshift) while keeping the vector similarity search queries on Aurora. It also correctly predicts that it can downscale Aurora (to a db.r6g.xlarge instance) to save cost, as much of Aurora's former query load was moved onto Redshift. After making this change, the workload's performance falls within the user's performance constraints (D) (E). The momentary spike in analytics latency at the 40-minute mark (C) is due to a cold Redshift cache when BRAD first moves queries onto Redshift. The serverless Aurora and Redshift design is 2.4× more expensive (F) because of the large Redshift minimum size and because Aurora has scaled up to support the new similarity search queries.

This scenario shows how BRAD is fundamentally different from single-system (e.g., HTAP) solutions like System H. When using a single system to run a diverse data workload, you can always run into situations where you want to use a feature that does not exist on your system of choice. In contrast, in the BRAD architecture, one can (in concept) always *incorporate* a system with the required functionality into the underlying infrastructure.

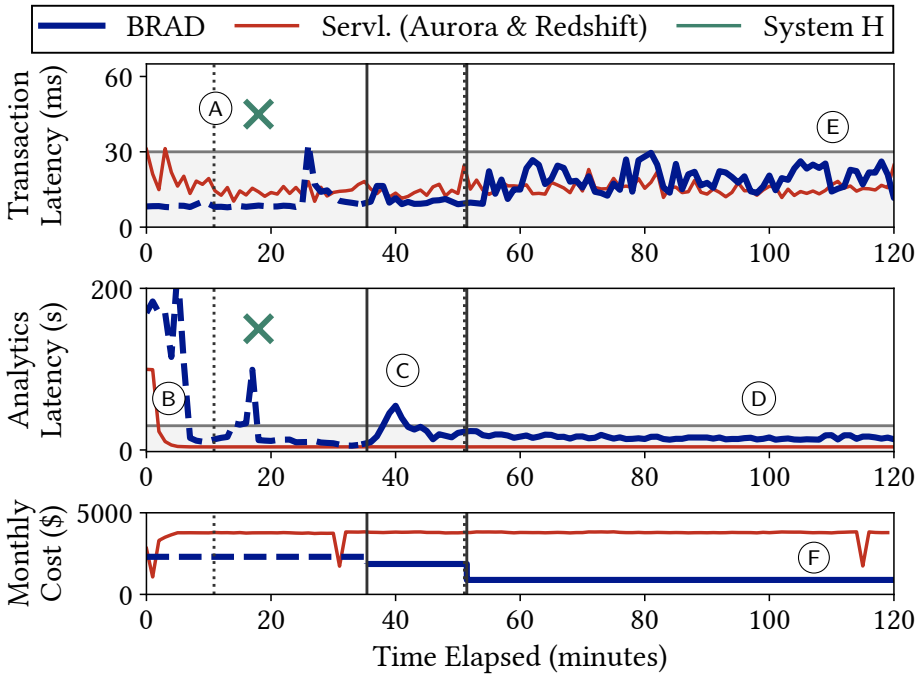


Figure 3.7: BRAD runs vector similarity search on Aurora and shifts other queries to Redshift for performance. System H does not support vector similarity search.

Adjusting to Changed Constraints

As QuickFlix's business changes, they revise their performance constraints; Figure 3.8 shows how BRAD adapts to their new needs. QuickFlix initially uses p90 transaction and query SLOs of 40 ms and 40 s respectively (A) (B). BRAD starts with one db.r6g.xlarge Aurora instance and two Redshift dc2.large nodes. Later, QuickFlix lowers their transaction and query SLOs to 20 ms and 20 s respectively (the change happens at the dashed line in Figure 3.8 (C)). This SLO change causes the transaction latency to exceed QuickFlix's constraints. Thus, BRAD scales up Aurora to one db.r6g.2xlarge instance (D) and leaves Redshift as-is. This change increases the operating cost (E) as BRAD switches to a larger Aurora instance. After BRAD finishes transitioning the infrastructure, the transaction latency falls under the new 20 ms p90 constraint (F). The query latency also remains under the new 20 s p90 constraint (G). BRAD's operating costs are 4.2× lower than the serverless Aurora and Redshift baseline. This is because serverless Redshift has a large minimum size. While System H ends with a 4.4× lower monthly operating cost than BRAD, it does not meet the 20 ms transaction latency constraint (its transactions have a latency around 2 seconds). We again think that this elevated latency is due to System H's internal replication on writes. This scenario shows that BRAD adapts to changes to a user's constraints.

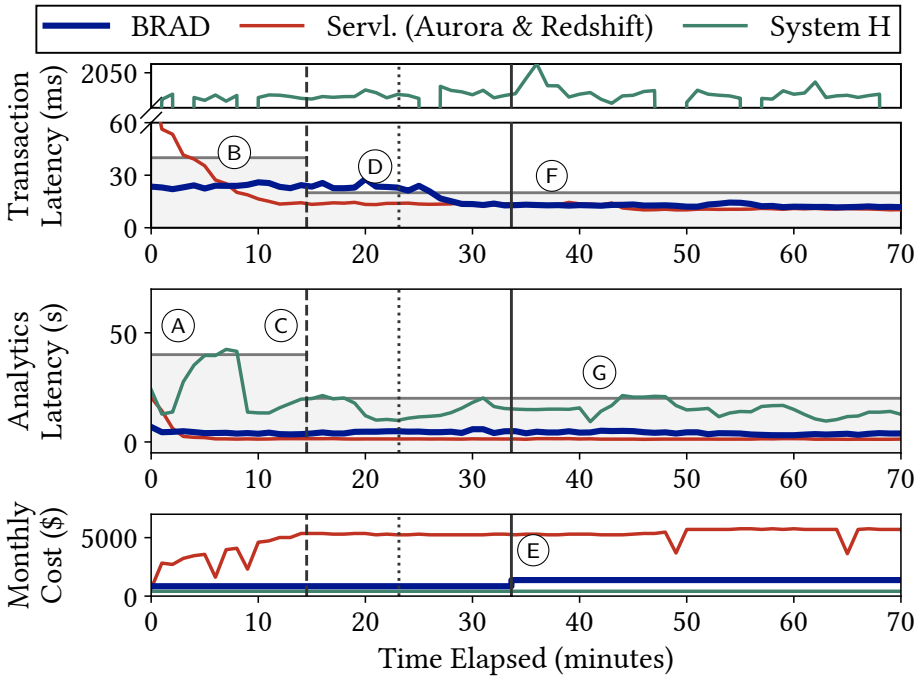


Figure 3.8: After the user changes their SLO constraints, BRAD selects a new blueprint to meet the new constraints.

Workload Intensity Variations During a Day

Finally, we run BRAD on a workload representing a full day at QuickFlix. For practical reasons, we scale the actual workload to 12 hours. Figure 3.9 shows performance and cost over the day. We use the workload and dataset from Section 3.5.1 adapted to mimic the Snowset trace [186]. Concretely, we run queries with a run time distribution similar to the Snowset trace and vary the number of clients issuing queries and transactions to mimic the diurnal pattern observed in Snowset (a peak near the middle of the day, Figure 3.9 (F)).

Initially, the workload is light. BRAD uses a blueprint with four dc2.large Redshift nodes and two Aurora db.t4g.medium instances, which is 2.5× cheaper than the serverless Aurora and Redshift baseline (A). As the workload intensity increases, BRAD detects the increases in latency (B) (C) and triggers blueprint optimization, ultimately scaling Redshift up to 16 nodes and Aurora to one db.r6g.xlarge instance at the peak. The serverless Aurora and Redshift baseline also scales up, but it does not consistently meet the analytics performance target of 30 s (D), despite being 2.1× more expensive than BRAD's design (E) at the workload peak. System H maintains the p90 analytics latency SLO throughout the workload, but its transactional latency is again almost two orders of magnitude higher than the other systems (we hypothesize for the same reasons as discussed earlier). The brief analytics latency spikes are due to Redshift resizes, which force clients to reconnect.

This result shows that BRAD effectively responds to load variations during the day. Over the day, BRAD maintains performance targets while reducing cumulative cost by 1.7× compared to the serverless Aurora and Redshift baseline.

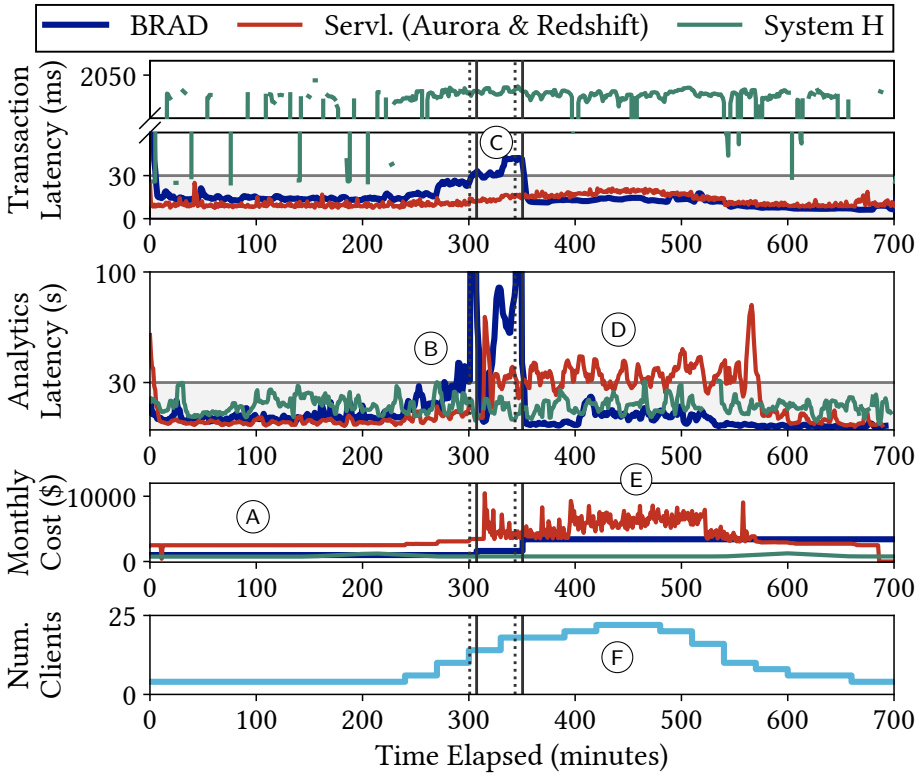


Figure 3.9: BRAD optimizes for load variations during the day.

3.5.3 Scoring: Model Accuracy and Generalization

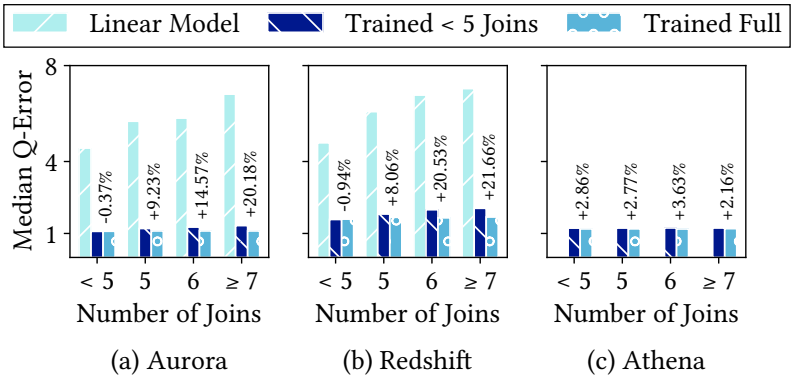
We next examine the test accuracy of our predictive models and their generalizability across common workload shifts.

Model Accuracy

Table 3.2 shows the median test Q-error of our models for each engine. Q-error is $Q(p, a) = \max(p/a, a/p)$, where p refers to the predicted value and a to the actual value. Lower is better; 1 is the best possible score. We train each run time and data accessed model using approximately 8000 queries, validate on 2000 queries, and test on 125 unseen queries. Our query dataset consists of over 1000 unique join templates. The models for Athena perform better than Aurora and Redshift because the run time distribution of Athena queries has a lower variance. We test our provisioning and transaction latency models on an unseen provisioning that is larger (i.e., has more resources) than all the training provisionings. Overall, we find that our models' prediction accuracy is sufficient for BRAD to design effective infrastructures (Section 3.5.2).

Table 3.2: Median test Q-error of our blueprint scoring models.

Prediction target	Aurora	Redshift	Athena
Query run time	1.5769	1.6539	1.3427
Data accessed	–	–	1.2614
Run time on different provisioning	1.6718	1.6824	–
Transaction latency	1.2030	–	–



(a) Aurora (b) Redshift (c) Athena

Figure 3.10: BRAD generalizes to unseen join templates.

Generalizability

We evaluate our query run time model's generalizability on three workload shifts: (i) unseen join templates, (ii) adding a new table to the dataset, and (iii) a larger dataset size.

Unseen join templates. We train our run time models on queries with fewer than 5 joins. We then test the model's predictions on queries with 5, 6, and ≥ 7 joins. Figure 3.10 shows each model's median test Q-error compared with (i) a model trained on all the join templates ("full"), and (ii) a naïve linear model that scales the cost returned by the engine's query optimizer to a run time. We label the percentage difference from the model trained on the full dataset. Our model generalizes across unseen join templates with a Q-error of at most 22% above the model trained on the full dataset. Our model still performs much better than the naïve linear model, which has a Q-error of at least 4.6. Since Athena's optimizer does not provide a query cost, we do not include a linear model result.

Added table. We train our run time models on queries that do not access the "person_info" table and test them only on queries that access the "person_info" table. This table is around 10 GB (the second largest table in the dataset); the overall dataset size is 160 GB. Figure 3.11 shows our results using the same baselines and notation as Figure 3.10. Our model generalizes to an added table with a Q-error at most 22.2% higher than the model trained on the full dataset. The linear model still does poorly, with a Q-error of 4.6.

Increased dataset size. Finally, we evaluate our run time model's generalizability to larger datasets. We train our models using queries executed on a 3 GB, 20 GB, 40 GB, and 60 GB

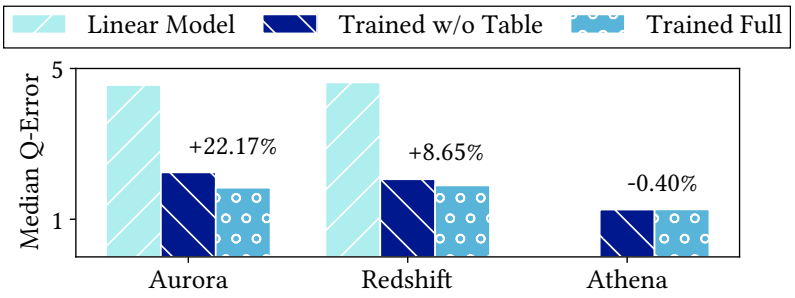


Figure 3.11: BRAD generalizes to an added table.

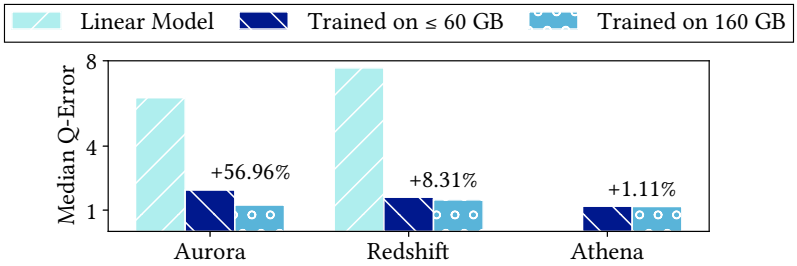


Figure 3.12: BRAD generalizes to an increased dataset size.

version of our workload dataset. Then we test the models on the original 160 GB dataset. Figure 3.12 shows that our model generalizes to the larger dataset with Q-errors 1.1%, 8.3%, and 57% higher than a model specifically trained on the 160 GB dataset on Athena, Redshift, and Aurora respectively. The Aurora model has a higher error due to a change in caching behavior that occurs beyond 60 GB. The linear model has Q-errors of 6.3 and 7.7 on Aurora and Redshift respectively. Overall, these results are sufficient for BRAD since an increase from 60 GB to 160 GB would likely happen over a longer period of time, allowing for BRAD to update its models given newly observed data.

3.5.4 Query Routing Quality and Overhead

Next, we evaluate BRAD's query routing (Section 3.3.4) against four baselines: (i) selecting an engine randomly, (ii) routing all queries to Redshift, (iii) routing using the BRAD run time model (Section 3.4.2) but excluding its inference overhead, and (iv) routing using the BRAD run time model. We use 125 queries from our workload; 80% of the queries represent recurring queries and 20% are complex ad-hoc queries. We report the geomean slowdown relative to optimal routing (i.e., lower is better, 1.0× is optimal). That is, for each query, we divide its run time on the chosen engine by its run time on the fastest engine, and take the geomean across queries.

Figure 3.13 shows our results. BRAD performs the best with a geomean slowdown of 1.31×, comparable to using the run time model and excluding its inference overhead (1.34×). Routing by actually using the run time model (i.e., including its inference overhead)

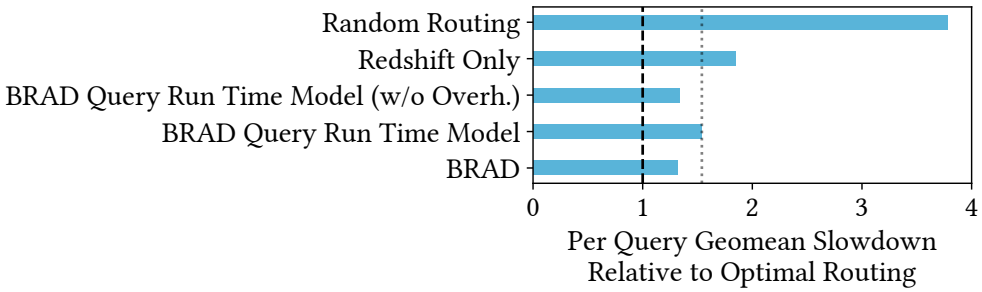


Figure 3.13: BRAD's routing quality is on par with its query run time model but without the inference overhead.

has a geomean slowdown of 1.54×. These results highlight why we do not directly use our run time model for routing; it imposes a high overhead on the query's critical path (up to 115 ms), negatively affecting the routing performance. They also show that routing needs an intelligent strategy; the Random (3.78×) and Redshift Only (1.85×) strategies both perform worse than BRAD.

Overall query processing overhead. Similar to common database proxies [23, 44, 143], BRAD imposes some overhead. We measure a median overhead of 2.46 ms per complete transaction (these are interactive multi-statement transactions) and around 10 ms per analytical query. These could be further reduced, as BRAD is now implemented in Python, but we believe they are reasonable given that BRAD operates in the cloud, serving remote clients.

3.5.5 Blueprint Search Effectiveness

We next evaluate the effectiveness of BRAD's blueprint search algorithm. To compare the search algorithms, we report the final scalar score computed by BRAD's optimizer (Section 3.4.5). All baselines use the same scoring models and optimize for the same workload; they only differ in how they search for candidate blueprints.

We compare against three baselines: (i) uniform random sampling, (ii) naïve greedy, and (iii) exhaustive search. In uniform random sampling, each query is mapped to an engine that is chosen uniformly at random. We repeat this process to sample 10,000 blueprints and select the best one. In naïve greedy, each query is mapped to the engine with the lowest predicted run time (Section 3.4.2) without consideration of any other queries. Finally, exhaustive search looks through all possible mappings and therefore has a run time that is exponential in the number of queries. To be tractable, we use only 12 randomly chosen queries from the IMDb workload, comprising a search space of 530,000 candidates.

We compare the algorithms on the scale-down scenario from Section 3.5.2. Figure 3.14 shows that BRAD's beam search finds a blueprint as good as exhaustive search. The score represents a monetary cost, and so lower is better. BRAD's chosen blueprint has a score that is significantly lower than the blueprints selected through uniform sampling and the naïve greedy approach. This result indicates that BRAD's beam strategy is effective at finding

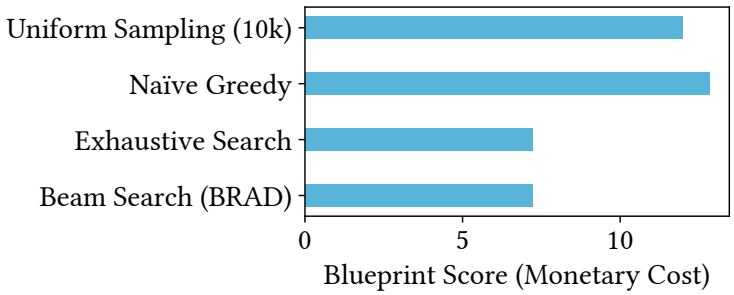


Figure 3.14: BRAD's blueprint planner compared to baselines.

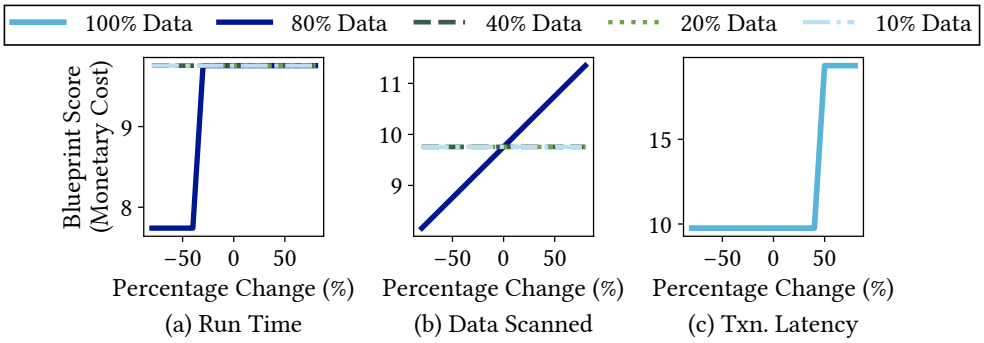


Figure 3.15: BRAD's planner is robust to prediction errors.

optimized blueprints.

3.5.6 Blueprint Planner Sensitivity

Finally, we examine our blueprint planner's sensitivity to prediction errors. We inject errors into BRAD's predictions during blueprint planning and record the selected blueprint's score. Concretely, we select a random subset of the predicted values (query run time, data scanned, and transaction latency) and increase (or decrease) their predicted values by a percentage. We use subsets that include 10%, 20%, 40%, and 80% of the predictions. We run the blueprint planner on the scale-down scenario from Section 3.5.2. Figure 3.15 shows our results. The x-axis is the amount of injected error, which we vary from −80% to +80%. The y-axis is the blueprint's scalar score. We study the effects of prediction error on query run time, data scanned, and transaction latency.

Figure 3.15(a) shows our results for query run time. There is no change in the selected blueprint even when up to 40% of the predicted query run times have injected errors of ±80%. When 80% of the predictions have injected under-predictions greater than 40%, BRAD selects a different (cheaper) blueprint as it predicts that the cheaper blueprint can meet the performance constraints.

Figure 3.15(b) shows results for the predicted amount of data a query scans. Similar to run times, there is no change to the chosen blueprint when up to 40% of the predictions have injected errors. At 80%, BRAD chooses to route queries onto Athena. Thus the blueprint's

monetary cost varies linearly with respect to the injected error.

Figure 3.15(c) shows results for transaction latency. Note that we inject errors into 100% of the data since BRAD makes just one latency prediction. Here, an injected error of +50% causes BRAD to select a larger Aurora instance (to meet the latency constraint), which increases the blueprint score (monetary cost).

Overall, our results indicate that BRAD's planner is robust to prediction errors; more than 40% of the predictions need to have errors outside ±50% for BRAD to choose a different blueprint. The intuition is that blueprints represent coarse-grained design decisions, and thus are more tolerant to prediction errors.

3.6 Conclusion

This chapter presented *blueprints*, *blueprint planning*, and BRAD: a system that virtualizes a cloud data infrastructure and leverages blueprint planning to automatically manage its physical realization. The key takeaway is to cast infrastructure design as a *cost-based optimization problem*, which we refer to as *blueprint planning*. This approach allows us to systematically search for an optimized design for a given workload by leveraging learned models to predict the utility of candidate blueprints. We show that BRAD automatically meets performance targets and achieves 1.6×–13× lower cost compared to existing serverless autoscaling systems.

Chapter 4

Query Accelerators

4.1 Introduction

Relational database management systems (RDBMSes) are widely used, in part, because they provide an expressive and general-purpose way to query structured data. But this generality leaves potential performance on the table. Purpose-built systems [2, 5, 83, 91, 114] are at the other extreme: they might process a specific class of queries significantly faster using a specialized algorithm and/or pre-computed side information, but as a consequence, lack the generality of an RDBMS. Consider the following example.

Specialization example. Consider TPC-H Q13 [178] (Figure 4.1), which computes a customer order count distribution for orders with comments that do not match a pattern. An RDBMS could run this query by filtering `orders` by the selection predicate (`NOT LIKE '%...%'`), hash joining it to `customer`, and then applying two aggregations.

Although this general approach works, we can do better. The filter on `orders` is not selective so the join involves most of the table. If we pre-compute the total (unfiltered) order counts for each customer, we can run the query faster using the *negation* of the predicate instead; we subtract the resulting order counts from the pre-computed totals to get the counts matching the original predicate.[1] Since the negated predicate is selective, fewer rows from `orders` match, leading to a smaller join and thus a faster query. Figure 4.1 compares Redshift's [19] and DuckDB's [65] query run times to this "domain negation" technique, showing 1.6× and 2.5× speedups respectively. Although we must pre-compute the total counts, this extra data is only 0.57% and 0.20% of the total dataset size in the two systems, and it supports any predicate on `orders`. This technique is specific: it only applies to filtered aggregations of numeric expressions. But in exchange, it exploits the query's structure and the underlying data to deliver a speedup. Yet, to our knowledge, RDBMSes such as Redshift and DuckDB do not implement it.

Query accelerators. We call such specialized techniques *query accelerators*: possibly stateful components that can run certain logical query sub-plans faster than a target RDBMS. Crucially,

[1] There are additional computational details related to `NULL` that we elide for clarity in the example.

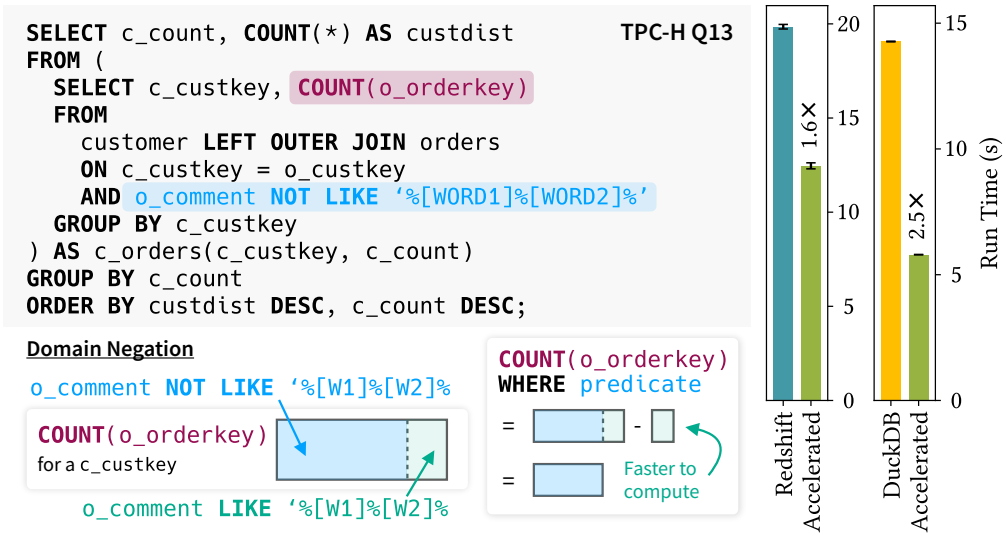


Figure 4.1: We can accelerate TPC-H Q13 by running the negated version of the inner query and *subtracting* its results from the total order counts for each customer, netting 1.6× and 2.5× speedups over Redshift and DuckDB respectively.

accelerators do not have to be capable of executing every possible query plan, giving them the freedom to exploit properties of the logical sub-plan and underlying data for performance.

Integrating accelerators. Accelerators help an RDBMS exploit workload-specific techniques without sacrificing generality. Yet, integrating them into an RDBMS can be impractical, if not impossible. Accelerator builders cannot integrate them directly into closed-source RDBMSes such as Redshift, and doing so in open-source systems requires substantial engine-specific changes across the optimizer, cost model, execution engine, and memory and storage layers. This large "blast radius" makes workload-specific accelerators difficult to upstream and maintain, and forces accelerator builders to re-implement integration logic for each supported RDBMS. Database extensions [99, 149] and UDFs [33, 94, 163] seemingly offer a less invasive alternative, but the mechanisms we know of [64, 149, 163] either (i) provide little to no query optimizer support [50, 189], requiring users to judge when an accelerator will help and explicitly rewrite their queries to use them; or (ii) require code changes to the underlying engine [94]. Extensions and UDFs also couple accelerators to a specific RDBMS, making them hard to reuse elsewhere. What is missing is a portable, non-invasive approach that automatically inserts accelerators into query plans when expected to help.

Tailwind. We introduce Tailwind: a non-invasive query planner and executor for adding query accelerators to any RDBMS that supports data import and export. Accelerator builders declare the logical plan fragments their accelerator can compute and provide an implementation against Tailwind's execution API. Offline, given a representative query log [152, 153] and space budget, Tailwind learns the accelerators' performance, selects accelerator instances to minimize predicted query run time, and manages their side information (if any). Online, when a query arrives, Tailwind acts like a database proxy [23, 143] and transparently rewrites queries to use those accelerator instances when beneficial. Tailwind

orchestrates execution across the accelerators and the underlying RDBMS, transferring intermediate data as needed so that accelerators can replace sub-plans anywhere within a query plan. TAILWIND is complementary to techniques that synthesize specialized query executors [70, 106, 181, 190], focusing instead on the problem of intelligently deciding when and where to use accelerators in a query. The end result is a system that automatically and *surgically* inserts accelerators into an RDBMS without modifying its internals, helping to close the gap between specialized and general-purpose systems.

Challenges. While simple to use, realizing TAILWIND presents a significant design challenge. We need a generic way to (**C1**) let accelerator builders describe which logical plan fragments their accelerator can correctly replace, (**C2**) expose query-specific information needed by the accelerator implementation, and (**C3**) automatically model the accelerator's performance to know when to use it. For example, for domain negation, builders need a simple but precise way to describe the "filtered aggregation" pattern, the implementation needs to know what predicate to negate and what aggregates to pre-compute, and TAILWIND needs to learn how these pieces influence the accelerator's performance.

Key component: Abstract logical plans (ALPs). To check if an accelerator can be used on a query (**C1**), a naïve approach is to try defining its applicability using a view and rely on view matching algorithms [75, 82]. But views are insufficient because they express a single concrete logical plan whereas an accelerator might support a variable-sized sub-plan (e.g., a left-deep join chain). On the other hand, requiring builders to write all of the code that checks if their accelerator is usable on a given query is also undesirable. This is because custom code provides no common structure for extracting an accelerator's parameters (e.g., the predicate to negate) (**C2**) and is hard to featurize for performance modeling (**C3**).

To address these challenges, we introduce *abstract logical plans* (ALPs): a new abstraction that accelerator builders use to describe their accelerator's semantics. At its core, an ALP centers on a parameterized regular tree expression [9, 55] over logical query plans that specifies the set of logical plan fragments an accelerator can correctly replace (**C1**). The parameters exist to capture query-specific information that can vary across accelerator uses (e.g., the predicate to negate in domain negation). The same ALP serves two further purposes. It can be compiled into a tree automaton [55, 63, 177] to efficiently search query plans for matches and extract parameter values (**C2**), and its tree structure and parameters can be used to construct a custom neural network that estimates when the accelerator is beneficial to use (**C3**, Section 4.3.3). For TAILWIND, ALPs therefore provide a unified abstraction for an accelerator's applicability, interface to its implementation, and performance modeling.

We demonstrate the generality of TAILWIND and ALPs using three case studies on distinct query workloads. Across all three cases, TAILWIND lets us integrate workload-specific accelerators to speed up queries on Redshift and DuckDB, achieving geomean query run time speedups of up to 1.38×, 1.76×, and 1.28×.

Contributions. In summary, we make the following contributions:

- Abstract logical plans (ALPs): an abstraction that specifies the sub-plans an accelerator supports, interfaces with the implementation, and enables performance modeling.

- A specialized graph neural network model featurization for ALPs, used to learn performance models for each accelerator.
- The TAILWIND system, which enables the automatic and surgical insertion of builder-defined accelerators into query plans.
- The implementation and evaluation of these ideas in TAILWIND over three case studies.

4.2 TAILWIND Overview

We begin with an overview of how accelerator builders and end-users use TAILWIND (Figure 4.2). We see accelerator builders as data engineers with expertise in query processing and workload specialization and end-users as any user of the underlying RDBMS; the roles can overlap too. Accelerator builders register a "library" of accelerators (A), each comprising an ALP (Section 4.3.1) and implementation. Offline, TAILWIND then learns a run time model (B) for each accelerator (Section 4.3.3) by generating accelerator instances and measuring their run times to get training data (Section 4.5.1).

Since accelerators can be stateful (e.g., domain negation pre-computes the unfiltered aggregations), TAILWIND next chooses the accelerator *instances* to create. End-users provide a query log (C), which is a representative (not exhaustive) list of previously seen queries, and a space budget (D). TAILWIND's offline planner (E) takes these inputs and finds accelerator instance candidates by enumerating ALP matches in the log. It then uses its performance models in a greedy search to choose instances (F) that optimize the workload's predicted speedup subject to the space budget (Section 4.4).

Online, end-users submit their queries to TAILWIND's query planner (G). Using its instantiated accelerators, (F) TAILWIND rewrites queries to use one or more accelerator instances if they apply and are predicted to be beneficial (H). It then coordinates query execution by invoking the accelerators and underlying RDBMS, moving intermediate data to/from these components as needed.

4.3 Abstract Logical Plans

Recall that abstract logical plans (ALPs) serve three key roles in TAILWIND: (i) describing the logical plan fragments an accelerator can replace, (ii) exposing matched plan elements and parameters to the accelerator implementation, and (iii) providing structure for performance modeling. We look at each role in detail next.

4.3.1 Specifying Accelerator Applicability

A naïve non-solution. A tempting idea is to express an accelerator's applicability using a database view and rely on view matching [75, 82] to check if the accelerator can be used on a query. However, views fall short because they only define a single concrete query expression

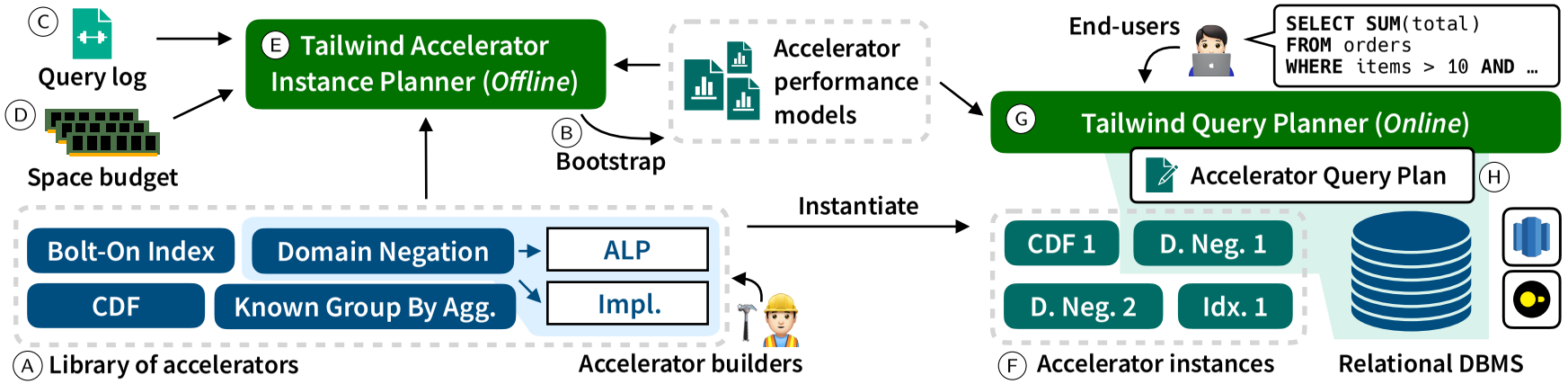


Figure 4.2: An overview of the TAILWIND system and its workflow.

bound to specific relations. Since an accelerator could potentially handle a variable-sized sub-plan (e.g., any left-deep join of arbitrary depth), we need a solution that is capable of specifying a (possibly infinite-sized) set of logical query sub-plans.

Overview. To specify the set of logical plan fragments that an accelerator can correctly replace, an ALP comprises:

1. A ***template***, which is a parameterized regular tree expression [9, 55] over logical query plans that describes the "shape" of the query plan fragments the accelerator supports.
2. A ***validator***, which is a builder-defined function that takes a match instance (the parameter values extracted from a query fragment that matched the template) and returns true if the accelerator supports the match instance.

An accelerator can be used on a query if the template matches a logical sub-plan in the query *and* the validator returns true for the match instance. ALPs need both components because templates alone cannot express non-regular patterns nor patterns that have semantic constraints (e.g., that two tables are being joined via a primary-key-foreign-key relationship). Using only a validator (i.e., only builder-written matching code) is undesirable because arbitrary code provides no common structure for extracting parameters and is hard to featurize for performance modeling (Section 4.5.1). Informally, the template captures the "structure" of the pattern and a validator "refines" it as needed. We now describe these two components in detail.

Templates

To help explain templates, we first provide some background on regular tree expressions (RTEs) [55].

Background on RTEs. RTEs generalize regular expressions [165] to tree expressions. RTEs are themselves directed trees. Using function notation to describe tree expressions (e.g., $t(a, b)$ denotes a tree of three nodes with root t, left child a, and right child b), an RTE is defined inductively as [55]:

- A *leaf token* a. The RTE matches a tree with a single a node.
- A *tree token* $t(R_1, \ldots, R_n)$. The RTE matches a tree with root node t and n children where child i matches R_i.
- An *alternation* $R_1 \mid R_2$, which matches RTEs R_1 or R_2.
- A *concatenation* $R_1 \odot_c R_2$, which means an RTE composed of R_1 with its leaf node c replaced with R_2.
- A *Kleene star*: R^{c*}, which matches R zero or more times where the root of each repetition is at the leaf node c (appears in R).

Table 4.1: The variable types used in ALP templates.

Variable type	Description
Column reference	A reference to a column (e.g., in a predicate or aggregation)
Table reference	A reference to a table (e.g., in a scan)
Table expression	A query sub-plan (e.g., scan, join, filter)
Column expression	An expression involving columns (e.g., col1 + col2)
Boolean expression	An expression that produces a boolean (e.g., a predicate)

For example, the RTE $R = [t(c, b)]^{c*} \odot_c a$ matches $t(a, b)$; $t(t(a, b), b)$; $t(t(t(a, b), b), b)$ and so on.

From RTEs to Templates. While RTEs provide a formal foundation for tree pattern matching, they are not usable out-of-the-box in Tailwind for two key reasons. First, classical RTEs lack a mechanism for indicating the parts of a pattern that will vary across distinct matches (e.g., the predicate to negate in domain negation). Second, recall that Tailwind has an offline phase where it selects accelerators to instantiate and an online phase where it searches for matches of these instantiated accelerators within incoming queries. We need a way to distinguish between parts of the pattern that must be resolved and fixed at instantiation-time (e.g., the specific aggregates to compute in domain negation) versus the parts resolved online in an incoming query (e.g., the predicate to negate). These requirements motivate our design of ALP templates.

ALP Templates. An ALP template T is also defined inductively and includes leaf tokens and tree tokens like classical RTEs. It includes three more constructs to support Tailwind's use case:

- **Typed variables.** A typed variable $\mathbf{Var}(x; \tau)$ has an identifier x and matches any plan expression of type τ. Variables are the parameters in the template and represent parts of the pattern that will vary across distinct matches. We use types to introduce matching constraints (e.g., the matched variable being a column reference versus a column expression) and to provide semantic information to featurize for performance modeling (Section 4.5.1). Table 4.1 lists the variable types ALPs support.
- **Alternations.** An alternation $\mathbf{Alt}(x; T_1, \ldots, T_n)$ has an identifier x and is otherwise the same as an RTE's alternation.
- **Repetitions.** A repetition $\mathbf{Rep}(x; T_B, T_R, c)$ has an identifier x and represents a template T_R that repeats zero or more times (or one or more times) at the c leaf node, followed by one occurrence of T_B. Repetitions are implemented using an RTE's Kleene star and concatenation. Instead of exposing those constructs, ALP templates use repetitions to serve a dual practical purpose: (i) providing a single constrained way to describe arbitrarily deep but bounded repetition in query sub-plans (e.g., nested joins), which in turn provides (ii) a single construct representing repeating plan substructures to featurize for performance modeling (Section 4.5.1).

If the same variable (i.e., having the same identifier) appears multiple times in the ALP, the template will only match query sub-plans where each occurrence of the variable resolves to the same value. Repetitions are an exception: each repeating instance creates a new scope. Variables and alternations inside a repetition can resolve to different values in different repeating instances. Finally, for each variable, alternation, and repetition, accelerator builders must indicate if the construct is resolved at *instantiation time* (i.e., during TAILWIND's offline planning) or *online* (i.e., when matching accelerator instances to queries just before execution).

Validator

Validators have the following signature:

```
fn is_valid(match_inst, schema) -> bool
```

Validators are called with a `match_inst` (described in Section 4.3.2) and the database `schema` (used to check semantic constraints, e.g., being a primary-key-foreign-key join).

Expressiveness. ALPs can describe any decidable logical query sub-plan pattern. This is because the template can match an arbitrary sub-plan (e.g., using a table expression variable), and the validator can be any decidable function. However, for practical ease-of-use and performance modeling, what matters is how much of a pattern's structure can be expressed using the declarative ALP template. Empirically, in our case studies (Section 4.6), we were always able to express the pattern's core structure using the template and only relied on the validator to check semantic constraints (e.g., that a matched join condition is a primary-key-foreign-key join).

ALP Example

To provide intuition for ALPs, we go through an ALP template for the domain negation accelerator. ALP templates are directed trees. Therefore, for clarity, we visualize the template in Figure 4.3 instead of relying on notation. We start with a table expression variable (A) matching any logical query plan. We declare that it is resolved at instantiation time because the pre-computed aggregates depend on this sub-plan. Next, we declare a filter token (B) since the template must match a filter over some sub-plan. The filter has an online-resolved predicate variable (C) because the accelerator supports any predicate over its fixed sub-plan. Finally, we add a group by aggregation token (D) since the matched sub-plan must be an aggregation. We add a repetition (E) over a new instantiation-time resolved column reference variable (F) since there can be multiple key columns in the group by. Similarly, we create another repetition (G) over a new instantiation-time resolved variable (H) since there can be multiple aggregation expressions. The accelerator supports `SUM` and `COUNT`, so we wrap the aggregation expression variable in an alternation with these two options (J).

Note that we must also specially capture filtered aggregations on left joins when the filter is part of the join condition, as these filters cannot be lifted above the join. The full

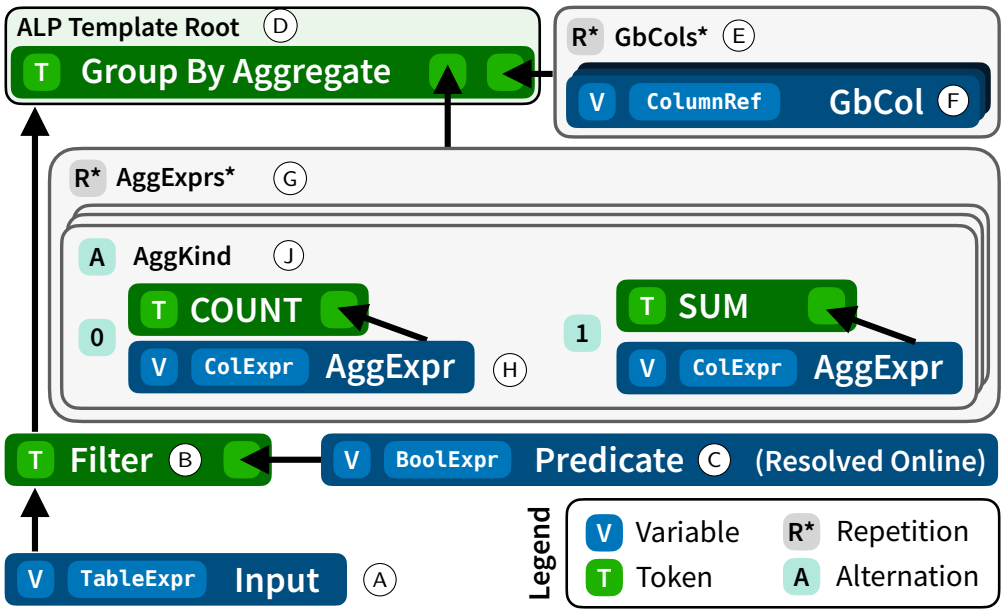


Figure 4.3: The domain negation ALP template (tree depiction).

domain negation ALP must place the input-filter-predicate sequence (Ⓐ, Ⓑ, Ⓒ) inside an alternation at its base. The alternation contains another option with a left join token. There, the predicate variable captures the join condition, which allows the accelerator to distinguish these cases from the filtered sub-plan case. We omit this detail from the figure for simplicity and clarity.

4.3.2 Interfacing with Implementations

When an ALP matches a query, we say that it produces a *match instance*, which is a map that contains

- The matched value for each variable in the ALP's template
- The matched option for each alternation in the template
- The number of times each repetition matched, with the variable values and alternation options for each repeating instance

An ALP and match instance define a concrete logical plan that the accelerator can execute. A match instance thus represents the "query-specific information" that an accelerator implementation needs during Tailwind's offline planning and online at runtime.

Accelerator API. To add an accelerator to Tailwind, accelerator builders need to implement three functions:

- `fn alp() -> ALP`
- `fn build(match_inst, ctx) -> State`

- `fn run(state, match_inst, input, ctx) -> Output`

The `alp()` function returns the accelerator's ALP. Given a match instance and context about the underlying RDBMS's SQL semantics, `build()` performs any pre-computation needed to create an accelerator instance (e.g., computing the unfiltered aggregates in domain negation). TAILWIND invokes `run()` with the match instance, the state returned by `build()`, any input data, and the same context just mentioned. Builders must ensure that their accelerator implementation returns the same results that the underlying RDBMS would for the logical sub-plan that they replace. Accelerators at the bottom of a query plan only produce outputs. Accelerators in the middle of a query plan receive inputs and produce outputs. TAILWIND uses Arrow [32] as the input and output data format.

4.3.3 Accelerator Performance Modeling

Finally, we look at how ALPs support performance modeling. A key challenge is that TAILWIND must support builder-defined accelerators without making assumptions about how they are implemented, which precludes prior approaches that featurize UDF code [189].

We address this challenge by observing that ALPs provide a structured description of what an accelerator computes. Critically, the template distinguishes between the parameterized and constant parts of the accelerator and encodes their dependencies. For example, in the domain negation template (Figure 4.3), the input query plan varies across accelerator instantiations and its value influences the accelerator run time, which the template's tree structure expresses.

TAILWIND thus models an accelerator's performance by encoding its ALP template as a tree-structured neural network. The learned weights correspond to the varying parts of the template (its variables, alternations, and repetitions) and the message passing follows the dependency relationships in the ALP (Section 4.5.1). TAILWIND learns one model per accelerator. We use the same featurization strategy for each, but each model will have a different set (and number) of weights because they are based on different ALP templates.

Bootstrapping. Before TAILWIND uses its accelerators, it must train its models offline. This only needs to be done once per accelerator. TAILWIND uses the ALP definitions to generate accelerator instances to collect training data; we detail this process in Section 4.5.1.

4.4 Query Planning with Accelerators

Bringing everything together, we now look at TAILWIND's planners.

Offline planning. Before processing any queries, TAILWIND selects accelerator candidates to *instantiate*, given a user-provided space budget in bytes S. We do this because accelerators are parameterized and can be stateful; typically the space usage of all possible candidates greatly exceeds S. This offline step ensures TAILWIND's online planner only considers the

"most useful" set of accelerator options given S. This selection problem is a generalized version of automatic index selection, which itself is NP-hard [51, 54, 145].

TAILWIND tackles this challenge using a two-step approach. First, it enumerates a set of possible accelerator instance candidates by matching its ALPs against the user-provided query workload to narrow the candidate search space. Then, it uses a greedy search over the instance candidates, picking candidates that provide the best predicted run time reduction normalized by each candidate's space usage. We found this strategy to be acceptable in selecting useful candidates in practice (Section 4.6) and discuss details in Section 4.5.4.

ALP matching. To find accelerator candidates, TAILWIND searches for ALP matches in a query plan. There can be multiple equivalent versions of a query plan and only some may match an ALP. TAILWIND handles this by converting the query plan into an e-graph [131, 134] and applying equality saturation [176]. E-graphs are compact data structures that encode equivalent versions of expression trees (e.g., logical query plans), similar to the tree of groups in the Cascades optimizer [80]. TAILWIND uses e-graphs for engineering reasons [191]; our techniques would also apply to a Cascades optimizer. TAILWIND searches for ALP matches by converting the ALP template into a nondeterministic finite tree automaton [55] and checking for matches in each e-class within the e-graph. We describe our matching and ALP resolving algorithms in Section 4.5.3.

Online planning. At runtime, TAILWIND receives a query and searches for ALP matches using the instantiated candidates' ALPs. It enumerates all possible ways to execute the query using the instances that match and selects the option predicted to be fastest. An exhaustive search is practical because, at runtime, few accelerator instances match a query (zero to two in our experimental workload). We evaluate this planner's overhead in Section 4.7.

Query performance modeling. Given a query with accelerator(s), TAILWIND individually predicts the run times of (i) the accelerators, (ii) any data transfer that would occur, and (iii) the remaining query (the remaining operators after parts of a query have been replaced by accelerator(s)). It sums these predictions to estimate an end-to-end run time. For (i), it uses the learned model for the accelerator (Section 4.3.3). For (ii), it uses a linear model based on the amount of data transferred (using a cardinality estimate). For (iii), TAILWIND scales the run time of the bare query (without accelerators) by the ratios of the logical plan operators and estimated input cardinality between the bare and remaining queries. TAILWIND assumes access to (i) a standard cardinality estimator (e.g., [160]) whose statistics are periodically maintained, similar to how production RDBMSes routinely run `ANALYZE`, and (ii) a query run time predictor (e.g., [10, 66, 119, 192, 193, 195], orthogonal to this work); see Section 4.5.2.

4.5 Additional TAILWIND Details

Now we look at TAILWIND's planner and models in detail.

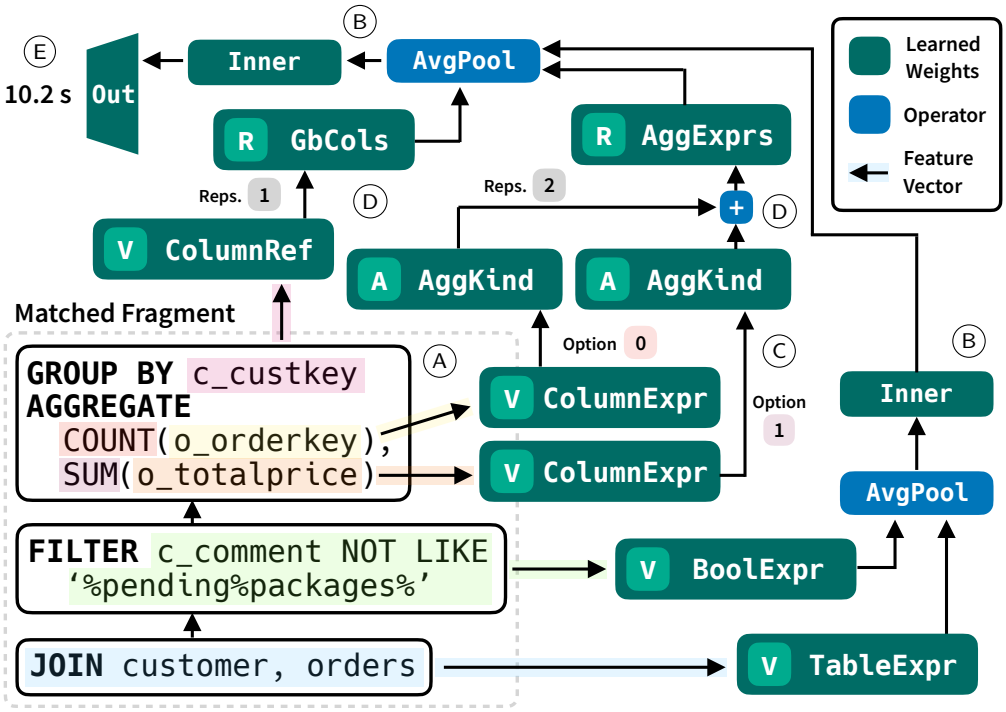


Figure 4.4: The domain negation accelerator's featurization.

4.5.1 ALP Model Details

Recall that TAILWIND models the performance of an accelerator by representing its ALP template as a tree-structured neural network. We now look at how TAILWIND constructs and trains these models.

Featurization

We encode the ALP template's parameterized constructs (i.e., variables, repetitions, and alternations) and then propagate and pool the encoded values up the template tree following its directed edges. At the ALP's root, we pass the output vector through a multi-layer perceptron (MLP) [76] to get a predicted run time. We use this model architecture because it captures the varying parts of an accelerator (e.g., what predicate to negate, which aggregates to compute) and encodes their dependencies using its tree structure. We walk through an example featurization for the domain negation accelerator (Figure 4.4) to illustrate the details.

Learned weights. We create one encoder MLP per variable type, repetition, and alternation in the ALP template. We also create one MLP for inner template nodes and one output MLP. Our domain negation ALP template (Figure 4.3) has four variable types, one alternation, and two repetitions. So we create nine MLPs: seven for the parameterized constructs plus the two additional ones. The teal boxes in Figure 4.4 depict the learned MLPs. The boxes with the same name refer to the same MLP; we show duplicates only to make the model's inference procedure easier to understand.

Table 4.2: The features used for each variable type.

Variable type	Features
Table expression	Cardinality, width (bytes), Stage / T3 op. features [154, 195]
Table reference	Cardinality, width (bytes)
Boolean expression	Selected fraction (%)
Column exp. or ref.	Cardinality, number of unique values

Inference

Suppose our domain negation ALP matches the plan fragment shown in Figure 4.4 (A). TAILWIND creates a run time prediction by traversing the ALP template and recursively constructing an output embedding vector bottom-up by using the following procedure for each ALP construct.

Variables. Variables are always leaf nodes in the ALP template. We create a vector representation for each variable's value, using the set of features shown in Table 4.2. We then pass each variable vector through its corresponding encoder MLP to obtain an output vector.

Internal tokens. We take the output vectors from the token's children and perform average pooling to obtain a single fixed-size vector. We then pass this vector through the inner template node MLP to obtain an output vector (B).

Alternations. We take the output vector from the option that matched in the template instance. We concatenate a one-hot feature vector corresponding to the chosen option and pass this new vector through the alternation's MLP to obtain an output vector.

Repetitions. We take the output vectors from the repetition's children and add them together element-wise. We concatenate the number of repetitions to the summed vector and pass this new vector through the repetition's MLP to get an output vector (D).

Final output. We take the output vector from the ALP's root and pass that through the output MLP to get a run time (E) prediction.

Bootstrapping Details

To learn a model for each accelerator, TAILWIND requires labeled performance data. Although ALPs are for recognizing accelerator patterns, we can repurpose them to *generate* possible accelerator instances. TAILWIND measures these instances' run times to get labeled data for training.

The high-level idea is to select random values for each construct (e.g., variable) in the ALP and return the instances that pass the validator. However, this random selection must ensure semantic correctness and diversity. For example, in the domain negation ALP (Figure 4.3), the sampled predicate variable value must involve columns accessible from the input plan and TAILWIND needs to avoid generating only true (or only false) predicates. To generate

Table 4.3: The methods we use to generate variable values.

Variable type	We randomly select
Table reference	A table in the dataset
Table expression	A query slice from the planning workload
Column reference	A column in the available columns
Column expression	A column, binary arithmetic expression (column and literal, or two random columns), or depth-2 binary expression
Boolean expression	An open range, a closed range, equality, two predicates joined using `AND` or `OR`

instances, TAILWIND first selects repetition counts and alternation options, which fixes the "structure" of the resulting instance (e.g., selecting the number of group by columns in domain negation). Then, it traverses the ALP template bottom up and computes the set of "available columns" at that stage in the query fragment (e.g., the output schema of the child query plan node) and then selects values for any variables using the strategies summarized in Table 4.3, taking care to only select available columns. TAILWIND repeats this procedure to generate a sufficient number of training data instances per accelerator. We found that 10k instances were sufficient, but this number is a hyperparameter that users can adjust if needed.

4.5.2 TAILWIND's Other Models

To help decide whether using accelerators is better than the base RDBMS, TAILWIND also leverages other models. They are not the focus of this chapter, but we describe them here for completeness.

Query run time. TAILWIND needs to estimate the run time of a query. Since this is a problem that has been extensively studied [87, 116, 117, 119, 154, 173, 195, 197, 200], we assume an acceptable model exists. In our evaluation, we assume we have the query's run time on the RDBMS (e.g., it is cached) and study TAILWIND's robustness to query run time prediction errors (Section 4.7.3). For remaining queries (the operators after parts of a query have been replaced by accelerator(s)), TAILWIND scales the bare query's run time by the ratio of logical operators and estimated input cardinalities between the bare and remaining plans. This coarse approach is effective because accelerators typically replace large portions of a query.

Data transfer. We estimate TAILWIND's intermediate data transfer time using $\max(S/k_i + S/k_e, C)$ where S is the size of the data transferred (e.g., in bytes), k_i and k_e are empirically measured data import/export rates, and C is a minimum transfer time. TAILWIND measures these constants offline before handling user queries. We use S3 [21] to transfer data between the accelerators and Redshift. For DuckDB, accelerators run in-process, so there is a negligible transfer overhead.

4.5.3 ALP Matching and Resolution

TAILWIND's planner searches for accelerator use opportunities in a workload. This process consists of three conceptual parts: (i) matching ALPs to queries, (ii) resolving the matched constructs in the ALP (e.g., assigning concrete values to its variables), and (iii) enumerating query plans that contain the matched accelerators. We look at each of these next, starting with ALP matching.

ALP Matching

ALPs match patterns in tree expressions after being compiled into non-deterministic finite tree automata (NFTAs) [36, 63]. NFTAs are analogous to NFAs for strings [165]. However, simply matching ALPs to query plans is insufficient because the query might need to be rewritten before it matches (e.g., reordering a filter in the query plan). TAILWIND addresses this problem by converting queries into an e-graph [191], using equality saturation to find equivalent queries [176], and then matching ALPs on the resulting e-graph. To realize this approach, we extend a top-down NFTA matching algorithm to e-graphs, as existing algorithms are for tree expressions (not e-graphs) [55].

Background on NFTAs. To help with understanding our matching algorithm, we first give some background on NFTAs. An NFTA is a state machine for trees. Formally, it is a 4-tuple (Q, F, Q_f, Δ) comprising a set of states Q, accepting states Q_f where $Q_f \subseteq Q$, an alphabet F, and a set of transitions Δ [55, 63, 177]. Informally, F is the set of nodes that can appear in an expression tree. We use the notation $a(c_1, c_2, \ldots, c_k)$ to refer to a node a that has k child nodes c_1 to c_k. For example, a filter in a logical plan could be represented as Filter(x, p) where x is its child plan operator and p represents its predicate expression. Transitions are given as $(l_1, \ldots l_k) \rightarrow_t r$ where $l_1, \ldots, l_k, r \in Q$. If we encounter a node t whose k children are in states l_1 to l_k, we transition to state r. A transition for a leaf node t is expressed as $() \rightarrow_t r$. A bottom-up NFTA matches an expression tree if there exists a sequence of states from its leaf nodes to its root and the root state is in Q_f [55].

Background on E-Graphs. E-graphs are data structures that represent multiple equivalent tree expressions in a single data structure [131, 134], much like logical groups in a Cascades optimizer [80]. Concretely, they are directed graphs of e-classes, which contain e-nodes. An e-node is any node that would appear in a concrete tree with the twist that its children are e-classes. The e-nodes in the same e-class describe logically equivalent expressions. For example, Filter(Filter$(x, p_1), p_2)$ is equivalent to Filter$(x, p_1 \wedge p_2)$, so their top-level nodes would be in the same e-class. Critically, e-graphs are unlike expression trees because they can contain cycles.

NFTA Matching Algorithm on E-Graphs. E-graphs are data structures that represent equivalent tree expressions (e.g., multiple equivalent logical query plans) in a single data structure [131, 134], similar to logical groups in a Cascades optimizer [80]. Algorithm 2 shows our matching algorithm, which makes two extensions (highlighted in green) to NFTA top-down matching [55]. TAILWIND runs this matching algorithm on each e-class in the

Algorithm 2 Top-down NFTA matching on an e-graph.

Input: N: NFTA, E: Query e-graph, R: Root e-class
Output: True if N matches E at R, False otherwise

```
▷ S: Curr. states, Δ: Transitions, C: Current e-class, P: Match path ◁
function TopDownMatch(S, Δ, C, P)
    ▷ Traverse the transitions and find child states. ◁
    M ← [ ]
    for all transition ((l_1, l_2, ..., l_k) →_t r) ∈ Δ and r ∈ S do
        if t in e-class C and it has k children then
            if (r, C) does not create a cycle in P then
                Append ((l_1, l_2, ..., l_k) →_t r) to M
    if any transition in M is to a leaf (e.g., () →_t r) then
        output True
    ▷ Recurse to children. ◁
    M' ← Group transitions in M by terminal t. Collect child states by position into sets.
    for all ((L_1, L_2, ... L_k) →_t R) in M' do
        Out ← [ ]
        for all i from 1 to k do
            C_i ← terminal t's child i
            P' ← P ∪ {(r, C) | r ∈ R}
            Append TopDownMatch(L_i, Δ, C_i, P') to Out
        if Out is all True then
            output True
    output False

(Q, _, Q_f, Δ) = N                ▷ Q: States, Q_f: Final states, Δ: Transitions
output TopDownMatch(Q_f, Δ, R, {})
```

e-graph.

Our first extension is to search over all e-nodes in the e-class. We accept (match) the root e-class if there is at least one e-node in the root class where all of its children reach a leaf transition. We examine all e-nodes in the e-class because the NFTA might only match a tree rooted at one of the nodes in the class (i.e., corresponding to a specific way of expressing the accelerator's plan fragment).

Our second extension is in rejecting match cycles. NFTA transitions can contain state cycles and e-graphs can contain cycles. Match algorithms for trees do not have this problem because tree expressions are acyclic, making match cycles impossible. Our approach is to reject matching paths that contain *both* a transition cycle and e-class cycle. Rejecting only transition cycles is incorrect because they describe repetitions in the tree expression. Rejecting only e-class cycles is incorrect because the NFTA could be matching a tree expression that has a finite number of recursive repetitions. We track the *path* of NFTA states and e-classes during a match and stop following any transitions that would create both a state and e-class cycle.

Resolving ALP Constructs

Next, TAILWIND resolves (i.e., assigns values to) the ALP's variables, alternations, and repetitions to provide the necessary inputs for its models and the accelerator's implementation. When building an ALP's NFTA [36], TAILWIND stores the identifier of the "accepting state" associated with each template construct. During matching, it records the e-classes it encounters when following a transition from these stored states, which lets it map an ALP construct to its matched e-class in the e-graph. TAILWIND resolves a variable by extracting the smallest tree expression from its matched e-class. It resolves alternations by checking which option's output state matched. For repetitions, which can contain nested variables and alternations, TAILWIND records discovery and finishing logical timestamps during e-graph matching. It can then unambiguously map nested variables and alternations to their correct repetition instance by finding the smallest interval that contains the matched construct's traversal interval.

Extracting Accelerator Plans

Finally, TAILWIND enumerates accelerator execution plans using the e-graph. During matching, TAILWIND inserts a new e-node for the accelerator into the e-class that it matches. This indicates that the accelerator is equivalent to the other expressions in the e-class. Then, to enumerate the query execution options, TAILWIND constructs the options recursively per e-class. For each non-accelerator e-node, we compute its execution options as the Cartesian product of all options for its children e-classes. There are typically few accelerator candidates that match a query (zero to two times), so the Cartesian product is small in practice and usually has a size of one. We union each node's options with the accelerator(s) to get the set of options for that e-class. TAILWIND then returns the execution options from the root e-class.

4.5.4 Selecting Accelerator Instances

Recall that TAILWIND uses a greedy search to select the accelerator instances to use for a storage budget. TAILWIND first enumerates possible instance candidates by matching its ALPs against the user-provided query log. It then evaluates these candidates by computing their normalized benefit: the predicted reduction in workload execution time divided by the accelerator's space usage. The algorithm iteratively selects the instance with the highest normalized benefit, recomputing benefits after each selection, until there is no more run time reduction or it exhausts the space budget.

Discussion. We take this two-step approach for two reasons. First, matching ALPs to queries in the query log to enumerate candidates ensures we only consider instances that would match the workload. This strategy prunes the search space using our assumption that the workload log is representative of the online workload. Second, we use a greedy search because our candidates usually affect one query each and multiple candidates have a monotonic effect on a query's speedup. While complex interactions are possible (e.g., two

candidates accelerating a query together but not individually), our experiments show that a greedy search is practical (Section 4.6).

Analysis. Let n be the number of accelerator instance candidates. Our greedy search has a complexity of $O(n^2)$ since it recomputes the normalized benefit (for the remaining candidates) each time it selects a candidate. An exhaustive search would consider $O(2^n)$ candidate sets. The algorithm terminates because, on each iteration, it selects a candidate and the set of candidates is finite.

4.6 Workload Case Studies

Across three distinct query workloads, we now show how users can use Tailwind to incorporate workload-specific accelerators that speed up selected queries on both closed- and open-source RDBMSes. We begin by describing our common experimental setup.

Base systems. We run Tailwind with Redshift [19] and DuckDB [65]. Redshift is a closed-source managed distributed database system. DuckDB is an embedded single-process database system. We run Redshift on a cluster of two `ra3.large` nodes. We run DuckDB v1.3.2 on an `r6id.xlarge` EC2 instance [18], placing the DuckDB file on its NVMe drive. The `r6id.xlarge` instance has the same number of vCPUs and memory as the Redshift cluster in aggregate.

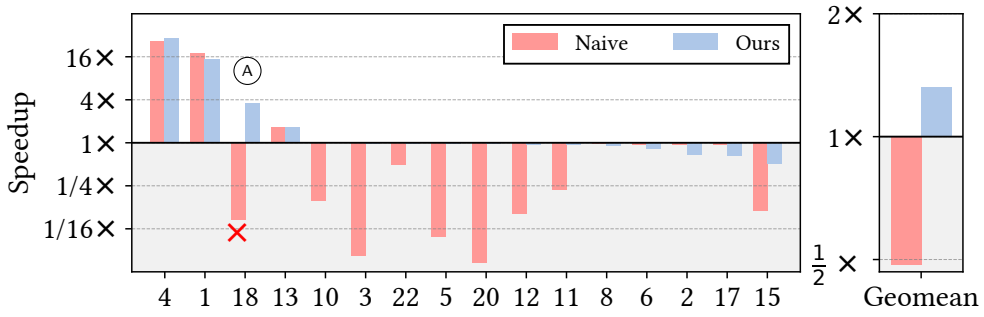
Accelerator compute. For DuckDB, we run the accelerators on the same `r6id.xlarge` machine as the RDBMS. As Redshift is a managed system, we run the accelerators on our own machine (20-core Intel Xeon Gold 6230, 128 GiB of memory). We limit the accelerators to 4 threads so they use similar compute resources.

Naïve baseline. We compare Tailwind to a naïve strategy that always uses an accelerator if it matches the query. It greedily selects the instances that replace the largest portion of a query in the workload until it exhausts the space budget. At runtime, if there are multiple ways to use the accelerators, it again picks the option that replaces the largest portion of the query with accelerators. We use a timeout of 5 minutes to avoid "getting stuck" running poor plans chosen by the naïve strategy.

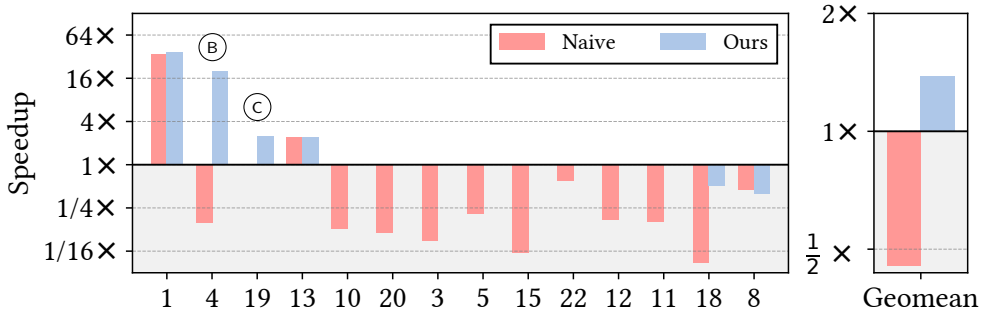
Implementation. We implemented Tailwind and the accelerators used in these case studies in Rust using ∼65,000 lines of code.

4.6.1 Case Study 1: TPC-H

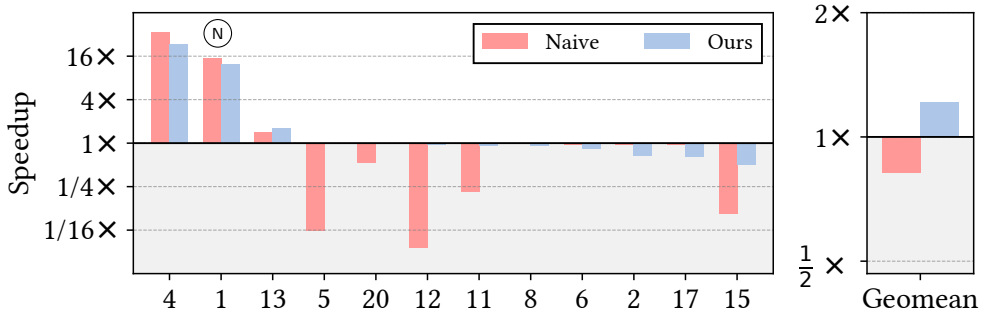
We start by looking at TPC-H [178], which is a standard analytical query benchmark. We use scale factor 100 (i.e., 100 GB of data).



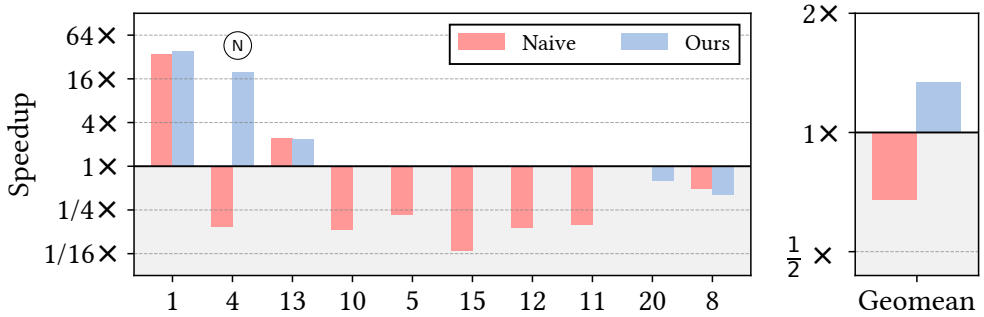
(a) Redshift (10%) (X indicates timeout)



(b) DuckDB (10%)



(c) Redshift (1%)



(d) DuckDB (1%)

Figure 4.5: TAILWIND accelerates TPC-H (SF = 100) on both Redshift and DuckDB by 1.32× and 1.38× respectively (geomean).

Accelerator Library

The first step in using TAILWIND on a workload is to identify accelerators that exploit the characteristics of the queries and/or underlying dataset. Recall that the goal of this work is not to propose new general optimizations that apply to all queries, but to design a new framework that integrates workload-specific accelerators with an RDBMS. We use four such accelerators in this case study (not necessarily an exhaustive set of accelerators).

Domain negation. This is the accelerator described in Section 4.1.

Cumulative filtered aggregates (CDFs). Another acceleration opportunity is in queries involving invertible aggregations [174] (`SUM`, `COUNT`, `AVG`) over data filtered by an ordered column (e.g., a date range). If we pre-compute a cumulative aggregation over the range, answering these query fragments reduces to looking up the cumulative values for the range endpoints and returning the difference, which is faster than executing the query from scratch.

Known group by with fused arithmetic. We can accelerate queries that aggregate over a few known groups by hard-coding the expected groups into a query executor and fusing the aggregation computation into one compute kernel. For example, Query 1 from TPC-H only produces four groups and performs arithmetically heavy aggregations [40, 98]. The performance gain is from (i) avoiding a hash map for grouping and (ii) keeping the intermediate aggregations in registers during execution. This technique differs from standard query compilation, as hash maps are used to collect groups [132]. While a sophisticated query compiler could in theory use this specialization too, general-purpose systems would not.

Ordered indexes. Some analytical RDBMSes (e.g., Redshift) do not have indexes [27, 77]. Thus, point lookups and selective scans where the predicate is on an unsorted column must rely on a slow full table scan. Having indexes (i.e., a secondary sort order) in these cases would speed up the selection portion of those queries.

Results and Key Takeaways

We run TAILWIND with 10% and 1% space budgets (i.e., 10 GB and 1 GB budgets for a 100 GB dataset), which is comparable in size to other techniques that use additional space to accelerate analytical queries [60, 171, 172]. We generate one query instance per query template in the workloads; TAILWIND's offline planner uses these queries to select accelerator instances. We then run the workload with a different set of query instances (i.e., with different placeholder values). Note that we exclude query 21 from the experiment because one of its subqueries (that TAILWIND considers) triggers a query planning performance bug in Redshift (the subquery takes over an hour to `EXPLAIN`).

Figure 4.5 shows our results on Redshift and DuckDB with a 10% and 1% space budget. The y-axis is the speedup relative to running the full query on the RDBMS and is in log scale; higher is better. Bars below $1\times$ represent a slowdown. The x-axis lists queries sorted by decreasing speedup. We report a speedup summary by taking the geomean across all queries in the workload [178]. For clarity, we only plot the queries where there was a speedup (or

slowdown); for some queries, no accelerator was used so their performance is unchanged. From these results, we draw the following conclusions.

On average (geomean), Tailwind speeds up queries by 1.32× on Redshift and 1.38× on DuckDB. On TPC-H with a 10% (1%) space budget, Tailwind accelerates queries by 1.32× (1.21×) on Redshift and 1.38× (1.33×) on DuckDB (geomean). The speedup on TPC-H comes from the CDF (Q1, Q4), index (Q18, Q19), and domain negation (Q13) accelerators. The known group by accelerator also applies to Q1, but Tailwind selects the CDF as it provides a larger speedup. This result shows that Tailwind can leverage its accelerators across a diverse set of queries, RDBMSes, and space budgets.

While Tailwind achieves overall speedups, there are a few queries where it adds a small slowdown. In all but four cases, the slowdown is from Tailwind's optimization time, as those queries are short-running (hundreds of milliseconds). We study Tailwind's overhead in Section 4.7.4. For DuckDB Q18, the slowdown is due to a misprediction in the remaining query's run time. While the possibility of mispredictions is fundamental, Tailwind can cache its mistakes and avoid the poor plan the next time the query arrives. This mitigation is effective in repetitive query workloads, which are common in practice [152].

Naïvely using accelerators when they match leads to significant slowdowns, as slow as 0.46× (DuckDB). The slowdowns on Redshift are mostly due to data transfer: moving intermediate results from the accelerator to Redshift (or vice versa) can take longer than running the entire query on Redshift. DuckDB has a negligible data transfer overhead as the accelerators run in-process. Its slowdowns are because the naïve strategy uses the domain negation accelerator whenever it matches, even when not beneficial. When the predicate is already selective, its negation processes more data (and is thus slower). Thus overall, considering an accelerator's full effect on the query is essential for achieving workload speedups using accelerators.

When multiple accelerator options exist for a query, Tailwind's performance models help it select a better option. For Q18 and Q4 in Figures 4.5a (A), 4.5b (B), and 4.5d (N), Tailwind and the naïve strategy make different decisions. For Q18, the naïve strategy uses domain negation because it can push down most of the query into the accelerator. But that decision leads to a slowdown. In contrast, Tailwind uses an index on a subquery of Q18, which it correctly predicts as the faster option. Q4 is similar: the naïve strategy uses domain negation but Tailwind selects the CDF accelerator as it correctly predicts it to be the faster option.

Tailwind successfully tailors its accelerator choices to the underlying RDBMS. For TPC-H Q19 (C), Tailwind uses an index on DuckDB but avoids it on Redshift as it correctly predicts that the data transfer would be too slow on Redshift.

Tailwind's planner selects good accelerator plans even with a constrained space budget. On TPC-H, as we reduce the space budget from 10% to 1%, Tailwind intelligently prunes its index accelerator instances for Q18 (Redshift and DuckDB) (A) and Q19 (DuckDB) (C) while preserving the CDF and domain negation accelerators for Q1, Q4, and Q13 (N). This result shows how Tailwind prioritizes accelerators providing the most speedup within the space budget.

4.6.2 Case Study 2: TPC-H-Plus

As TPC-H queries are structurally simple, we develop an extended workload called TPC-H-Plus for this next case study. TPC-H-Plus contains 20 query templates that merge fragments of TPC-H queries (hence its name) to provide opportunities for multiple accelerator matches per query. Half of the workload's queries match one or more accelerators that provide a speedup. The other half are queries that match our accelerators but would slow down the query if used. We run TPC-H-Plus over a scale factor 100 TPC-H dataset.

Accelerator Library

Since TPC-H-Plus is based on TPC-H query fragments, we use the same accelerator library (Section 4.6.1).

Results and Key Takeaways

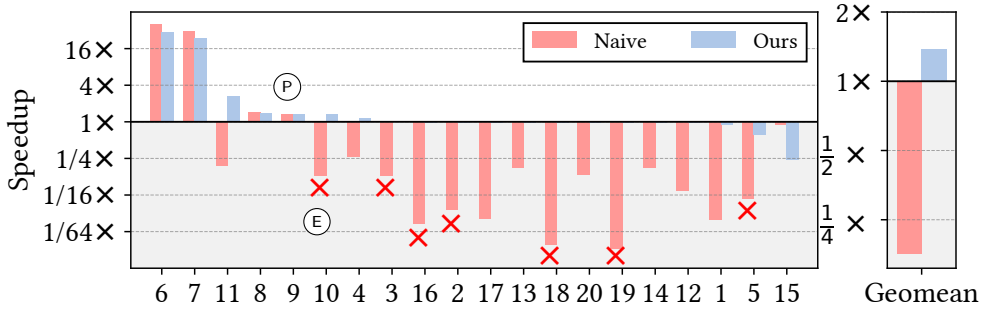
Like our first case study (Section 4.6.1), we run Tailwind with 10% and 1% space budgets and use the same methodology to create offline planning queries and online queries for execution. Figure 4.6 shows results for a 10% and 1% space budget, from which we draw the following conclusions.

Tailwind speeds up TPC-H-Plus queries on Redshift by 1.37× and DuckDB by 1.76× on average (geomean). With a 10% (1%) space budget, Tailwind accelerates queries by 1.37× (1.49×) on Redshift and 1.76× (1.78×) on DuckDB (geomean). The speedup comes from all four accelerators. Q6 and Q7 see large speedups because they use the CDF accelerator, which leverages pre-computation. These queries have a higher speedup on DuckDB (Figure 4.6b (D)) because it has a negligible data transfer cost. This result shows that Tailwind identifies opportunities to use its accelerators across a diverse set of queries, RDBMSes, and space budgets.

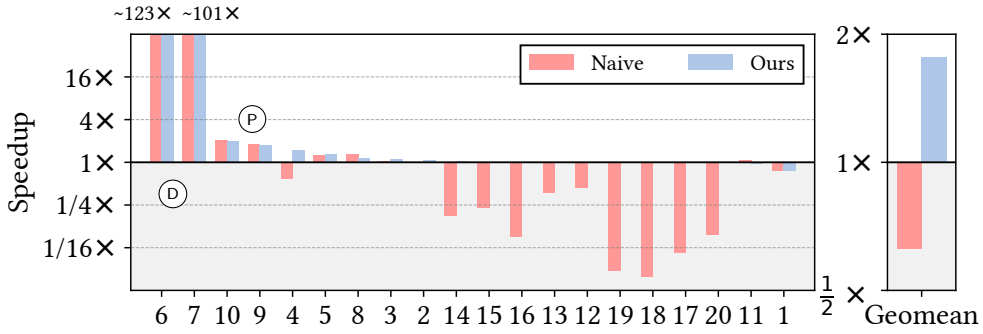
While Tailwind achieves overall speedups, it introduces a slowdown on Redshift Q5, Q15, and DuckDB Q1 due to a misprediction in the accelerator's run time or the remaining query's run time. While possible mispredictions are fundamental, Tailwind can cache its mistakes and avoid the poor plan the next time the query arrives (effective in repetitive query workloads, common in practice [152]).

Like in our first case study, Tailwind (i) avoids accelerator instances that introduce slowdowns and (ii) tailors its accelerator choices to the underlying RDBMS. Many of the naïve strategy's timeouts in Figure 4.6a are because it blindly applies the known group by accelerator. Known group by is usually a poor choice for Redshift, as Tailwind must transfer the input data from the RDBMS to the accelerator to aggregate (usually large). The slowdowns in Figure 4.6b are due to the domain negation accelerator, which is not always a good choice to use. Tailwind, in contrast, leverages its models to avoid these poor choices.

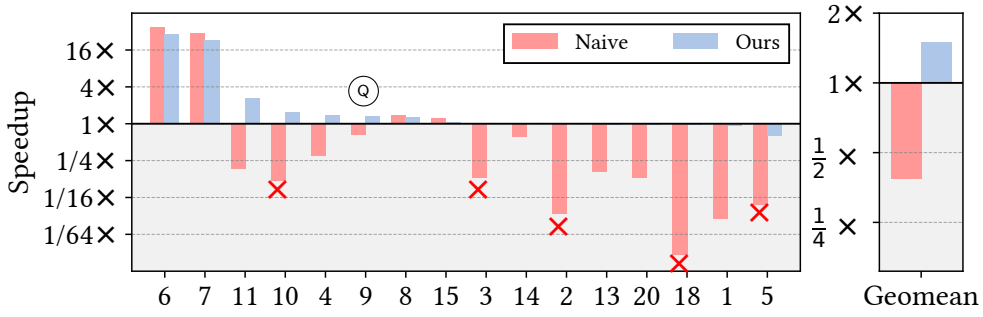
When beneficial, Tailwind correctly leverages multiple accelerators per query. It achieves geomean speedups of 5.62× on Redshift (Queries 7, 9) and 4.60× on DuckDB (Queries 5, 7, 9, 10) (geomean over these queries). Figure 4.7 shows an example of how



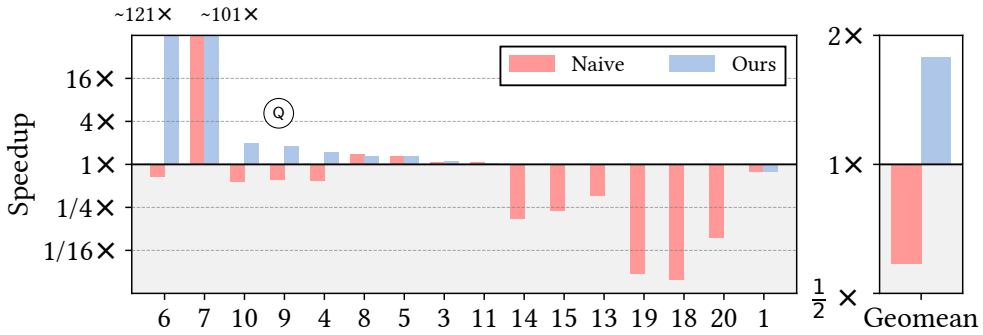
(a) Redshift (10%) (X indicates timeout)



(b) DuckDB (10%)



(c) Redshift (1%) (X indicates timeout)



(d) DuckDB (1%)

Figure 4.6: TAILWIND accelerates TPC-H-Plus on Redshift and DuckDB by 1.37× and 1.76× respectively (geomean).

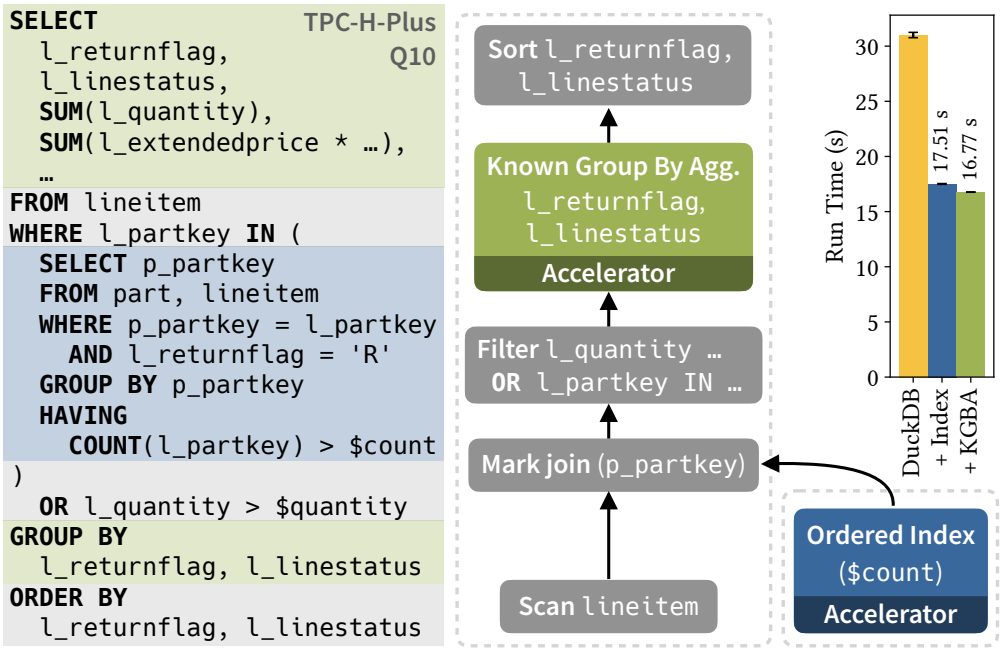


Figure 4.7: TAILWIND leverages two accelerators to speed up TPC-H-Plus Q10 by 1.9× on DuckDB.

TAILWIND uses the known group by accelerator together with the index over a subquery to speed up query 10 on DuckDB. Tailwind uses multiple accelerators more frequently on DuckDB because it can exploit the known group by accelerator; on Redshift, high data transfer overheads make this accelerator a poor choice. For example, the naïve strategy slows down query 10 on Redshift because it incorrectly applies the known group by (Figure 4.6a Ⓔ).

TAILWIND also selects good accelerator plans with a constrained space budget, similar to our first case study. For Q9 on Redshift, both strategies perform the same with a 10% budget Ⓟ. However, they diverge at 1%. The naïve strategy chooses two domain negation accelerators that use less space compared to TAILWIND's choice (to save space for other accelerators it greedily selected), but this choice causes a slowdown. In contrast, TAILWIND correctly keeps its original accelerator choices to maintain the speedup Ⓠ.

4.6.3 Case Study 3: SQLStorm Stack Overflow

In our final case study, we work with the Stack Overflow dataset [170] and SQLStorm queries [159]. Unlike TPC-H, which uses synthetic data, the Stack Overflow dataset comprises 220 GB of real-world data from the Stack Overflow website [170]. SQLStorm is a query workload comprising ∼18k diverse queries generated for this schema [159].

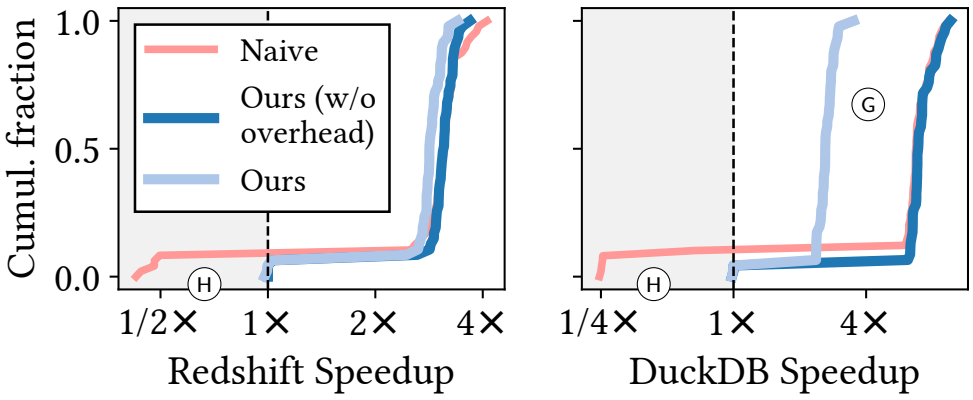


Figure 4.8: Speedups on our SQLStorm Stack Overflow queries.

Accelerator Library

This case study involves different queries and data, so we add two accelerators that are well-suited to SQLStorm. These are not necessarily an exhaustive set for SQLStorm.

Top-k pushdown. A common SQLStorm query pattern involves joining tables followed by a top-k operation (`ORDER BY` with a small `LIMIT`). A known optimization is to push the top-k operation below the join [46], which reduces the number of tuples processed by the join. But this optimization is only correct if the join does not eliminate any of the pre-selected top-k rows. We empirically found that neither Redshift nor DuckDB performs this pushdown, even when we explicitly declare foreign key constraints. Thus, we leverage domain-specific knowledge of the Stack Overflow schema to identify join cases where this pushdown can be correctly applied, and add these schema-specific cases to a stateless accelerator.

Badges MV. SQLStorm queries include subqueries aggregating user badge counts, which can be accelerated using a classical materialized view (MV) in Tailwind. We implement this MV with a "sparse table" optimization that decouples users with zero badges from badge earners. Over half of Stack Overflow users have no badges [170], so the accelerator only stores their IDs instead of full rows for zero counts. Compared to classic MVs, this approach lowers the accelerator's space usage for skewed distributions.

Previous accelerators. The domain negation, CDF, and ordered index accelerators (Section 4.6.1) also match queries in this workload, so we include them in Tailwind's accelerator library as well.

Workload Setup

SQLStorm contains around 18k distinct queries [159]. To construct a workload, we sample 1000 queries. We then execute them on Redshift and DuckDB and retain the queries that complete without error within 60 seconds on both engines, resulting in 356 queries. We use a `r6id.2xlarge` instance for DuckDB to have a large enough NVMe drive for the dataset.

Results and Key Takeaways

Similar to our previous case studies, we use a 10 GB space budget, which corresponds to ∼4.5% of the total dataset size. We use half of the sampled queries (178) in TAILWIND's offline planning pass and run the other half online. Figure 4.8 shows our results, plotted as a speedup CDF. The x-axis is the achieved speedup (on a log scale) over running the queries directly on the RDBMS. We plot the naïve strategy, TAILWIND, and TAILWIND without its query optimization overhead. For clarity, Figure 4.8 only plots the queries where an accelerator matched.

Averaging over the queries in our workload, TAILWIND speeds up queries by 1.28× and 1.27× (geomean) on Redshift and DuckDB respectively. On both RDBMSes, the speedup mostly comes from the top-k accelerator. The difference between TAILWIND and the naïve strategy at the upper end (Figure 4.8 Ⓖ) is due to TAILWIND's optimization overhead (performance prediction). This overhead is similar on both systems, but is a larger proportion of the query run times on DuckDB (the queries run faster). Note that the query run times are not directly comparable between Redshift and DuckDB because they are running on different hardware.

As in previous case studies, TAILWIND's models help it avoid accelerators that slow down a query. While the naïve approach can occasionally luck into the right accelerator choice, thereby gaining the benefits of TAILWIND without the optimization overheads, it will also make poor choices that lead to regressions. On both Redshift and DuckDB, it incorrectly uses domain negation and introduces slowdowns Ⓗ. TAILWIND pays some optimization overhead, therefore reducing our relative advantage on short-running queries, but it has the benefit of avoiding such regressions.

Summary. Our three case studies show that, given a workload, one can identify query-specific accelerators that speed up the workload and use them with TAILWIND. These accelerators are diverse, including physical execution optimizations like the known group by, structured pre-computation techniques like CDFs and domain negation, conditional query rewrites like top-k pushdown [46], and classical accelerators like indexes and materialized views. TAILWIND can learn their performance and can correctly apply them to achieve speedups.

4.7 Drill-Down Evaluation

We now examine how TAILWIND's components contribute to its performance. We seek to answer the following questions:

- How accurate are TAILWIND's accelerator performance models and how do different models affect its decisions? (Section 4.7.1)
- How does the space budget affect speedups? (Section 4.7.2)
- How do query run time prediction errors affect TAILWIND's query acceleration decisions? (Section 4.7.3)

Table 4.4: The ALP neural network's prediction error (p50, p90 Q-error) compared to simpler models (lower is better).

	Mean val.		Reg. forest		MLP		ALP NN	
Accelerator	**p50**	**p90**	**p50**	**p90**	**p50**	**p90**	**p50**	**p90**
Domain Negation (Redshift)	3.58	11.50	1.99	8.50	2.25	6.96	**1.43**	**3.36**
Domain Negation (DuckDB)	3.02	1196	1.76	10.58	2.18	8.20	**1.31**	**2.74**
CDF	11.05	131.4	**1.31**	2.81	1.36	3.45	1.34	**2.15**
Ordered Index	3.29	16.34	**1.39**	2.67	1.46	2.57	1.48	**2.53**
Known Group By Aggregation	61.36	89.85	**1.06**	1.21	1.09	1.23	1.07	**1.18**

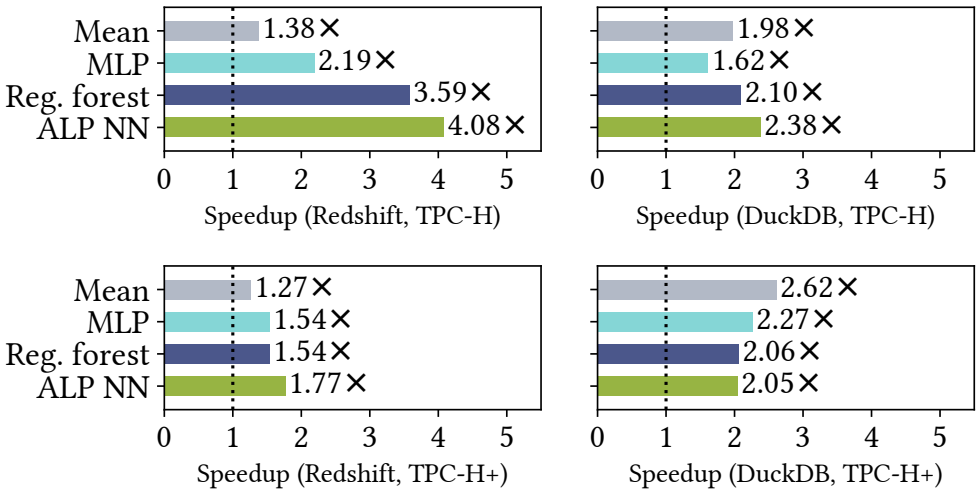


Figure 4.9: End-to-end effect of different accelerator modeling strategies on query performance (higher speedup is better).

- What is Tailwind's run time overhead? (Section 4.7.4)
- How do writes impact Tailwind's acceleration? (Section 4.7.5)

We run our drill-down evaluation on TPC-H and TPC-H-Plus as they sufficiently capture the representative behavior of Tailwind.

4.7.1 Accelerator Performance Models

Prediction Accuracy.

We train an ALP neural network for the accelerators in our first case study (Section 4.6.1) using a dataset of ∼10k data points per accelerator with an 80/10/10 train/validation/test split. We compare our ALP neural network to three baselines: (i) always predicting the mean (geomean) run time, (ii) a regression random forest [88] (3 trees), and (iii) a multi-layer perceptron (MLP) [76] (3 layers). For the regression forest and MLP, we manually design a

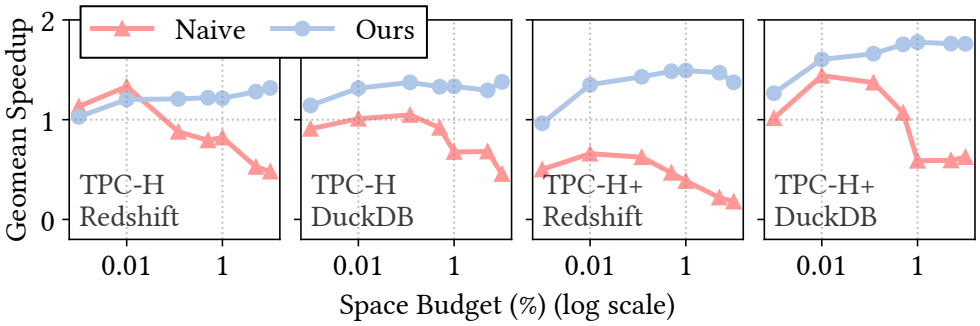


Figure 4.10: Workload speedup for varied space budgets.

feature vector for each accelerator. We evaluate prediction accuracy using Q-error, defined as $\max(p/a, a/p)$, where p and a are the predicted and actual run times respectively [125]. Lower is better and 1.0 is the best possible Q-error. Table 4.4 lists our results, from which we draw the following conclusions.

The ALP neural network achieves the lowest p90 Q-error on our tested accelerators and a p50 Q-error comparable to the best baseline. On the domain negation accelerator, our ALP neural network achieves the lowest Q-error compared to the other baselines. This is because (i) it is harder to model compared to the other accelerators in our library, and (ii) our ALP neural network provides a better inductive bias (encoding the accelerator's logical structure) compared to a simple MLP and regression forest.

Our ALP model featurization is general enough to work across diverse accelerators. Unlike our baselines, which use hand-designed features, our ALP model uses one featurization method for all the accelerators (Section 4.5.1) and achieves the best p90 Q-error.

End-to-End Effect.

Next, we study how different accelerator performance models affect Tailwind's query execution decisions. We run our workloads with each model option and compare the geomean achieved speedup. Figure 4.9 shows our results; higher is better. Here we compute the geomean only among the queries where Tailwind uses an accelerator to highlight the impact of different models. From these results, we draw the following conclusions.

Non-trivial learned models achieve the highest speedups and avoid slowdowns. Except for TPC-H-Plus on DuckDB, predicting the mean value does not lead to the highest speedup. On Redshift, using the mean value causes slowdowns as Tailwind mistakenly chooses an accelerator that is slower than the base system. For TPC-H-Plus on DuckDB, all model options happen to result in the same accelerator choices. The mean value and MLP baselines have higher speedups because they require fewer features (meaning a faster optimization time) compared to the other two models.

The ALP NN provides the best speedup in three of the four database/workload combinations. It makes a better run time prediction for a domain negation accelerator instance, which lets Tailwind correctly choose that instance in the three cases (whereas the

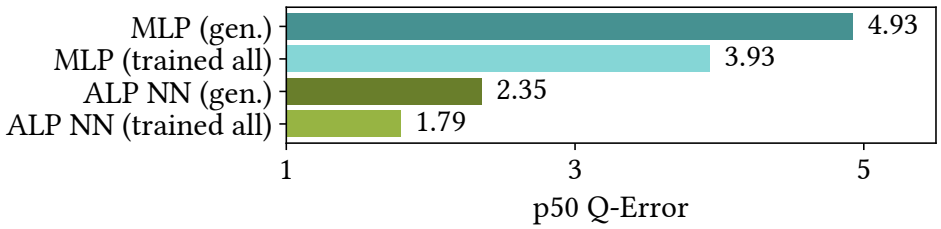


Figure 4.11: Model generalization across dataset sizes.

other models do not). While the simple models do well, they use hand-designed features for each accelerator. In contrast, the ALP neural network model uses one common featurization method.

Generalizing to Larger Datasets

Finally, we evaluate the out-of-distribution generalization of the ALP neural network as the dataset size increases. Specifically, we train the network for domain negation on Redshift using TPC-H scale factors (SF) 1 and 10, and test on SF = 100. We compare this generalizing configuration (denoted by the "gen." suffix) against the same model trained on all three scale factors. We repeat this setup for a standard MLP too. Figure 4.11 shows the p50 test Q-error, where lower is better. **From these results, we can conclude that the ALP network generalizes better than the MLP to unseen dataset sizes.** The ALP network's p50 Q-error increases by only 0.56 when generalizing, compared to a larger increase of 1.00 for the MLP.

4.7.2 Space Budget Sensitivity

Next, we study the space budget's effect on Tailwind's acceleration as it varies from 0.001% to 10%. Figure 4.10 shows our results. The horizontal axis is in log scale and the vertical axis is the workload's geomean speedup (higher is better). We draw two conclusions.

Tailwind's speedup increases with a larger space budget and it accelerates the workload with budgets as small as 0.01%. This is because we can use more accelerator instances given a larger budget. At small space budgets (up to 0.01%), the naïve strategy's speedup increases; there are only a few usable candidates and they happen to lead to a speedup. However, cost models are still important to use, as the naïve strategy causes slowdowns at large space budgets and its speedup is always below 1.0× for TPC-H-Plus on Redshift. The naïve strategy has a slightly higher speedup compared to Tailwind on TPC-H Redshift up to a budget of 0.01% due to the computational overhead of using Tailwind's models.

With larger space budgets, the naïve strategy degrades performance. A larger space budget allows for more accelerator candidates, but the naïve strategy does not consider whether these options are beneficial before using them (hence the worsening slowdowns). On Redshift, the slowdowns are mostly from slow intermediate result transfers. On DuckDB,

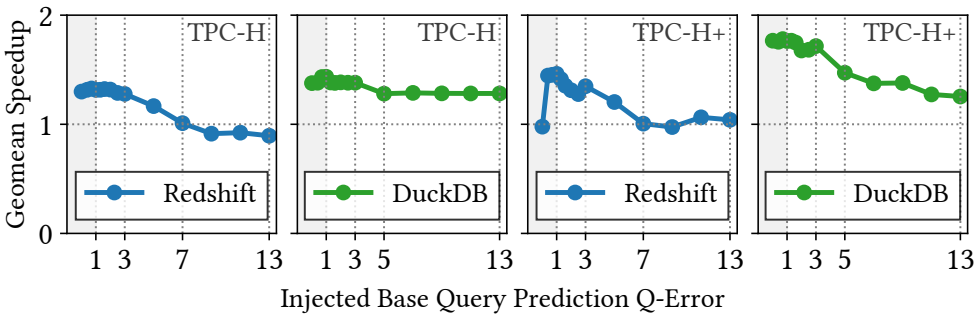


Figure 4.12: Robustness to query run time prediction error.

Table 4.5: TAILWIND's query planning overhead.

		TPC-H		TPC-H-Plus	
RDBMS	**Percentile**	**Time**	**Percent**	**Time**	**Percent**
Redshift	p50	65.1 ms	1.88%	75.5 ms	1.09%
	p90	109 ms	43.7%	202 ms	3.14%
DuckDB	p50	4.11 ms	0.0900%	5.15 ms	0.0804%
	p90	14.1 ms	1.69%	528 ms	2.85%

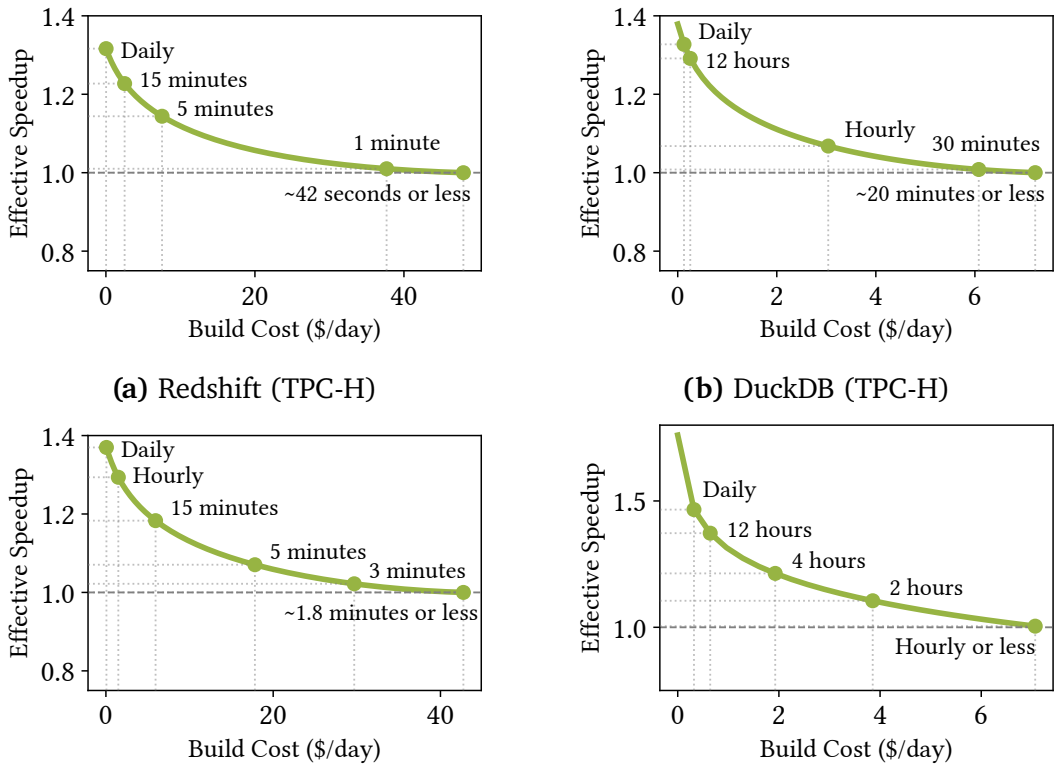
the slowdowns beyond a 1% budget are because the naïve strategy selects slow domain negation instances.

4.7.3 Query Run Time Prediction Robustness

Now we study how the query run time predictor's accuracy affects TAILWIND's decisions. We inject error into the ground truth query run times, and measure the geomean speedup on our workloads. Figure 4.12 shows our results. The horizontal axis is the injected Q-error; we test over-predictions up to a Q-error of 13 and under-predictions (the shaded region) up to a Q-error of 1.99. From these results, we draw the following conclusions.

TAILWIND is robust to query run time prediction error on Redshift and DuckDB up to a Q-error of 3.0 (1.6 on TPC-H-Plus for Redshift). On Redshift, large query run time over-predictions cause TAILWIND to use accelerators whose data transfer time would exceed the accelerator's benefit because it thinks the original query is even slower. DuckDB is less sensitive to query prediction errors because the data transfer time is negligible. However, large over-predictions lead TAILWIND to select slow accelerators that appear faster than the inflated base query estimate, but are slower than the true time.

TAILWIND can achieve its results with existing query run time predictors. On both Redshift and DuckDB, TAILWIND maintains most of its performance up to a prediction Q-error of 3.0 (1.6 on TPC-H-Plus for Redshift). Prior work on query run time predictors shows that this accuracy is achievable [87, 116, 117, 119, 154, 173, 195, 197, 200].



(a) Redshift (TPC-H) **(b)** DuckDB (TPC-H)

(c) Redshift (TPC-H-Plus) **(d)** DuckDB (TPC-H-Plus)

Figure 4.13: Impact of simulated bulk writes (e.g., ETLs) on TAILWIND's achieved speedup as we vary their frequency.

4.7.4 Runtime Overhead

Table 4.5 shows that TAILWIND's overhead, comprising plan enumeration and performance prediction, is below 5% in all but one case. Relative overhead is generally lower on TPC-H-Plus than TPC-H because its queries run longer. End-to-end latency still improves despite this overhead (Figure 4.5). For Redshift, most overhead comes from remote calls to the cardinality estimator. Since our Redshift experiments ran outside AWS, they have higher overhead than DuckDB. Replacing the remote call with an embedded estimator would reduce latency. For workloads comprising short-running queries, users can have TAILWIND run optimization in the background when it first encounters a query and use only *cached* accelerator execution plans. Since industrial workloads have repetition [152, 158], this approach would still allow speedups without slowing first-run latency.

4.7.5 Impact of Writes

TAILWIND's accelerators may use pre-computed state, which writes can make stale. TAILWIND targets engines running with bulk writes (common in OLAP settings [48]). After a bulk write, TAILWIND routes queries to the underlying RDBMS instead of its stale accelerators. In

the background, it rebuilds the accelerators and uses them until the next write. We leave incremental accelerator state maintenance [8, 43, 205] to future work. Here, we study how write frequency affects Tailwind's speedups. Under this write model, Tailwind is beneficial when the time between writes exceeds the rebuild time. Using the 10% budget results from Sections 4.6.1 and 4.6.2, we measure speedups over a one-day period by simulating write frequencies from once a day to once a minute. Queries arriving during a rebuild run on the RDBMS; those arriving after a rebuild but before a write use Tailwind. We assume a cloud setting where rebuilds can be offloaded to additional compute at additional cost.

Figure 4.13 shows our results. The vertical axis is the workload's geomean speedup over all queries. The horizontal axis is the build cost, which is the time spent rebuilding the accelerators multiplied by the on-demand EC2 [18] price of an equivalent VM performing the rebuilding. The labeled points represent different write frequencies (e.g., once every 5 minutes, etc.). When the time between writes is equal to or less than the build time, the effective workload speedup is 1.0× because the rebuild will not keep up with the new writes.

Tailwind supports daily bulk writes with a negligible performance impact. Our rebuild times are 47 seconds (Redshift, TPC-H), 25 minutes (DuckDB, TPC-H), 2 minutes (Redshift, TPC-H-Plus), and around 1 hour (DuckDB, TPC-H-Plus). Overall, the build time is much less than 24 hours, meaning Tailwind can provide its speedups with daily ETLs. Our Redshift setup has more resources available for accelerator building, so it supports more frequent writes (e.g., every 15 minutes) with only a small speedup trade-off.

4.8 Related Work

Database extensions, UDFs, UDOs. User-defined functions and operators (UDFs/UDOs) [33, 71, 163, 169], extensions [64, 149], and declarative sub-operators [94] let users add functionality to an RDBMS (e.g., query accelerators, among other uses). However, these mechanisms are constrained by the host RDBMS's extension framework, which (i) limits which accelerators it can support (e.g., Redshift only supports scalar UDFs [28]), (ii) may provide limited optimizer support [50, 189], and (iii) couples implementations to a particular RDBMS. Declarative sub-operators [94], in particular, require modifying the underlying RDBMS to support their operator abstraction, whereas Tailwind is non-invasive.

Query plan patterns. Abstract operator trees [107] and Calcite [35] also provide methods to specify logical plan patterns, but both are less expressive than ALPs. Abstract operator trees use abstract edges to capture a varying number of unary operators [107], meaning they cannot match arbitrarily nested binary operators (e.g., left-deep joins). Calcite's rules define fixed-depth patterns, meaning they cannot match variable length operator chains (e.g., arbitrarily nested filters) in a single rule application. ALPs can express both patterns using repetitions.

Auto materialized views. Choosing accelerator(s) to instantiate and use is similar to automated materialized view (MV) selection [7, 124, 162] and exploitation [75, 82]. Our

problem setting differs in two key ways. First, MV selection algorithms can materialize any sub-query. Tailwind uses a fixed set of accelerator ALPs, which changes the problem as it constrains the decision space. Second, Tailwind makes query planning decisions over ALPs, which each represent a set of query fragments. An MV is only a single concrete query sub-expression.

Auto index selection. Tailwind's accelerator instance selection problem is similar to automatic index selection [7, 49, 58, 194]. The key difference is that accelerators are more general than indexes as they can represent any query fragment, which requires a new approach. Indeed, indexes are one specific kind of accelerator.

Micro-adaptivity. Micro-adaptive techniques accelerate queries by finding low-level optimizations that exploit the hardware and data properties during query execution [81, 120, 151]. Compared to Tailwind, micro-adaptive techniques occupy a different spot on the speedup versus integration effort trade-off spectrum. To get speedups using low-level optimizations and to rapidly switch between optimizations at runtime, micro-adaptive techniques must be deeply integrated into the query execution engine. In contrast, Tailwind does not require code changes to the underlying RDBMS, allowing it to even work with proprietary systems (e.g., Redshift).

Synthesizing query engines. Orthogonal work generates specialized query engines [70, 106, 181, 190]. However, users must declare all query templates upfront and only queries matching the template can then leverage the custom executor. Tailwind, in contrast, replaces subqueries with accelerators. One instantiated accelerator can thus support many queries (e.g., whenever its supported fragment appears as a subquery). LLM-based query engine generators [106, 190] could also help generate Tailwind accelerators, which we leave to future work.

4.9 Conclusion

We presented abstract logical plans (ALPs) and Tailwind: a new non-invasive query planning and execution framework that provides a practical way to integrate query accelerators with any RDBMS that supports data import/export. ALPs and a layer of indirection let Tailwind strike the sweet spot between specializing the RDBMS for performance and maintaining its generality, all without having to modify its source code. Our three case studies show that users can integrate workload-specific accelerators that Tailwind automatically applies when beneficial, achieving geomean speedups of up to 1.38×, 1.76×, and 1.28×.

Chapter 5

Related Work

In this chapter, we review work that is related to the virtual data infrastructure abstraction.

Polystores and federated database management systems. Prior work on polystores [6, 11, 67, 147, 185, 188, 204] and federated databases [37, 41, 42, 73, 90, 93, 150, 161, 203] also aims to distribute query workloads across multiple heterogeneous engines. Unlike a virtual data infrastructure, these systems focus on unifying heterogeneous data models and query languages and optimizing queries within a given set of engines and hardware configuration, leaving it up to engineers to deploy, maintain, size, and curate the underlying systems. In contrast, a virtual data infrastructure decouples application requirements from the underlying physical engines. This decoupling enables automated management of the underlying physical engines and the ability to jointly optimize query execution with engine composition and data placement (Chapter 3).

Data lakehouses and fabrics. Data lakehouses (and fabrics) process, curate, and store data directly on object storage and provide access to data through different front-end database engines [34, 121]. For end-users, virtual data infrastructures look like data lakehouses: a number of different front-end engines operating over a managed data layer. Indeed, a data lakehouse could be implemented using a virtual data infrastructure (Section 2.4). However, the focus and emphasis of the two approaches are quite different. A virtual data infrastructure, with a planner, focuses on the automatic management of multiple structured data engines. Lakehouses typically rely on open direct-access data formats, support unstructured and semi-structured data as a first-class concern, and do not perform automated infrastructure design. Moreover, unlike virtual data infrastructures, lakehouses are still fundamentally a physical architectural design. They do not separate application use cases from the underlying database engines.

Instance-optimized, self-driving, and auto-tuning systems. Recent work has proposed techniques to automatically (i) adapt data systems to the workload [1, 61, 62, 100–102, 104, 116–118, 129, 142, 199, 202], (ii) manage complex systems [113, 127, 139–141, 157], (iii) adapt cloud database instance sizing [135–137, 184], and (iv) tune their knobs [95, 138, 183]. In contrast, a virtual data infrastructure's planner optimizes an entire multi-engine data infrastructure instead of tuning individual services. The virtual data infrastructure

planner can be seen as applying instance-optimization at the granularity of cloud database services instead of within a database engine [101].

Simplifying and optimizing the cloud. Like virtual data infrastructures, recent research has explored ways to simplify and optimize the design and operation of cloud infrastructures. These thrusts include (i) high-level cloud programming abstractions [52, 112, 126, 156], (ii) infrastructure as code [22, 86], (iii) a simulation framework for cloud data analytics [123], (iv) enhancing cross-cloud compatibility [47, 182], and (v) improving resilience across services [111]. The key difference is that a virtual data infrastructure focuses on simplifying cloud infrastructures containing multiple relational database systems while automatically optimizing their use for cost under a performance constraint.

Single-system (HTAP) solutions. Another way to handle diverse data workloads is to use a single specialized (e.g., HTAP) database system designed for high performance across many workloads [69, 89, 97, 105, 164]. However, these single-system solutions can be difficult to migrate to and they limit users to their specific feature set. A virtual data infrastructure, on the other hand, focuses on decomposing application requirements into distinct data use cases. These use cases, expressed as VDBEs, can then be realized using any suitable physical engine, including a specialized HTAP system. In this view, virtual data infrastructures are more broadly applicable than such single-system solutions since they can use such systems in their implementation.

Chapter 6

Conclusion and Future Work

6.1 Thesis Summary and Takeaways

This thesis introduced *data infrastructure virtualization*: separating a data infrastructure's logical application-facing requirements from the physical engines that could be used to realize them.

We began in Chapter 2 by defining the virtual database engine (VDBE) abstraction, which is a specification of a database's logical properties: its tables, supported SQL dialect, maximum staleness constraint with respect to other VDBEs in the infrastructure, and an optional performance constraint. VDBEs are a powerful abstraction because the same table can be in multiple VDBEs; each VDBE represents a logical snapshot over a set of tables. This property lets data architects design their infrastructure around data use cases instead of choosing specific physical engines.

We then presented blueprint planning in Chapter 3: our solution for automatically realizing a virtual infrastructure using physical database engines. The key takeaway is to cast the infrastructure design problem as a cost-based optimization problem over a unified design space of plans called *blueprints*. This approach allows us to systematically search for an optimized design for a given workload by leveraging learned models to predict the utility of candidate blueprints. We showed that this technique achieves 1.6×–13× lower cost than existing cloud autoscaling serverless engines.

Finally, in Chapter 4, we showed how we could leverage VDBEs to non-invasively integrate specialized query accelerators into any RDBMS that supports data import/export. Central to our approach are ALPs, a new abstraction that serves three purposes: (i) describing an accelerator's semantics, (ii) providing an interface for the accelerator implementation, and (iii) serving as a systematic object to featurize for performance prediction. The key takeaway is that ALPs and VDBEs let Tailwind strike the sweet spot between specializing the RDBMS for performance and maintaining its generality without extraneous engineering overhead. Our three case studies show that users can integrate workload-specific accelerators that Tailwind automatically applies when beneficial, achieving geomean speedups of up to 1.38×, 1.76×, and 1.28×.

We hope that this thesis helps show the power of data infrastructure virtualization

and the physical design independence it provides: how it simplifies the design of complex multi-engine infrastructures, enables automated data infrastructure design, and supports non-invasive and targeted per-query execution optimizations.

6.2 Future Work

While this thesis makes important contributions towards defining data infrastructure virtualization and making it practical, there are still plenty of exciting related research questions to pursue. In the following, we discuss a few of these future avenues of research.

Transactional isolation levels. Our VDBE abstraction provides serializable isolation within its isolation scopes. This is a strong correctness condition that makes it easier to reason about VDBEs with overlapping table sets. However, weaker isolation levels [4] such as snapshot isolation [38] are also available for use in practice [45]. An interesting direction for future research is to expand a virtual data infrastructure's transactional isolation semantics to support weaker isolation levels.

Specifying performance constraints. In Chapter 3, we presented a method that realizes a virtual infrastructure using concrete database engines while minimizing cost subject to performance constraints. Users have to declare performance constraints a priori in the form of p90 query latency values. While conceptually simple to understand, in practice, users may not know what p90 query latency to choose for all of their data use cases. Moreover, latency ceilings impose a hard cut-off at the specified value. While this may be appropriate for strict transactional latency guarantees, analytical workloads may instead allow for a soft performance target, where small violations are acceptable (e.g., users specifying a latency ceiling of 30 minutes may find a latency of 31 minutes acceptable too). Thus, an area of future research lies in designing new, intuitive ways to specify performance constraints.

Dynamic query routing and planning. The blueprint planning solution we presented makes static query routing decisions (i.e., our chosen query routing policies are static and do not change until a new blueprint is selected). While this approach is fine for workloads with gradual changes, it does not gracefully handle workloads with rapid load changes (e.g., bursts of arriving queries). An interesting next step is to extend our blueprint planner to consider dynamic routing policies that, for example, can route queries to different engines depending on their current load.

Optimizing for maintaining freshness. Virtual database engines have maximum staleness constraints. This implicitly means that the virtual infrastructure runtime must be able to replicate writes to the VDBE's physical snapshot quickly enough to meet the constraints. Our blueprint planning solution made a simplifying assumption that all writes originate on Aurora and that the analytical VDBE had a permissive enough staleness constraint that periodic data replication is enough to meet the constraint. In practice, users may declare more stringent constraints. In that case, the blueprint planner must consider the different physical replication options available (e.g., export and load, zero-ETL [29], co-locate VDBEs)

when making table placement decisions. This more complex optimization problem would be an interesting area for future research.

Automatically finding acceleration candidates. In Chapter 4, our method for accelerating queries assumes end-users are sophisticated and already know what parts of their queries they can accelerate. However, finding acceleration opportunities can be almost as challenging as implementing a specialized acceleration technique. Moreover, finding these acceleration opportunities might be out of reach for the average user. An interesting line of future work would investigate methods for automatically finding acceleration opportunities given a query workload and dataset.

Automatically synthesizing query accelerators. Related to automatically finding acceleration opportunities, a follow-on line of work could explore synthesizing (i.e., generating) query accelerators entirely. Recently, large language models have made significant progress in code generation quality, making this direction increasingly practical [106, 190]. A major challenge in this direction is ensuring that the generated code is correct without having to resort to manual review.

References

[1] M. Abebe, H. Lazu, and K. Daudjee. “Proteus: Autonomous Adaptive Storage for Mixed Workloads.” In: *Proceedings of the 2022 International Conference on Management of Data*. SIGMOD ’22. 2022, pp. 700–714. DOI: 10.1145/3514221.3517834.

[2] L. Abraham, J. Allen, O. Barykin, V. Borkar, B. Chopra, C. Gerea, D. Merl, J. Metzler, D. Reiss, S. Subramanian, J. L. Wiener, and O. Zed. “Scuba: Diving Into Data at Facebook.” *Proceedings of the VLDB Endowment*, **6**(11), Aug. 2013, pp. 1057–1067. DOI: 10.14778/2536222.2536231.

[3] I. Adan and J. Resing. *Queueing Systems*. https://www.win.tue.nl/~iadan/queueing.pdf. 2015.

[4] A. Adya. “Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions.” PhD thesis. Massachusetts Institute of Technology, 1999.

[5] A. Agiwal, K. Lai, G. N. B. Manoharan, I. Roy, J. Sankaranarayanan, H. Zhang, T. Zou, M. Chen, Z. Chen, M. Dai, T. Do, H. Gao, H. Geng, R. Grover, B. Huang, Y. Huang, Z. Li, J. Liang, T. Lin, L. Liu, Y. Liu, X. Mao, Y. Meng, P. Mishra, J. Patel, R. S. R., V. Raman, S. Roy, M. S. Shishodia, T. Sun, Y. Tang, J. Tatemura, S. Trehan, R. Vadali, P. Venkatasubramanian, G. Zhang, K. Zhang, Y. Zhang, Z. Zhuang, G. Graefe, D. Agrawal, J. Naughton, S. Kosalge, and H. Hacıgümüş. “Napa: Powering Scalable Data Warehousing with Robust Query Performance at Google.” *Proceedings of the VLDB Endowment*, **14**(12), July 2021, pp. 2986–2997. DOI: 10.14778/3476311.3476377.

[6] D. Agrawal, S. Chawla, B. Contreras-Rojas, A. Elmagarmid, Y. Idris, Z. Kaoudi, S. Kruse, J. Lucas, E. Mansour, M. Ouzzani, P. Papotti, J.-A. Quiané-Ruiz, N. Tang, S. Thirumuruganathan, and A. Troudi. “RHEEM: Enabling Cross-Platform Data Processing: May the Big Data Be with You!” *Proceedings of the VLDB Endowment*, **11**(11), July 2018, pp. 1414–1427. DOI: 10.14778/3236187.3236195.

[7] S. Agrawal, S. Chaudhuri, and V. R. Narasayya. “Automated Selection of Materialized Views and Indexes in SQL Databases.” In: *Proceedings of the 26th International Conference on Very Large Data Bases*. VLDB ’00. 2000, pp. 496–505. DOI: 10.5555/645926.671701.

[8] Y. Ahmad and C. Koch. “DBToaster: A SQL Compiler for High-Performance Delta Processing in Main-Memory Databases.” *Proceedings of the VLDB Endowment*, **2**(2), Aug. 2009, pp. 1566–1569. DOI: 10.14778/1687553.1687592.

[9] A. Aiken and B. R. Murphy. “Implementing Regular Tree Expressions.” In: *Proceedings of the 5th ACM Conference on Functional Programming Languages and Computer Architecture*. FPCA ’91. 1991, pp. 427–447. DOI: 10.1007/3540543961_21.

[10] M. Akdere, U. Çetintemel, M. Riondato, E. Upfal, and S. B. Zdonik. “Learning-based Query Performance Modeling and Prediction.” In: *IEEE 28th International Conference on Data Engineering*. ICDE ’12. 2012, pp. 390–401. DOI: 10.1109/ICDE.2012.64.

[11] R. Alotaibi, D. Bursztyn, A. Deutsch, I. Manolescu, and S. Zampetakis. “Towards Scalable Hybrid Stores: Constraint-Based Rewriting to the Rescue.” In: *Proceedings of the 2019 International Conference on Management of Data*. SIGMOD ’19. 2019, pp. 1660–1677. DOI: 10.1145/3299869.3319895.

[12] Amazon Web Services. *Achieve up to 35% better price/performance with Amazon Aurora using new Graviton2 instances*. https://aws.amazon.com/about-aws/whats-new/2021/03/achieve-up-to-35-percent-better-price-performance-with-amazon-aurora-using-new-graviton2-instances/. 2021. (Visited on 08/11/2026).

[13] Amazon Web Services. *AWS announces Amazon Aurora I/O-Optimized*. https://aws.amazon.com/about-aws/whats-new/2023/05/amazon-aurora-i-o-optimized/. 2023. (Visited on 08/11/2026).

[14] Amazon Web Services. *Amazon Athena*. https://aws.amazon.com/athena/. 2026. (Visited on 08/11/2026).

[15] Amazon Web Services. *Amazon Athena Pricing*. https://aws.amazon.com/athena/pricing/. 2026. (Visited on 08/11/2026).

[16] Amazon Web Services. *Amazon Aurora*. https://aws.amazon.com/rds/aurora/. 2026. (Visited on 08/11/2026).

[17] Amazon Web Services. *Amazon Aurora Pricing*. https://aws.amazon.com/rds/aurora/pricing/. 2026. (Visited on 08/11/2026).

[18] Amazon Web Services. *Amazon EC2*. https://aws.amazon.com/ec2/. 2026. (Visited on 08/11/2026).

[19] Amazon Web Services. *Amazon Redshift*. https://aws.amazon.com/redshift/. 2026. (Visited on 08/11/2026).

[20] Amazon Web Services. *Amazon Redshift Pricing*. https://aws.amazon.com/redshift/pricing/. 2026. (Visited on 08/11/2026).

[21] Amazon Web Services. *Amazon S3*. https://aws.amazon.com/s3/. 2026. (Visited on 08/11/2026).

[22] Amazon Web Services. *AWS CloudFormation*. https://aws.amazon.com/pm/cloudformation/. 2026. (Visited on 08/11/2026).

[23] Amazon Web Services. *AWS RDS Proxy*. https://aws.amazon.com/rds/proxy/. 2026. (Visited on 08/11/2026).

[24] Amazon Web Services. *Cloud Databases on AWS*. https://aws.amazon.com/products/databases/. 2026. (Visited on 08/11/2026).

[25] Amazon Web Services. *Data Lakes and Analytics on AWS*. https://aws.amazon.com/big-data/datalakes-and-analytics/. 2026. (Visited on 08/11/2026).

[26] Amazon Web Services. *How do I resize an Amazon Redshift cluster?* https://repost.aws/knowledge-center/resize-redshift-cluster. 2026. (Visited on 08/11/2026).

[27] Amazon Web Services. *SQL Commands - Amazon Redshift*. https://docs.aws.amazon.com/redshift/latest/dg/c_SQL_commands.html. 2026. (Visited on 08/11/2026).

[28] Amazon Web Services. *User-defined functions in Amazon Redshift*. https://docs.aws.amazon.com/redshift/latest/dg/user-defined-functions.html. 2026. (Visited on 08/11/2026).

[29] Amazon Web Services. *zero-ETL*. https://aws.amazon.com/what-is/zero-etl/. 2026. (Visited on 08/11/2026).

[30] G. M. Amdahl. "Validity of the single processor approach to achieving large scale computing capabilities." In: *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*. AFIPS '67 (Spring). 1967, pp. 483–485. DOI: 10.1145/1465482.1465560.

[31] L. Antova, D. Bryant, T. Cao, M. Duller, M. A. Soliman, and F. M. Waas. "Rapid Adoption of Cloud Data Warehouse Technology Using Datometry Hyper-Q." In: *Proceedings of the 2018 International Conference on Management of Data*. SIGMOD '18. 2018, pp. 825–839. DOI: 10.1145/3183713.3190652.

[32] Apache Software Foundation. *Apache Arrow*. https://arrow.apache.org. 2026. (Visited on 08/11/2026).

[33] S. Arch, Y. Liu, T. C. Mowry, J. M. Patel, and A. Pavlo. "The Key to Effective UDF Optimization: Before Inlining, First Perform Outlining." *Proceedings of the VLDB Endowment*, **18**(1), Feb. 2025, pp. 1–13. DOI: 10.14778/3696435.3696436.

[34] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia. "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics." In: *Proceedings of the 11th Annual Conference on Innovative Data Systems Research*. CIDR '21. 2021. URL: https://vldb.org/cidrdb/papers/2021/cidr2021_paper17.pdf.

[35] E. Begoli, J. Camacho-Rodríguez, J. Hyde, M. J. Mior, and D. Lemire. "Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources." In: *Proceedings of the 2018 International Conference on Management of Data*. SIGMOD '18. 2018, pp. 221–230. DOI: 10.1145/3183713.3190662.

[36] A. Belabbaci, H. Cherroun, L. Cleophas, and D. Ziadi. "Tree Pattern Matching From Regular Tree Expressions." *Kybernetika*, **54**(2), 2018, pp. 221–242.

[37] G. Bent, P. Dantressangle, D. Vyvyan, A. Mowshowitz, and V. Mitsou. "A Dynamic Distributed Federated Database." In: *Proceedings of the 2nd Annual Conference on International Technology Alliance*. ACITA '08. 2008.

[38] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil. "A Critique of ANSI SQL Isolation Levels." In: *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*. SIGMOD '95. 1995, pp. 1–10. DOI: 10.1145/223784.223785.

[39] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[40] P. Boncz, T. Neumann, and O. Erling. "TPC-H Analyzed: Hidden Messages and Lessons Learned from an Influential Benchmark." In: *Technology Conference on Performance Evaluation and Benchmarking*. Springer. 2013, pp. 61–76. DOI: 10.1007/978-3-319-04936-6_5.

[41] Y. Breitbart, H. Garcia-Molina, and A. Silberschatz. "Overview of Multidatabase Transaction Management." *VLDB Journal*, **1**, Oct. 1992, pp. 181–239. DOI: 10.1145/1925805.1925811.

[42] Y. Breitbart and A. Silberschatz. "Multidatabase Update Issues." In: *Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data*. SIGMOD '88. 1988, pp. 135–142. DOI: 10.1145/50202.50217.

[43] M. Budiu, T. Chajed, F. McSherry, L. Ryzhyk, and V. Tannen. "DBSP: Automatic Incremental View Maintenance for Rich Query Languages." *Proceedings of the VLDB Endowment*, **16**(7), Mar. 2023, pp. 1601–1614. DOI: 10.14778/3587136.3587137.

[44] M. Butrovich, K. Ramanathan, J. Rollinson, W. S. Lim, W. Zhang, J. Sherry, and A. Pavlo. "Tigger: A Database Proxy That Bounces with User-Bypass." *Proceedings of the VLDB Endowment*, **16**(11), 2023, pp. 3335–3348. DOI: 10.14778/3611479.3611530.

[45] M. J. Cahill, U. Röhm, and A. D. Fekete. "Serializable Isolation for Snapshot Databases." In: *Proceedings of the ACM SIGMOD International Conference on Management of Data*. SIGMOD '08. 2008, pp. 729–738. DOI: 10.1145/1376616.1376690.

[46] M. J. Carey and D. Kossmann. "On Saying "Enough Already!" in SQL." In: *Proceedings of the International Conference on Management of Data*. SIGMOD '97. 1997. DOI: 10.1145/253260.253302.

[47] S. Chasins, A. Cheung, N. Crooks, A. Ghodsi, K. Goldberg, J. E. Gonzalez, J. M. Hellerstein, M. I. Jordan, A. D. Joseph, M. W. Mahoney, A. Parameswaran, D. Patterson, R. A. Popa, K. Sen, S. Shenker, D. Song, and I. Stoica. *The Sky Above The Clouds*. 2022. arXiv: 2205.07147 [cs.DC].

[48] S. Chaudhuri and U. Dayal. "An Overview of Data Warehousing and OLAP Technology." *SIGMOD Record*. SIGMOD '97, **26**(1), Mar. 1997, pp. 65–74. DOI: 10.1145/248603.248616.

[49] S. Chaudhuri and V. Narasayya. "AutoAdmin "What-if" Index Analysis Utility." *SIGMOD Record*, **27**(2), June 1998, pp. 367–378. DOI: 10.1145/276305.276337.

[50] S. Chaudhuri and K. Shim. "Query Optimization in the Presence of Foreign Functions." In: *Proceedings of the 19th International Conference on Very Large Data Bases*. VLDB '93. 1993, pp. 529–542. URL: https://www.vldb.org/conf/1993/P529.PDF.

[51] S. Chaudhuri, M. Datar, and V. Narasayya. “Index Selection for Databases: A Hardness Study and A Principled Heuristic Solution.” *IEEE Transactions on Knowledge and Data Engineering*, **16**(11), 2004, pp. 1313–1323. DOI: 10.1109/TKDE.2004.75.

[52] A. Cheung, N. Crooks, J. M. Hellerstein, and M. Milano. *New Directions in Cloud Programming*. 2021. arXiv: 2101.01159 [cs.DC].

[53] Y. Chi, H. J. Moon, H. Hacıgümüş, and J. Tatemura. “SLA-Tree: A Framework for Efficiently Supporting SLA-based Decisions in Cloud Computing.” In: *Proceedings of the 14th International Conference on Extending Database Technology*. EDBT ’11. 2011, pp. 129–140. DOI: 10.1145/1951365.1951383.

[54] D. Comer. “The Difficulty of Optimum Index Selection.” *ACM Transactions on Database Systems*, **3**(4), Dec. 1978, pp. 440–445. DOI: 10.1145/320289.320296.

[55] H. Comon, M. Dauchet, R. Gilleron, F. Jacquemard, D. Lugiez, C. Löding, S. Tison, and M. Tommasi. *Tree Automata Techniques and Applications*. 2008.

[56] cppreference.com. *C++ named requirements: Compare*. https://en.cppreference.com/w/cpp/named_req/Compare. 2026. (Visited on 08/11/2026).

[57] Databricks, Inc. *Introducing Lakehouse//RT: Real-Time Performance on a Unified Lakehouse*. https://www.databricks.com/blog/introducing-lakehousert-real-time-performance-unified-lakehouse. 2026. (Visited on 08/11/2026).

[58] B. Ding, S. Das, R. Marcus, W. Wu, S. Chaudhuri, and V. R. Narasayya. “AI Meets AI: Leveraging Query Executions to Improve Index Recommendations.” In: *Proceedings of the 2019 International Conference on Management of Data*. SIGMOD ’19. 2019, pp. 1241–1258. DOI: 10.1145/3299869.3324957.

[59] J. Ding, M. Abrams, S. Bandyopadhyay, L. Di Palma, Y. Ji, D. Pagano, G. Paliwal, P. Parchas, P. Pfeil, O. Polychroniou, G. Saxena, A. Shah, A. Voloder, S. Xiao, D. Zhang, and T. Kraska. “Automated Multidimensional Data Layouts in Amazon Redshift.” In: *Companion of the 2024 International Conference on Management of Data*. SIGMOD ’24. Santiago AA, Chile, 2024, pp. 55–67. DOI: 10.1145/3626246.3653379.

[60] J. Ding, R. Marcus, A. Kipf, V. Nathan, A. Nrusimha, K. Vaidya, A. van Renen, and T. Kraska. “SageDB: An Instance-Optimized Data Analytics System.” *Proceedings of the VLDB Endowment*, **15**(13), Sept. 2022, pp. 4062–4078. DOI: 10.14778/3565838.3565857.

[61] J. Ding, U. F. Minhas, B. Chandramouli, C. Wang, Y. Li, Y. Li, D. Kossmann, J. Gehrke, and T. Kraska. “Instance-Optimized Data Layouts for Cloud Analytics Workloads.” In: *Proceedings of the 2021 International Conference on Management of Data*. SIGMOD ’21. 2021, pp. 418–431. DOI: 10.1145/3448016.3457270.

[62] J. Ding, V. Nathan, M. Alizadeh, and T. Kraska. “Tsunami: A Learned Multi-Dimensional Index for Correlated Data and Skewed Workloads.” *Proceedings of the VLDB Endowment*, **14**(2), 2020, pp. 74–86. DOI: 10.14778/3425879.3425880.

[63] J. Doner. “Tree Acceptors and Some of Their Applications.” *Journal of Computer and System Sciences*, **4**(5), 1970, pp. 406–451. DOI: 10.1016/S0022-0000(70)80041-1.

[64] DuckDB Authors. *DuckDB Extensions*. https://duckdb.org/docs/extensions/working_with_extensions.html. 2026. (Visited on 08/11/2026).

[65] DuckDB Labs. *DuckDB: An in-process SQL OLAP database management system*. https://duckdb.org/. 2026. (Visited on 08/11/2026).

[66] J. Duggan, U. Çetintemel, O. Papaemmanouil, and E. Upfal. “Performance Prediction for Concurrent Database Workloads.” In: *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data*. SIGMOD ’11. Athens, Greece, 2011, pp. 337–348. DOI: 10.1145/1989323.1989359.

[67] J. Duggan, A. J. Elmore, M. Stonebraker, M. Balazinska, B. Howe, J. Kepner, S. Madden, D. Maier, T. Mattson, and S. Zdonik. “The BigDAWG Polystore System.” *SIGMOD Record*, **44**(2), Aug. 2015, pp. 11–16. DOI: 10.1145/2814710.2814713.

[68] J. Duggan, O. Papaemmanouil, U. Çetintemel, and E. Upfal. “Contender: A Resource Modeling Approach for Concurrent Query Performance Prediction.” In: *Proceedings of the 17th International Conference on Extending Database Technology*. EDBT ’14. 2014, pp. 109–120. DOI: 10.5441/002/EDBT.2014.11.

[69] F. Färber, N. May, W. Lehner, P. Große, I. Müller, H. Rauhe, and J. Dees. “The SAP HANA Database – An Architecture Overview.” *IEEE Data Engineering Bulletin*, **35**, Mar. 2012, pp. 28–33.

[70] J. Feser, S. Madden, N. Tang, and A. Solar-Lezama. “Deductive Optimization of Relational Data Storage.” *Proceedings of the ACM on Programming Languages*, **4**(OOPSLA), Nov. 2020. DOI: 10.1145/3428238.

[71] Y. Foufoulas, A. Simitsis, L. Stamatogiannakis, and Y. Ioannidis. “YeSQL: “You extend SQL” with Rich and Highly Performant User-Defined Functions in Relational Databases.” *Proceedings of the VLDB Endowment*, **15**(10), June 2022, pp. 2270–2283. DOI: 10.14778/3547305.3547328.

[72] J. Fürnkranz and E. Hüllermeier, eds. *Preference Learning*. Springer-Verlag, 2010. ISBN: 978-3-642-14124-9. DOI: 10.1007/978-3-642-14125-6.

[73] D. Georgakopoulos, M. Rusinkiewicz, and A. P. Sheth. “On Serializability of Multidatabase Transactions Through Forced Local Conflicts.” In: *Proceedings of the Seventh International Conference on Data Engineering*. ICDE ’91. 1991, pp. 314–323. DOI: 10.1109/ICDE.1991.131479.

[74] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. “Neural Message Passing for Quantum Chemistry.” In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 1263–1272. URL: http://proceedings.mlr.press/v70/gilmer17a.html.

[75] J. Goldstein and P.-Å. Larson. “Optimizing Queries Using Materialized Views: A Practical, Scalable Solution.” *SIGMOD Record*, **30**(2), May 2001, pp. 331–342. DOI: 10.1145/376284.375706.

[76] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[77] Google. *Data definition language (DDL) statements in GoogleSQL*. https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/data-definition-language. 2026. (Visited on 08/11/2026).

[78] Google Cloud. *Google Cloud Databases and Data Analytics*. https://cloud.google.com/products?pds=CAQ#data-analytics. 2026. (Visited on 08/11/2026).

[79] Google, Inc. *BigQuery Omni*. https://docs.cloud.google.com/bigquery/docs/omni-introduction. 2026. (Visited on 08/11/2026).

[80] G. Graefe. “The Cascades Framework for Query Optimization.” *IEEE Data Engineering Bulletin*, **18**(3), 1995, pp. 19–29.

[81] T. Gubner and P. Boncz. “Excalibur: A Virtual Machine for Adaptive Fine-grained JIT-Compiled Query Execution based on VOILA.” *Proceedings of the VLDB Endowment*, **16**(4), Dec. 2022, pp. 829–841. DOI: 10.14778/3574245.3574266.

[82] A. Y. Halevy. “Answering Queries Using Views: A Survey.” *The VLDB Journal*, **10**(4), 2001, pp. 270–294. DOI: 10.1007/s007780100054.

[83] A. Hall, O. Bachmann, R. Buessow, S.-I. Ganceanu, and M. Nunkesser. “Processing a Trillion Cells per Mouse Click.” *Proceedings of the VLDB Endowment*, **5**(11), 2012, pp. 1436–1446. URL: https://vldb.org/pvldb/vol5/p1436_alexanderhall_vldb2012.pdf.

[84] Y. Han, Z. Wu, P. Wu, R. Zhu, J. Yang, L. W. Tan, K. Zeng, G. Cong, Y. Qin, A. Pfadler, Z. Qian, J. Zhou, J. Li, and B. Cui. “Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation.” *Proceedings of the VLDB Endowment*, **15**(4), 2021, pp. 752–765. DOI: 10.14778/3503585.3503586.

[85] M. Harchol-Balter. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013. ISBN: 9781107027503.

[86] HashiCorp. *Terraform*. https://developer.hashicorp.com/terraform. 2026. (Visited on 08/11/2026).

[87] B. Hilprecht and C. Binnig. “Zero-Shot Cost Models for Out-of-the-box Learned Cost Prediction.” *Proceedings of the VLDB Endowment*, **15**(11), 2022, pp. 2361–2374. DOI: 10.14778/3551793.3551799.

[88] T. K. Ho. “Random Decision Forests.” In: *Proceedings of 3rd International Conference on Document Analysis and Recognition*. Vol. 1. ICDAR ’95. 1995, pp. 278–282. DOI: 10.1109/ICDAR.1995.598994.

[89] D. Huang, Q. Liu, Q. Cui, Z. Fang, X. Ma, F. Xu, L. Shen, L. Tang, Y. Zhou, M. Huang, W. Wei, C. Liu, J. Zhang, J. Li, X. Wu, L. Song, R. Sun, S. Yu, L. Zhao, N. Cameron, L. Pei, and X. Tang. "TiDB: A Raft-Based HTAP Database." *Proceedings of the VLDB Endowment*, **13**(12), Aug. 2020, pp. 3072–3084. DOI: 10.14778/3415478.3415535.

[90] S.-Y. Hwang, E.-P. Lim, H.-R. Yang, S. Musukula, K. Mediratta, M. Ganesh, D. Clements, J. Stenoien, and J. Srivastava. "The MYRIAD Federated Database Prototype." In: *Proceedings of the 1994 ACM SIGMOD International Conference on Management of Data*. SIGMOD '94. 1994. DOI: 10.1145/191839.191986.

[91] InfluxData Inc. *InfluxDB*. https://www.influxdata.com/. 2026. (Visited on 08/11/2026).

[92] T. Jörg and S. Deßloch. "Towards Generating ETL Processes for Incremental Loading." In: *Proceedings of the 2008 International Symposium on Database Engineering & Applications*. IDEAS '08. 2008, pp. 101–110. DOI: 10.1145/1451940.1451956.

[93] V. Josifovski, P. Schwarz, L. Haas, and E. Lin. "Garlic: A New Flavor of Federated Query Processing for DB2." In: *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*. SIGMOD '02. 2002, pp. 524–532. DOI: 10.1145/564691.564751.

[94] M. Jungmair and J. Giceva. "Declarative Sub-Operators for Universal Data Processing." *Proceedings of the VLDB Endowment*, **16**(11), 2023, pp. 3461–3474. DOI: 10.14778/3611479.3611539.

[95] K. Kanellis, C. Ding, B. Kroth, A. Müller, C. Curino, and S. Venkataraman. "LlamaTune: Sample-Efficient DBMS Configuration Tuning." *Proceedings of the VLDB Endowment*, **15**(11), 2022, pp. 2953–2965. DOI: 10.14778/3551793.3551844.

[96] A. Karagiannis, P. Vassiliadis, and A. Simitsis. "Scheduling Strategies for Efficient ETL Execution." *Information Systems*, **38**(6), 2013, pp. 927–945. DOI: 10.1016/j.is.2012.12.001.

[97] A. Kemper and T. Neumann. "HyPer: A Hybrid OLTP & OLAP Main Memory Database System Based on Virtual Memory Snapshots." In: *Proceedings of the 2011 IEEE 27th International Conference on Data Engineering*. ICDE '11. 2011, pp. 195–206. DOI: 10.1109/ICDE.2011.5767867.

[98] T. Kersten, V. Leis, A. Kemper, T. Neumann, A. Pavlo, and P. Boncz. "Everything you always wanted to know about compiled and vectorized queries but were afraid to ask." *Proceedings of the VLDB Endowment*, **11**(13), Sept. 2018, pp. 2209–2222. DOI: 10.14778/3275366.3284966.

[99] A. Kim, M. Slot, D. G. Andersen, and A. Pavlo. "Anarchy in the Database: A Survey and Evaluation of Database Management System Extensibility." *Proceedings of the VLDB Endowment*, **18**(6), Feb. 2025, pp. 1962–1976. DOI: 10.14778/3725688.3725719.

[100] F. Kossmann, Z. Wu, E. Lai, N. Tatbul, L. Cao, T. Kraska, and S. Madden. "Extract-Transform-Load for Video Streams." *Proceedings of the VLDB Endowment*, **16**(9), May 2023, pp. 2302–2315. DOI: 10.14778/3598581.3598600.

[101] T. Kraska, M. Alizadeh, A. Beutel, E. H. Chi, A. Kristo, G. Leclerc, S. Madden, H. Mao, and V. Nathan. "SageDB: A Learned Database System." In: *Proceedings of the 9th Biennial Conference on Innovative Data Systems Research*. CIDR '19. 2019. URL: http://cidrdb.org/cidr2019/papers/p117-kraska-cidr19.pdf.

[102] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis. "The Case for Learned Index Structures." In: *Proceedings of the 2018 International Conference on Management of Data*. SIGMOD '18. 2018, pp. 489–504. DOI: 10.1145/3183713.3196909.

[103] T. Kraska, T. Li, S. Madden, M. Markakis, A. Ngom, Z. Wu, and G. X. Yu. "Check Out the Big Brain on BRAD: Simplifying Cloud Data Processing with Learned Automated Data Meshes." *Proceedings of the VLDB Endowment*, **16**(11), Aug. 2023, pp. 3293–3301. DOI: 10.14778/3611479.3611526.

[104] S. Krishnan, Z. Yang, K. Goldberg, J. Hellerstein, and I. Stoica. *Learning to Optimize Join Queries With Deep Reinforcement Learning*. 2019. arXiv: 1808.03196 [cs.DB].

[105] T. Lahiri, S. Chavan, M. Colgan, D. Das, A. Ganesh, M. Gleeson, S. Hase, A. Holloway, J. Kamp, T.-H. Lee, J. Loaiza, N. Macnaughton, V. Marwah, N. Mukherjee, A. Mullick, S. Muthulingam, V. Raja, M. Roth, E. Soylemez, and M. Zait. "Oracle Database In-Memory: A Dual Format In-Memory Database." In: *Proceedings of the 2015 IEEE 31st International Conference on Data Engineering*. ICDE '15. 2015, pp. 1253–1258. DOI: 10.1109/ICDE.2015.7113373.

[106] J. Lao and I. Trummer. *GenDB: The Next Generation of Query Processing – Synthesized, Not Engineered*. 2026. arXiv: 2603.02081 [cs.DB].

[107] J. Leeka and K. Rajan. "Incorporating Super-Operators in Big-Data Query Optimizers." *Proceedings of the VLDB Endowment*, **13**(3), 2019, pp. 348–361. DOI: 10.14778/3368289.3368299.

[108] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. "How Good Are Query Optimizers, Really?" *Proceedings of the VLDB Endowment*, **9**(3), 2015, pp. 204–215. DOI: 10.14778/2850583.2850594.

[109] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." In: *Advances in Neural Information Processing Systems*. NeurIPS '20. 2020. DOI: 10.5555/3495724.3496517.

[110] J. Li, A. C. König, V. R. Narasayya, and S. Chaudhuri. "Robust Estimation of Resource Consumption for SQL Queries using Statistical Techniques." *Proceedings of the VLDB Endowment*, **5**(11), 2012, pp. 1555–1566. DOI: 10.14778/2350229.2350269.

[111] T. Li, B. Chandramouli, S. Burckhardt, and S. Madden. "DARQ Matter Binds Everything: Performant and Composable Cloud Programming via Resilient Steps." *Proceedings of the ACM on Management of Data*, **1**(2), 2023. DOI: 10.1145/3589262.

[112] T. Li, B. Chandramouli, S. Burckhardt, and S. Madden. "Serverless State Management Systems." In: *Proceedings of the Conference on Innovative Data Research*. CIDR '24. 2024. URL: https://www.cidrdb.org/cidr2024/papers/p16-li.pdf.

[113] W. S. Lim, M. Butrovich, W. Zhang, A. Crotty, L. Ma, P. Xu, J. Gehrke, and A. Pavlo. "Database Gyms." In: *Proceedings of the Conference on Innovative Data Systems Research*. CIDR '23. 2023. URL: https://vldb.org/cidrdb/papers/2023/p27-lim.pdf.

[114] L. Lins, J. T. Klosowski, and C. Scheidegger. "Nanocubes for Real-Time Exploration of Spatiotemporal Datasets." *IEEE Transactions on Visualization and Computer Graphics*, **19**(12), 2013, pp. 2456–2465. DOI: 10.1109/TVCG.2013.179.

[115] B. T. Lowerre. "The HARPY speech recognition system." PhD thesis. Carnegie Mellon University, Apr. 1976.

[116] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska. "Bao: Making Learned Query Optimization Practical." In: *Proceedings of the International Conference on Management of Data*. SIGMOD '21. 2021. DOI: 10.1145/3448016.3452838.

[117] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul. "Neo: A Learned Query Optimizer." *Proceedings of the VLDB Endowment*, **12**(11), 2019, pp. 1705–1718. DOI: 10.14778/3342263.3342644.

[118] R. Marcus and O. Papaemmanouil. "Deep Reinforcement Learning for Join Order Enumeration." In: *Proceedings of the First International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*. aiDM '18. 2018. DOI: 10.1145/3211954.3211957.

[119] R. Marcus and O. Papaemmanouil. "Plan-Structured Deep Neural Network Models for Query Performance Prediction." *Proceedings of the VLDB Endowment*, **12**(11), 2019, pp. 1733–1746. DOI: 10.14778/3342263.3342646.

[120] P. Menon, A. Ngom, L. Ma, T. C. Mowry, and A. Pavlo. "Permutable Compiled Queries: Dynamically Adapting Compiled Queries without Recompiling." *Proceedings of the VLDB Endowment*, **14**(2), 2020, pp. 101–113. DOI: 10.14778/3425879.3425882.

[121] Microsoft. *Microsoft Fabric Documentation*. 2026. URL: https://learn.microsoft.com/en-us/fabric/ (visited on 08/11/2026).

[122] E. Milkai, Y. Chronis, K. P. Gaffney, Z. Guo, J. M. Patel, and X. Yu. "How Good is My HTAP System?" In: *Proceedings of the 2022 International Conference on Management of Data*. SIGMOD '22. 2022, pp. 1810–1824. DOI: 10.1145/3514221.3526148.

[123] M. G. Misegiannis, D. Ritter, V. Leis, and J. Giceva. "CloudGlide: Deconstructing the Landscape of Cloud-Based Analytics." *Proceedings of the VLDB Endowment*, **18**(13), Sept. 2025, pp. 5638–5651. DOI: 10.14778/3773731.3773739.

[124] H. Mistry, P. Roy, S. Sudarshan, and K. Ramamritham. "Materialized View Selection and Maintenance Using Multi-Query Optimization." In: *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*. SIGMOD '01. 2001, pp. 307–318. DOI: 10.1145/375663.375703.

[125] G. Moerkotte, T. Neumann, and G. Steidl. "Preventing Bad Plans by Bounding the Impact of Cardinality Estimation Errors." *Proceedings of the VLDB Endowment*, **2**(1), Aug. 2009, pp. 982–993. DOI: 10.14778/1687627.1687738.

[126] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica. "Ray: A Distributed Framework for Emerging AI Applications." In: *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation*. OSDI '18. 2018, pp. 561–577. URL: https://www.usenix.org/conference/osdi18/presentation/moritz.

[127] B. Mozafari, R. A. Burcuta, A. Cabrera, A. Constantin, D. Francis, D. Grömling, A. Jindal, M. Konkolowicz, V. Marian Spac, Y. Park, R. R. Carranzo, N. Richardson, A. Roy, A. Srivastava, I. Tarte, B. Westphal, and C. Zhang. "Making Data Clouds Smarter at Keebo: Automated Warehouse Optimization Using Data Learning." In: *Companion of the 2023 International Conference on Management of Data*. SIGMOD '23. 2023, pp. 239–251. DOI: 10.1145/3555041.3589681.

[128] B. Mozafari, C. Curino, A. Jindal, and S. Madden. "Performance and Resource Modeling in Highly-Concurrent OLTP Workloads." In: *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. SIGMOD '13. 2013. DOI: 10.1145/2463676.2467800.

[129] V. Nathan, J. Ding, M. Alizadeh, and T. Kraska. "Learning Multi-Dimensional Indexes." In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. SIGMOD '20. 2020, pp. 985–1000. DOI: 10.1145/3318464.3380579.

[130] P. Negi, Z. Wu, A. Kipf, N. Tatbul, R. Marcus, S. Madden, T. Kraska, and M. Alizadeh. "Robust Query Driven Cardinality Estimation under Changing Workloads." *Proceedings of the VLDB Endowment*, **16**(6), 2023, pp. 1520–1533. DOI: 10.14778/3583140.3583164.

[131] C. G. Nelson. "Techniques for Program Verification." PhD thesis. Stanford University, 1980.

[132] T. Neumann. "Efficiently Compiling Efficient Query Plans for Modern Hardware." *Proceedings of the VLDB Endowment*, **4**(9), 2011, pp. 539–550. DOI: 10.14778/2002938.2002940.

[133] A. L. Ngom and T. Kraska. "Mallet: SQL Dialect Translation with LLM Rule Generation." In: *Proceedings of the Seventh International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*. aiDM '24. 2024. DOI: 10.1145/3663742.3663973.

[134] R. Nieuwenhuis and A. Oliveras. "Proof-Producing Congruence Closure." In: *Term Rewriting and Applications*. Springer Berlin Heidelberg, 2005, pp. 453–468. ISBN: 978-3-540-32033-3. DOI: 10.1007/978-3-540-32033-3_33.

[135] J. Ortiz. "Performance-Based Service Level Agreements for Data Analytics in the Cloud." PhD thesis. University of Washington, 2019.

[136] J. Ortiz, B. Lee, M. Balazinska, J. Gehrke, and J. L. Hellerstein. "SLAOrchestrator: Reducing the Cost of Performance SLAs for Cloud Data Analytics." In: *Proceedings of the 2018 USENIX Annual Technical Conference*. USENIX ATC '18. 2018, pp. 547–560. URL: https://www.usenix.org/conference/atc18/presentation/ortiz.

[137] J. Ortiz, B. Lee, M. Balazinska, and J. L. Hellerstein. *PerfEnforce: A Dynamic Scaling Engine for Analytics with Performance Guarantees*. 2016. arXiv: 1605.09753 [cs.DB].

[138] OtterTune, Inc. *OtterTune | AI Powered Automatic PostgreSQL & MySQL Tuning*. https://web.archive.org/web/20240605143522/https://ottertune.com/. 2024. (Visited on 06/05/2024).

[139] A. Pavlo, G. Angulo, J. Arulraj, H. Lin, J. Lin, L. Ma, P. Menon, T. Mowry, M. Perron, I. Quah, S. Santurkar, A. Tomasic, S. Toor, D. Van Aken, Z. Wang, Y. Wu, R. Xian, and T. Zhang. "Self-Driving Database Management Systems." In: *Proceedings of the 8th Biennial Conference on Innovative Data Systems Research*. CIDR '17. 2017.

[140] A. Pavlo, M. Butrovich, A. Joshi, L. Ma, P. Menon, D. Van Aken, L. Lee, and R. Salakhutdinov. "External vs. Internal: An Essay on Machine Learning Agents for Autonomous Database Management Systems." *IEEE Data Engineering Bulletin*, June 2019, pp. 32–46. URL: https://db.cs.cmu.edu/papers/2019/pavlo-icde-bulletin2019.pdf.

[141] A. Pavlo, M. Butrovich, L. Ma, W. S. Lim, P. Menon, D. Van Aken, and W. Zhang. "Make Your Database System Dream of Electric Sheep: Towards Self-Driving Operation." *Proceedings of the VLDB Endowment*, **14**(12), 2021, pp. 3211–3221. DOI: 10.14778/3476311.3476411.

[142] M. Perron, R. Castro Fernandez, D. DeWitt, M. Cafarella, and S. Madden. "Cackle: Analytical Workload Cost and Performance Stability With Elastic Pools." *Proceedings of the ACM on Management of Data*, **1**(4), Dec. 2023, pp. 1–25. DOI: 10.1145/3626720.

[143] Pgbouncer Authors. *Pgbouncer - Lightweight connection pooler for PostgreSQL*. https://www.pgbouncer.org/. 2026. (Visited on 08/11/2026).

[144] pgvector Authors. *Open-source vector similarity search for Postgres*. https://github.com/pgvector/pgvector. 2026. (Visited on 08/11/2026).

[145] G. Piatetsky-Shapiro. "The Optimal Selection of Secondary Indices is NP-Complete." *SIGMOD Record*, **13**(2), Jan. 1983, pp. 72–75. DOI: 10.1145/984523.984530.

[146] PingCAP. *TiDB: Modern Database Architecture for Real-Time Workloads*. https://www.pingcap.com/tidb/. 2026. (Visited on 08/11/2026).

[147] M. Podkorytov and M. Gubanov. "Hybrid.Poly: A Consolidated Interactive Analytical Polystore System." In: *Proceedings of the 2019 IEEE 35th International Conference on Data Engineering*. ICDE '19. 2019, pp. 1996–1999. DOI: 10.1109/ICDE.2019.00223.

[148] PostgreSQL Global Development Group. *PostgreSQL*. https://www.postgresql.org/. 2026. (Visited on 08/11/2026).

[149] PostgreSQL Global Development Group. *PostgreSQL Extensions*. https://www.postgresql.org/docs/current/external-extensions.html. 2026. (Visited on 08/11/2026).

[150] C. Pu. “Superdatabases for Composition of Heterogeneous Databases.” In: *Proceedings of the Fourth International Conference on Data Engineering*. ICDE ’88. 1988, pp. 548–555. DOI: 10.1109/ICDE.1988.105502.

[151] B. Răducanu, P. Boncz, and M. Zukowski. “Micro Adaptivity in Vectorwise.” In: *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. SIGMOD ’13. 2013, pp. 1231–1242. DOI: 10.1145/2463676.2465292.

[152] A. van Renen, D. Horn, P. Pfeil, K. Vaidya, W. Dong, M. Narayanaswamy, Z. Liu, G. Saxena, A. Kipf, and T. Kraska. “Why TPC is Not Enough: An Analysis of the Amazon Redshift Fleet.” *Proceedings of the VLDB Endowment*, **17**(11), July 2024, pp. 3694–3706. ISSN: 2150-8097. DOI: 10.14778/3681954.3682031.

[153] A. van Renen and V. Leis. “Cloud Analytics Benchmark.” *Proceedings of the VLDB Endowment*, **16**(6), Feb. 2023, pp. 1413–1425. DOI: 10.14778/3583140.3583156.

[154] M. Rieger and T. Neumann. “T3: Accurate and Fast Performance Prediction for Relational Database Systems With Compiled Decision Trees.” *Proceedings of the ACM on Management of Data*, **3**(3), June 2025. DOI: 10.1145/3725364.

[155] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis. “INFaaS: Automated Model-less Inference Serving.” In: *Proceedings of the 2021 USENIX Annual Technical Conference*. USENIX ATC ’21. July 2021, pp. 397–411. URL: https://www.usenix.org/conference/atc21/presentation/romero.

[156] M. Samuel. “Hydroflow: A Model and Runtime for Distributed Systems Programming.” MA thesis. EECS Department, University of California, Berkeley, Aug. 2021. URL: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2021/EECS-2021-201.html.

[157] G. Saxena, M. Rahman, N. Chainani, C. Lin, G. Caragea, F. Chowdhury, R. Marcus, T. Kraska, I. Pandis, and B. M. Narayanaswamy. “Auto-WLM: Machine Learning Enhanced Workload Management in Amazon Redshift.” In: *Companion of the 2023 International Conference on Management of Data*. SIGMOD ’23. 2023, pp. 225–237. DOI: 10.1145/3555041.3589677.

[158] T. Schmidt, A. Kipf, D. Horn, G. Saxena, and T. Kraska. “Predicate Caching: Query-Driven Secondary Indexing for Cloud Data Warehouses.” In: *Companion of the 2024 International Conference on Management of Data*. SIGMOD ’24. Association for Computing Machinery, June 2024, pp. 347–359. DOI: 10.1145/3626246.3653395.

[159] T. Schmidt, V. Leis, P. Boncz, and T. Neumann. “SQLStorm: Taking Database Benchmarking into the LLM Era.” *Proceedings of the VLDB Endowment*, **18**(11), 2025, pp. 4144–4157. DOI: 10.14778/3749646.3749683.

[160] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. “Access Path Selection in a Relational Database Management System.” In: *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. SIGMOD ’79. 1979, pp. 23–34. DOI: 10.1145/582095.582099.

[161] A. P. Sheth and J. A. Larson. “Federated Database Systems for Managing Distributed, Heterogeneous, and Autonomous Databases.” *ACM Computing Surveys*, **22**(3), 1990, pp. 183–236. DOI: 10.1145/96602.96604.

[162] A. Shukla, P. Deshpande, and J. F. Naughton. “Materialized View Selection for Multidimensional Datasets.” In: *Proceedings of the 24th International Conference on Very Large Data Bases*. VLDB ’98. 1998, pp. 488–499. URL: https://www.vldb.org/conf/1998/p488.pdf.

[163] M. Sichert and T. Neumann. “User-Defined Operators: Efficiently Integrating Custom Algorithms into Modern Databases.” *Proceedings of the VLDB Endowment*, **15**(5), Jan. 2022, pp. 1119–1131. DOI: 10.14778/3510397.3510408.

[164] V. Sikka, F. Färber, W. Lehner, S. K. Cha, T. Peh, and C. Bornhövd. “Efficient Transaction Processing in SAP HANA Database: The End of a Column Store Myth.” In: *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. SIGMOD ’12. 2012, pp. 731–742. DOI: 10.1145/2213836.2213946.

[165] M. Sipser. *Introduction to the Theory of Computation*. Cengage, 2021.

[166] Snowflake, Inc. *ETL vs. ELT: Differences and Similarities*. https://www.snowflake.com/guides/etl-vs-elt. 2026. (Visited on 08/11/2026).

[167] Snowflake, Inc. *Set the target lag for a dynamic table*. https://docs.snowflake.com/en/user-guide/dynamic-tables/target-lag. 2026. (Visited on 08/11/2026).

[168] H. Song, W. Zhou, H. Cui, X. Peng, and F. Li. “A survey on hybrid transactional and analytical processing.” *The VLDB Journal*, **33**, 2024, pp. 1485–1515. DOI: 10.1007/s00778-024-00858-9.

[169] L. Spiegelberg, R. Yesantharao, M. Schwarzkopf, and T. Kraska. “Tuplex: Data Science in Python at Native Code Speed.” In: *Proceedings of the 2021 International Conference on Management of Data*. SIGMOD ’21. 2021, pp. 1718–1731. DOI: 10.1145/3448016.3457244.

[170] StackOverflow. *StackOverflow 220 GB dataset*. https://db.in.tum.de/~schmidt/data/stackoverflow.tar.gz. 2026. (Visited on 08/11/2026).

[171] M. Stoian, A. Zimmerer, S. Krid, A. L. Ngom, J. Ding, T. Kraska, and A. Kipf. “Parachute: Single-Pass Bi-Directional Information Passing.” *Proceedings of the VLDB Endowment*, **18**(10), June 2025, pp. 3299–3311. ISSN: 2150-8097. DOI: 10.14778/3748191.3748196.

[172] S. Sudhir, M. Cafarella, and S. Madden. “Replicated Layout for In-Memory Database Systems.” *Proceedings of the VLDB Endowment*, **15**(4), Dec. 2021, pp. 984–997. DOI: 10.14778/3503585.3503606.

[173] J. Sun and G. Li. “An End-to-End Learning-based Cost Estimator.” *Proceedings of the VLDB Endowment*, **13**(3), 2019, pp. 307–319. DOI: 10.14778/3368289.3368296.

[174] K. Tangwongsan, M. Hirzel, S. Schneider, and K.-L. Wu. "General Incremental Sliding-Window Aggregation." *Proceedings of the VLDB Endowment*, **8**(7), 2015, pp. 702–713. DOI: 10.14778/2752939.2752940.

[175] R. Tarjan. "Depth-First Search and Linear Graph Algorithms." *SIAM Journal on Computing*, **1**(2), 1972, pp. 146–160. DOI: 10.1137/0201010.

[176] R. Tate, M. Stepp, Z. Tatlock, and S. Lerner. "Equality Saturation: A New Approach to Optimization." *ACM SIGPLAN Notices*, **44**(1), Jan. 2009, pp. 264–276. DOI: 10.1145/1594834.1480915.

[177] J. W. Thatcher and J. B. Wright. "Generalized Finite Automata Theory with an Application to a Decision Problem of Second-Order Logic." *Mathematical Systems Theory*, **2**(1), 1968, pp. 57–81. DOI: 10.1007/BF01691346.

[178] Transaction Processing Performance Council. *TPC-H Benchmark*. https://www.tpc.org/TPC_Documents_Current_Versions/pdf/TPC-H_v3.0.1.pdf. 2026. (Visited on 08/11/2026).

[179] Transaction Processing Performance Council (TPC). *TPC-C*. https://www.tpc.org/tpcc/. 2026. (Visited on 08/11/2026).

[180] Transaction Processing Performance Council (TPC). *TPC-DS*. https://www.tpc.org/tpcds/default5.asp. 2026. (Visited on 08/11/2026).

[181] I. Trummer. "Demonstrating GPT-DB: Generating Query-Specific and Customizable Code for SQL Processing with GPT-4." *Proceedings of the VLDB Endowment*, **16**(12), Aug. 2023, pp. 4098–4101. DOI: 10.14778/3611540.3611630.

[182] University of California, Berkeley. *Sky Computing*. 2026. URL: https://sky.cs.berkeley.edu/ (visited on 08/11/2026).

[183] D. Van Aken, A. Pavlo, G. J. Gordon, and B. Zhang. "Automatic Database Management System Tuning Through Large-Scale Machine Learning." In: *Proceedings of the 2017 ACM International Conference on Management of Data*. SIGMOD '17. 2017, pp. 1009–1024. DOI: 10.1145/3035918.3064029.

[184] S. Venkataraman, Z. Yang, M. Franklin, B. Recht, and I. Stoica. "Ernest: Efficient Performance Prediction for Large-Scale Advanced Analytics." In: *Proceedings of the 13th USENIX Symposium on Networked Systems Design and Implementation*. NSDI '16. 2016, pp. 363–378. URL: https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/venkataraman.

[185] M. Vogt, A. Stiemer, and H. Schuldt. "Polypheny-DB: Towards a Distributed and Self-Adaptive Polystore." In: *Proceedings of the 2018 IEEE International Conference on Big Data*. IEEE Big Data '18. 2018, pp. 3364–3373. DOI: 10.1109/BigData.2018.8622353.

[186] M. Vuppalapati, J. Miron, R. Agarwal, D. Truong, A. Motivala, and T. Cruanes. "Building An Elastic Query Engine on Disaggregated Storage." In: *17th USENIX Symposium on Networked Systems Design and Implementation*. NSDI '20. 2020, pp. 449–462. URL: https://www.usenix.org/conference/nsdi20/presentation/vuppalapati.

[187] B. Wagner, A. Kohn, and T. Neumann. "Self-Tuning Query Scheduling for Analytical Workloads." In: *Proceedings of the 2021 International Conference on Management of Data*. SIGMOD '21. 2021, pp. 1879–1891. DOI: 10.1145/3448016.3457260.

[188] J. Wang, T. Baker, M. Balazinska, D. Halperin, B. Haynes, B. Howe, D. Hutchison, S. Jain, R. Maas, P. Mehta, D. Moritz, B. Myers, J. Ortiz, D. Suciu, A. Whitaker, and S. Xu. "The Myria Big Data Management and Analytics System and Cloud Services." In: *Proceedings of the Conference on Innovative Data Systems Research*. CIDR '17. 2017.

[189] J. Wehrstein, T. Bang, R. Heinrich, and C. Binnig. "GRACEFUL: A Learned Cost Estimator For UDFs." In: *Proceedings of the 2025 International Conference on Data Engineering*. ICDE '25. 2025. DOI: 10.1109/ICDE65448.2025.00185.

[190] J. Wehrstein, T. Eckmann, M. Jasny, and C. Binnig. *Bespoke OLAP: Synthesizing Workload-Specific One-size-fits-one Database Engines*. 2026. arXiv: 2603.02001 [cs.DB].

[191] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panchekha. "egg: Fast and Extensible Equality Saturation." *Proceedings of the ACM on Programming Languages*, **5**(POPL), Jan. 2021. DOI: 10.1145/3434304.

[192] W. Wu, Y. Chi, H. Hacıgümüş, and J. F. Naughton. "Towards Predicting Query Execution Time for Concurrent and Dynamic Database Workloads." *Proceedings of the VLDB Endowment*, **6**(10), 2013, pp. 925–936. ISSN: 2150-8097. DOI: 10.14778/2536206.2536219.

[193] W. Wu, H. Hacıgümüş, Y. Chi, S. Zhu, J. Tatemura, and J. F. Naughton. "Predicting Query Execution Time: Are Optimizer Cost Models Really Unusable?" In: *Proceedings of the 2013 IEEE International Conference on Data Engineering*. ICDE '13. 2013, pp. 1081–1092. DOI: 10.1109/ICDE.2013.6544899.

[194] Y. Wu, X. Zhou, Y. Zhang, and G. Li. "Automatic Index Tuning: A Survey." *IEEE Transactions on Knowledge and Data Engineering*, **36**(12), Dec. 2024, pp. 7657–7676. DOI: 10.1109/TKDE.2024.3422006.

[195] Z. Wu, R. Marcus, Z. Liu, P. Negi, V. Nathan, P. Pfeil, G. Saxena, M. Rahman, B. Narayanaswamy, and T. Kraska. "Stage: Query Execution Time Prediction in Amazon Redshift." In: *Companion of the 2024 International Conference on Management of Data*. SIGMOD '24. 2024, pp. 280–294. DOI: 10.1145/3626246.3653391.

[196] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, and S. Madden. "FactorJoin: A New Cardinality Estimation Framework for Join Queries." *Proceedings of the ACM on Management of Data*, **1**(1), 2023. DOI: 10.1145/3588721.

[197] Z. Wu, P. Yu, P. Yang, R. Zhu, Y. Han, Y. Li, D. Lian, K. Zeng, and J. Zhou. "A Unified Transferable Model for ML-Enhanced DBMS." In: *Proceedings of the 12th Conference on Innovative Data Systems Research*. CIDR '22. 2022. URL: https://www.cidrdb.org/cidr2022/papers/p6-wu.pdf.

[198] J. Yang, I. Rae, J. Xu, J. Shute, Z. Yuan, K. Lau, Q. Zeng, X. Zhao, J. Ma, Z. Chen, Y. Gao, Q. Dong, J. Zhou, J. Wood, G. Graefe, J. Naughton, and J. Cieslewicz. "F1 Lightning: HTAP as a Service." *Proceedings of the VLDB Endowment*, **13**(12), 2020, pp. 3313–3325. DOI: 10.14778/3415478.3415553.

[199] G. X. Yu, M. Markakis, A. Kipf, P.-Å. Larson, U. F. Minhas, and T. Kraska. "TreeLine: An Update-In-Place Key-Value Store for Modern Storage." *Proceedings of the VLDB Endowment*, **16**(1), 2022, pp. 99–112. DOI: 10.14778/3561261.3561270.

[200] G. X. Yu, Z. Wu, F. Kossmann, T. Li, M. Markakis, A. Ngom, S. Madden, and T. Kraska. "Blueprinting the Cloud: Unifying and Automatically Optimizing Cloud Data Infrastructures with BRAD." *Proceedings of the VLDB Endowment*, **17**(11), Aug. 2024, pp. 3629–3643. DOI: 10.14778/3681954.3682026.

[201] G. X. Yu, Z. Wu, F. Kossmann, T. Li, M. Markakis, A. Ngom, S. Zhang, T. Kraska, and S. Madden. "Virtualizing Cloud Data Infrastructures with BRAD." In: *Companion of the 2025 International Conference on Management of Data*. SIGMOD '25. 2025, pp. 271–274. DOI: 10.1145/3722212.3725141.

[202] X. Yu, G. Li, C. Chai, and N. Tang. "Reinforcement Learning with Tree-LSTM for Join Order Selection." In: *Proceedings of the 2020 IEEE 36th International Conference on Data Engineering*. ICDE '20. 2020, pp. 1297–1308. DOI: 10.1109/ICDE48307.2020.00116.

[203] J. Zhang, K. Huang, T. Wang, and K. Lv. "Skeena: Efficient and Consistent Cross-Engine Transactions." In: *Proceedings of the 2022 International Conference on Management of Data*. SIGMOD '22. 2022, pp. 34–48. DOI: 10.1145/3514221.3526171.

[204] X. Zheng, S. Dasgupta, A. Kumar, and A. Gupta. *AWESOME: Empowering Scalable Data Science on Social Media Data with an Optimized Tri-Store Data System*. 2022. arXiv: 2112.00833 [cs.DB].

[205] J. Zhou, P.-Å. Larson, and H. G. Elmongui. "Lazy Maintenance of Materialized Views." In: *Proceedings of the 33rd International Conference on Very Large Data Bases*. VLDB '07. 2007, pp. 231–242. URL: https://www.vldb.org/conf/2007/papers/research/p231-zhou.pdf.

[206] R. Zhu, W. Chen, B. Ding, X. Chen, A. Pfadler, Z. Wu, and J. Zhou. "Lero: A Learning-to-Rank Query Optimizer." *Proceedings of the VLDB Endowment*, **16**(6), 2023, pp. 1466–1479. DOI: 10.14778/3583140.3583160.